

comparing_3_datasets

March 29, 2025

```
[1]: import sys
from IPython.display import display, Javascript

def restart_kernel():
    """Restart the Jupyter Notebook kernel to reflect changes in modules and_
    ↪packages."""
    display(Javascript("Jupyter.notebook.kernel.restart()"))
    print("Kernel is restarting...")

restart_kernel()

import bptf
from bptf import BPTF
import numpy as np
import pandas as pd
import sparse
import os
import shutil
from tqdm import tqdm
import pickle
import scipy.stats as st
import matplotlib.pyplot as plt
import torch
import tensorly
import cupy

import multiprocessing
from joblib import Parallel, delayed
from tqdm.contrib.concurrent import process_map

import gc
gc.collect()
```

<IPython.core.display.Javascript object>

Kernel is restarting...

[1]: 20

```
[2]: os.getcwd()
      # tensorly.set_backend('pytorch')
```

```
[2]: 'c:\\Users\\luiyu\\Documents\\aaron_schein\\bptf_new'
```

1 Get reference label tables

```
[3]: parallel = False
if parallel:
    print(multiprocessing.cpu_count())
end_year = 18 # 24 for 2024 and 18 for 2018
end_year += 1
print([str(2000 + x) for x in range(0, end_year)])

folder = "/icews_gdelt_terrier/"
data_filepath = os.getcwd() + folder
files = os.listdir(data_filepath)
if parallel:
    files = Parallel(n_jobs=multiprocessing.cpu_count())(
        delayed(pd.read_csv)(data_filepath + filepath)
        for filepath in tqdm(files, desc = 'Reading data')
    )
    data = pd.concat(files)
    data = data.sort_values(by='Num_Events', ascending=False)
else:
    data = [pd.read_csv(data_filepath + filepath) for filepath in tqdm(files,
        desc = 'Reading data')]
    data = pd.concat(data)
    data = data.sort_values(by='Num_Events', ascending=False)

gdelt_filepaths = [os.path.join('GDELT', f) for f in os.listdir('GDELT')]

def read_gdelt(filepath):
    gdelt = pd.read_csv(filepath, dtype={'EventBaseCode': str})
    colnames = data.columns
    gdelt = gdelt.rename(columns={'Actor1CountryCode' : colnames[0],
                                'Actor2CountryCode' : colnames[1],
                                'EventBaseCode' : colnames[2],
                                'SQLDATE' : colnames[3],
                                'NumMentions' : colnames[4]})
    gdelt = gdelt[gdelt['CAMEO_Code'].str.isnumeric()]
    gdelt['CAMEO_Code'] = gdelt['CAMEO_Code'].astype(int)
    gdelt['formatteddate'] = pd.to_datetime(gdelt['formatteddate'],
        format='%Y%m%d')
    gdelt['formatteddate'] = gdelt['formatteddate'].dt.strftime('%Y-%m-%d')
    return gdelt
```

```

if parallel:
    if __name__ == '__main__':
        gdelt = Parallel(n_jobs = multiprocessing.cpu_count())(
            delayed(read_gdelt)(gdelt_filepath)
            for gdelt_filepath in tqdm(gdelt_filepaths, desc='Reading GDELT_
↳data')
        )
    else:
        gdelt = [read_gdelt(gdelt_filepath) for gdelt_filepath in_
↳tqdm(gdelt_filepaths, desc='Reading GDELT data')]

gdelt = pd.concat(gdelt)

gdelt['Database'] = 'GDELT'
data = pd.concat([data, gdelt])
print(data.head())
print(data.tail())
del gdelt
gc.collect()

years = [str(2000 + x) for x in range(0, 25)]
data = data[data['formatteddate'].str.startswith(tuple(years))]

data = data.dropna()

# collapse by month
data['formatteddate'] = data['formatteddate'].str[:7]
data = data.groupby(['Source_Country_Code', 'Target_Country_Code',_
↳'CAMEO_Code', 'formatteddate', 'Database'])
data = data.sum().reset_index()

# Make the list of countries, actions, times and databases_
↳=====
# get unique list of countries
countries = pd.concat([data['Source_Country_Code'],_
↳data['Target_Country_Code']])
countries = pd.unique(countries)
country_indices = pd.DataFrame({
    'country' : countries,
    'index' : range(len(countries))
})
del countries

# there's no need to get a list of actions since it's just 1 to 20

# get unique list of dates

```

```

# Make sure to change the date format and frequency of the date range if you
↳change the time unit
date_indices = pd.date_range(
    start=pd.to_datetime(data['formatteddate'], format='%Y-%m').min(),
    ↳strftime('%Y-%m'),
    end=pd.to_datetime(data['formatteddate'], format='%Y-%m').max(),
    ↳strftime('%Y-%m'),
    freq='MS')
date_indices = date_indices.strftime('%Y-%m').to_list()
date_indices = pd.DataFrame({
    'date' : date_indices,
    'index' : range(len(date_indices))
})
date_indices['date'] = date_indices['date'].str[:7]

# get unique list of databases
databases = pd.unique(data['Database'])
databases = pd.unique(databases)
database_indices = pd.DataFrame({
    'database' : databases,
    'index' : range(len(databases))
})
print(database_indices)
del databases

```

```

['2000', '2001', '2002', '2003', '2004', '2005', '2006', '2007', '2008', '2009',
'2010', '2011', '2012', '2013', '2014', '2015', '2016', '2017', '2018']

```

Reading data: 100%| | 103/103 [00:25<00:00, 4.08it/s]

Reading GDELT data: 100%| | 21/21 [00:15<00:00, 1.37it/s]

	Source_Country_Code	Target_Country_Code	CAMEO_Code	formatteddate	\
260543		NaN	IND	7	2009-10-05
261063		IND	IND	7	2009-10-05
814		NaN	NaN	1	2014-03-08
9481		NaN	NaN	4	2014-03-08
3724		NaN	NaN	1	2009-11-16

	Num_Events	Database
260543	11323	TERRIER
261063	11323	TERRIER
814	3269	TERRIER
9481	3149	TERRIER
3724	2625	TERRIER

	Source_Country_Code	Target_Country_Code	CAMEO_Code	formatteddate	\
62555	YEM	SYR	4	2020-01-08	
62556	YEM	SYR	4	2020-01-08	
62557	YEM	SYR	4	2020-01-08	

62558	ZWE	ZWE	4	2020-01-08
62559	ZWE	ZWE	4	2020-01-08

	Num_Events	Database
62555	2	GDELT
62556	2	GDELT
62557	6	GDELT
62558	4	GDELT
62559	4	GDELT

	database	index
0	ICEWS	0
1	GDELT	1
2	TERRIER	2

```
[5]: nan_rows = country_indices[country_indices['country'].isnull()]
      print(nan_rows)
```

```
Empty DataFrame
Columns: [country, index]
Index: []
```

2 Get tensor

```
[6]: with open('sptensor.pkl', 'rb') as f:
      Y = pickle.load(f)
      # need to enforce self-country actions = 0
      mask = Y.coords[0] != Y.coords[1]
      new_coords = Y.coords[:, mask].copy()
      new_data = Y.data[mask].copy()
      Y = sparse.COO(new_coords, new_data, shape=Y.shape)
      if end_year == 19:
          Y = Y[:, :, :, :(12*(2019-2000)), :].copy()
```

3 Tensor factorisation

```
[7]: # decompose the tensor using Poisson CP decomposition
      gc.collect()
      n_components = 100

      bptf_filepath = f'bptf_5mode_50iter_2000_{2000 + end_year - 1}.pkl'

      if os.path.exists(bptf_filepath):
          with open(bptf_filepath, 'rb') as f:
              bptf_5mode = pickle.load(f)
      else:
          print(f'{bptf_filepath} does not exist!')
```

4 Checks

```
[8]: # check the shapes
for j in range(len(Y.shape)):
    assert bptf_5mode.G_DK_M[j].shape == (Y.shape[j], n_components)
```

5 Reconstruction

```
[9]: Y_hat = bptf_5mode.reconstruct()
# Y_original = Y.todense()

# print(Y_hat[:5, :5, 0, 0, 0]); print(Y[:5, :5, 0, 0, 0].todense())
# print(Y[:5, :5, 0, 0, 0].todense() - Y_hat[:5, :5, 0, 0, 0])
```

6 Compare with Tensorly's CP decomposition

6.0.1 2-norm error

```
[10]: # if os.path.exists('nntf_parafac_5mode.pkl'):
#     with open('nntf_parafac_5mode.pkl', 'rb') as f:
#         tensor_mu = pickle.load(f)
# else:
#     cp_init = tensorly.cp_tensor.CPTensor(
#         tensorly.decomposition._cp.initialize_cp(
#             Y, non_negative = True, init = 'random', rank = n_components
#         )
#     )
#     tensor_mu, _ = tensorly.decomposition.non_negative_parafac(
#         Y, rank=n_components, init=cp_init, return_errors=True
#     )

#     with open('nntf_parafac_5mode.pkl', 'wb') as f:
#         pickle.dump(tensor_mu, f)

[11]: # print(torch.norm(torch.from_numpy(Y.todense()) - torch.from_numpy(tensorly.
#     ↪cp_to_tensor(tensor_mu)), 'fro') /
#     torch.norm(torch.from_numpy(Y.todense()), 'fro'))
# print(torch.norm(torch.from_numpy(Y.todense()) - bptf_5mode.
#     ↪reconstruct(drop_diag=True, style='geometric', n_samples=100), 'fro') /
#     torch.norm(torch.from_numpy(Y.todense()), 'fro'))
# it's pretty close to tensorly's implementation when using frobenius norm
# ↪forward error
```

6.0.2 Comparing non-zero element prediction accuracy and difference

```
[12]: # nonzero_indices = np.vstack(sparse.nonzero(Y)).T.tolist()
# nonzero_indices = [tuple(ele) for ele in nonzero_indices]

# largest_indices = np.argpartition(Y_hat.ravel(), -Y.nnz)[-Y.nnz:]
# largest_indices = np.unravel_index(largest_indices, Y_hat.shape)
# largest_indices = np.vstack(largest_indices).T.tolist()
# largest_indices = [tuple(ele) for ele in largest_indices]
# num_correct_predictions_BPTF = len(set(nonzero_indices) &
    ↪ set(largest_indices))
# print(f'Proportion of nonzero elements correctly predicted by BPTF:
    ↪ {num_correct_predictions_BPTF / Y.nnz}')
# rel_error_frob = torch.norm(torch.from_numpy(Y.todense()) - bptf_5mode.
    ↪ reconstruct(drop_diag=True, style='geometric', n_samples=100), 'fro') /
    ↪ torch.norm(torch.from_numpy(Y.todense()), 'fro')
# print(f"Frobenius norm relative error: {rel_error_frob}")

[13]: # largest_indices = np.argpartition(tensorly.cp_to_tensor(tensor_mu).ravel(),
    ↪ -Y.nnz)[-Y.nnz:]
# largest_indices = np.unravel_index(largest_indices, tensorly.
    ↪ cp_to_tensor(tensor_mu).shape)
# largest_indices = np.vstack(largest_indices).T.tolist()
# largest_indices = [tuple(ele) for ele in largest_indices]
# num_correct_predictions_NNCP = len(set(nonzero_indices) &
    ↪ set(largest_indices))

[14]: # print(f'Proportion of nonzero elements correctly predicted by BPTF:
    ↪ {num_correct_predictions_BPTF / Y.nnz}')
# print(f'Proportion of nonzero elements correctly predicted by tensorly
    ↪ nonnegative CP decomposition: {num_correct_predictions_NNCP / Y.nnz}')
```

7 Plot components

```
[15]: database_factor_matrix = bptf_5mode.G_DK_M[4]
database_factor_matrix = database_factor_matrix/database_factor_matrix.
    ↪ sum(axis=0, keepdims=1)
database_entropy = st.entropy(database_factor_matrix, axis=0)
database_components = pd.DataFrame({
    'ICEWS' : database_factor_matrix[0, :],
    'TERRIER' : database_factor_matrix[1, :],
    'GDELT' : database_factor_matrix[2, :],
    'entropy' : database_entropy,
    'index' : range(n_components)
}).sort_values(by='entropy', ascending=False)
```

```

def component_analysis_plot(component):

    fig = plt.figure(figsize=(20, 18))
    gs = fig.add_gridspec(3, 2, height_ratios=[1, 1, 1])

    ax1 = fig.add_subplot(gs[0, :])
    time_vector = bptf_5mode.G_DK_M[3][:, component]
    time_vector = pd.DataFrame({
        'time steps' : time_vector,
        'index' : range(time_vector.shape[0])
    })
    time_vector = pd.merge(
        left=time_vector, right=date_indices,
        how='left', on='index'
    )
    ax1.plot(time_vector['date'], time_vector['time steps'], color='b')
    ticks = ax1.get_xticks()[::12]
    ax1.set_xticks(ticks)
    ax1.tick_params(axis='x', rotation=90)
    ax1.set_title('Time steps')

    ax2 = fig.add_subplot(gs[1, 0])
    sender_vector = bptf_5mode.G_DK_M[0][:, component]
    sender_vector = pd.DataFrame({
        'sender' : sender_vector,
        'index' : range(sender_vector.shape[0])})
    sender_vector = pd.merge(
        left=sender_vector, right=country_indices,
        how='left', on='index'
    ).sort_values(
        by='sender',
        ascending=False).iloc[:10, :]
    sender_vector['country'] = sender_vector['country'].astype(str)
    ax2.vlines(x=sender_vector['country'], ymin=0,
    ymax=sender_vector['sender'], color='r')
    ax2.set_title('Sender')

    ax3 = fig.add_subplot(gs[1, 1])
    receiver_vector = bptf_5mode.G_DK_M[1][:, component]
    receiver_vector = pd.DataFrame({
        'receiver' : receiver_vector,
        'index' : range(receiver_vector.shape[0])})
    receiver_vector = pd.merge(
        left=receiver_vector, right=country_indices,
        how='left', on='index'
    ).sort_values(
        by='receiver', ascending=False

```



```

).iloc[:10, :]
ax3.vlines(x=receiver_vector['country'], ymin=0,
↪ymax=receiver_vector['receiver'], color='g')
ax3.set_title('Receiver')

ax4 = fig.add_subplot(gs[2, 0])
action_vector = bptf_5mode.G_DK_M[2][:, component]
action_vector = pd.DataFrame({
    'action' : action_vector,
    'action type' : range(1, 21)
})
ax4.vlines(x=action_vector['action type'], ymin=0,
↪ymax=action_vector['action'], color='black')
ax4.set_title('Action types')

ax5 = fig.add_subplot(gs[2, 1])
database_vector = bptf_5mode.G_DK_M[4][:, component]
database_vector = pd.DataFrame({
    'factor' : database_vector,
    'index' : range(database_vector.shape[0])
})
database_vector = pd.merge(
    left=database_vector, right=database_indices,
    how='left', on='index'
)
ax5.vlines(x=database_vector['database'], ymin=0,
↪ymax=database_vector['factor'], color='purple')
ax5.set_title('Database factors')

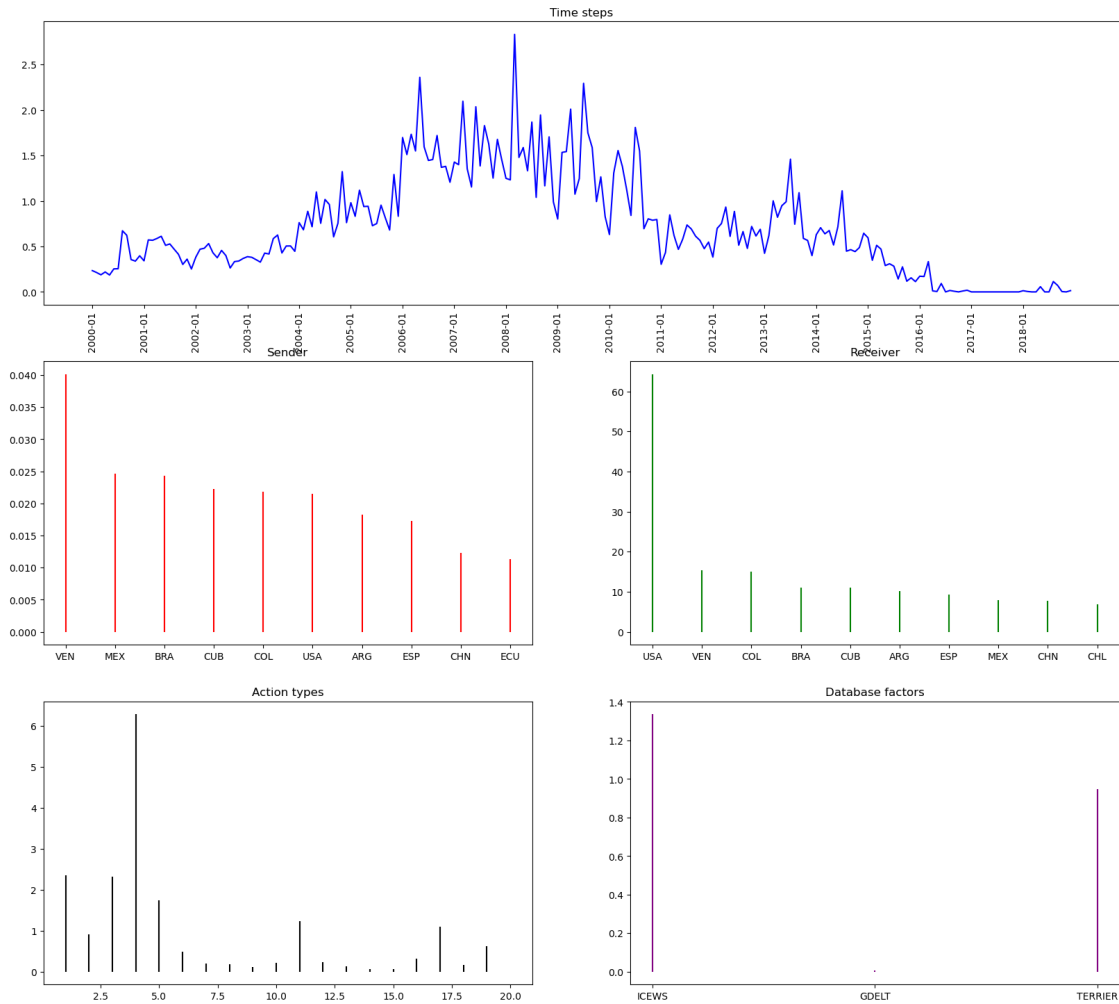
fig.suptitle('Component ' + str(component))

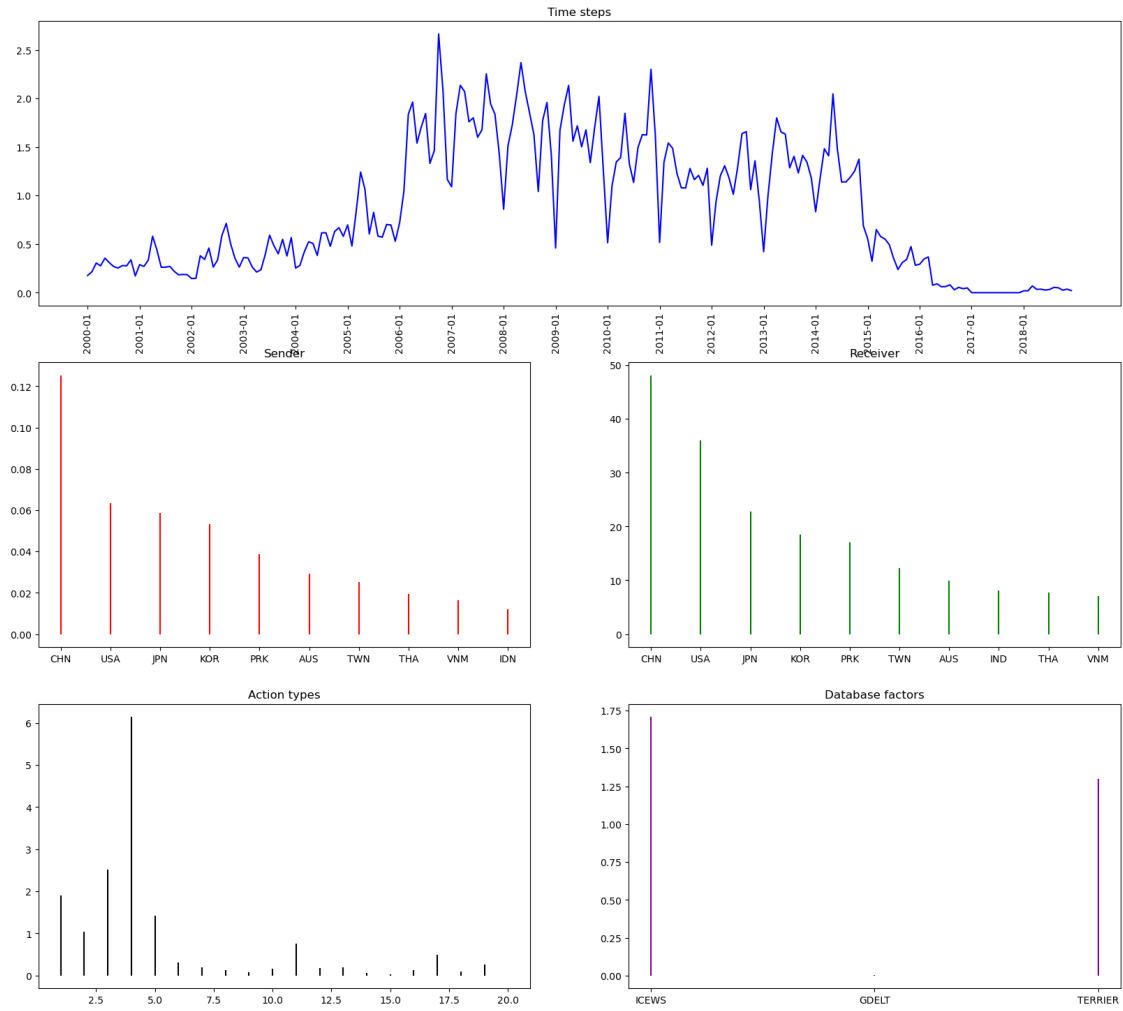
plt.show()

for component in database_components['index']:
    component_analysis_plot(component)

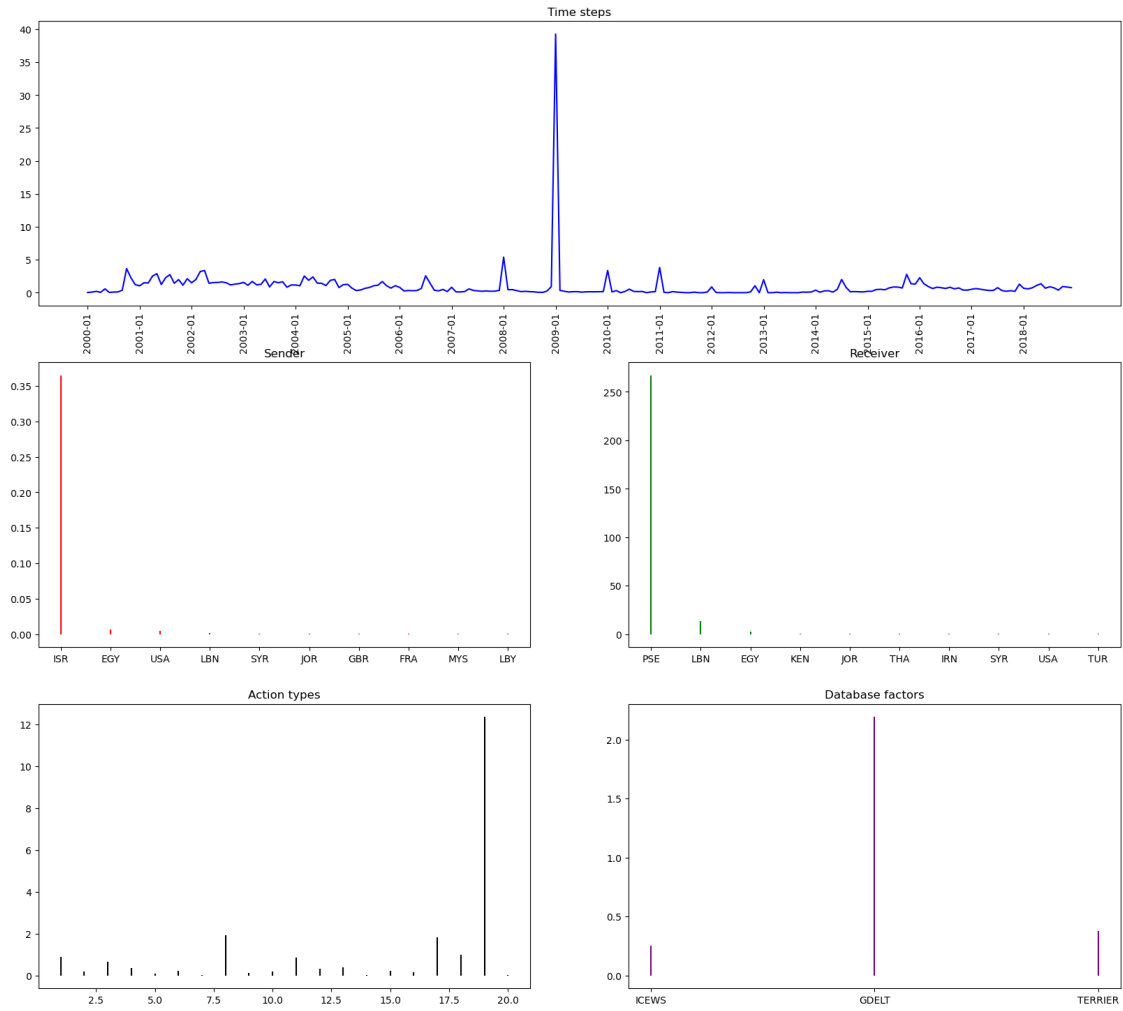
```

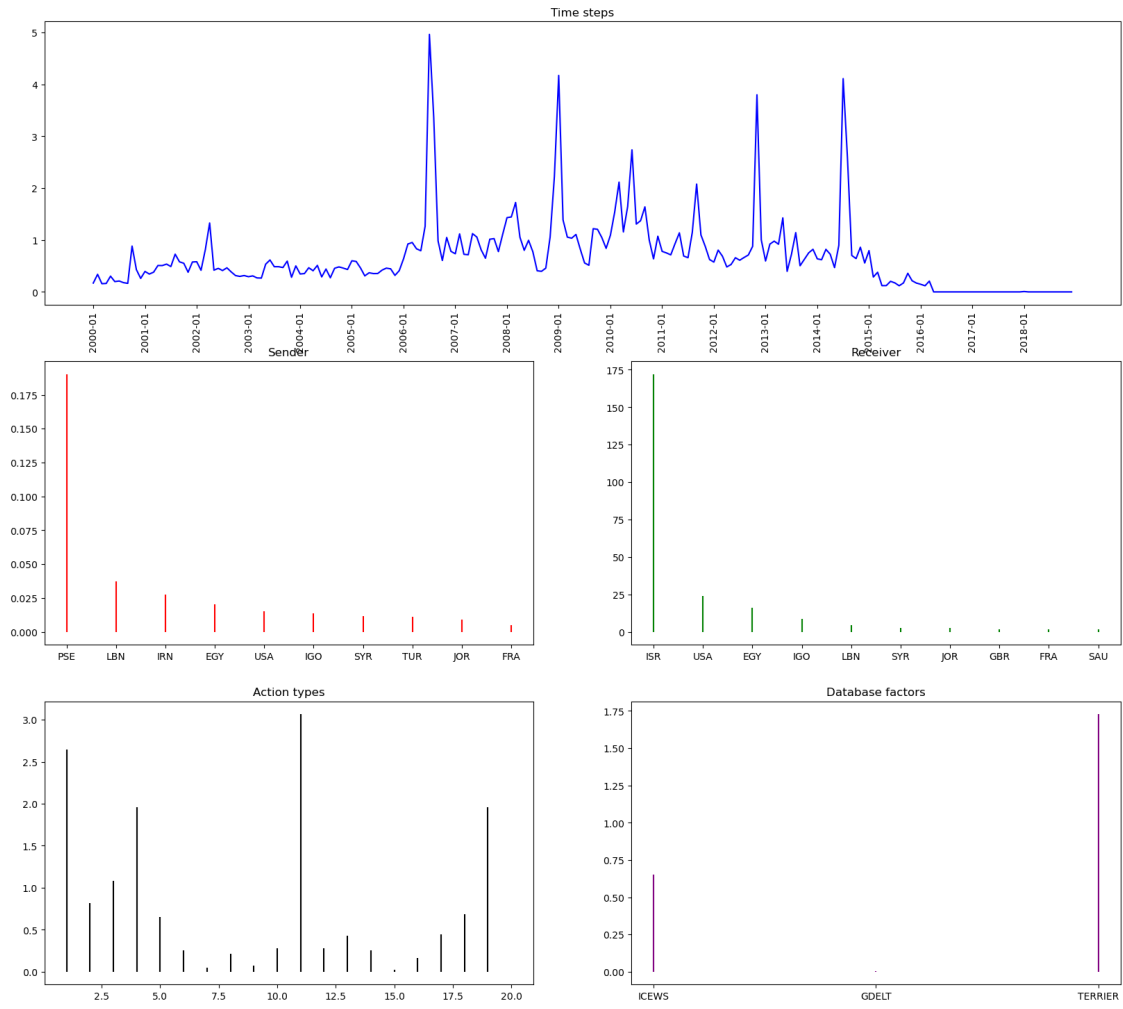
Component 41



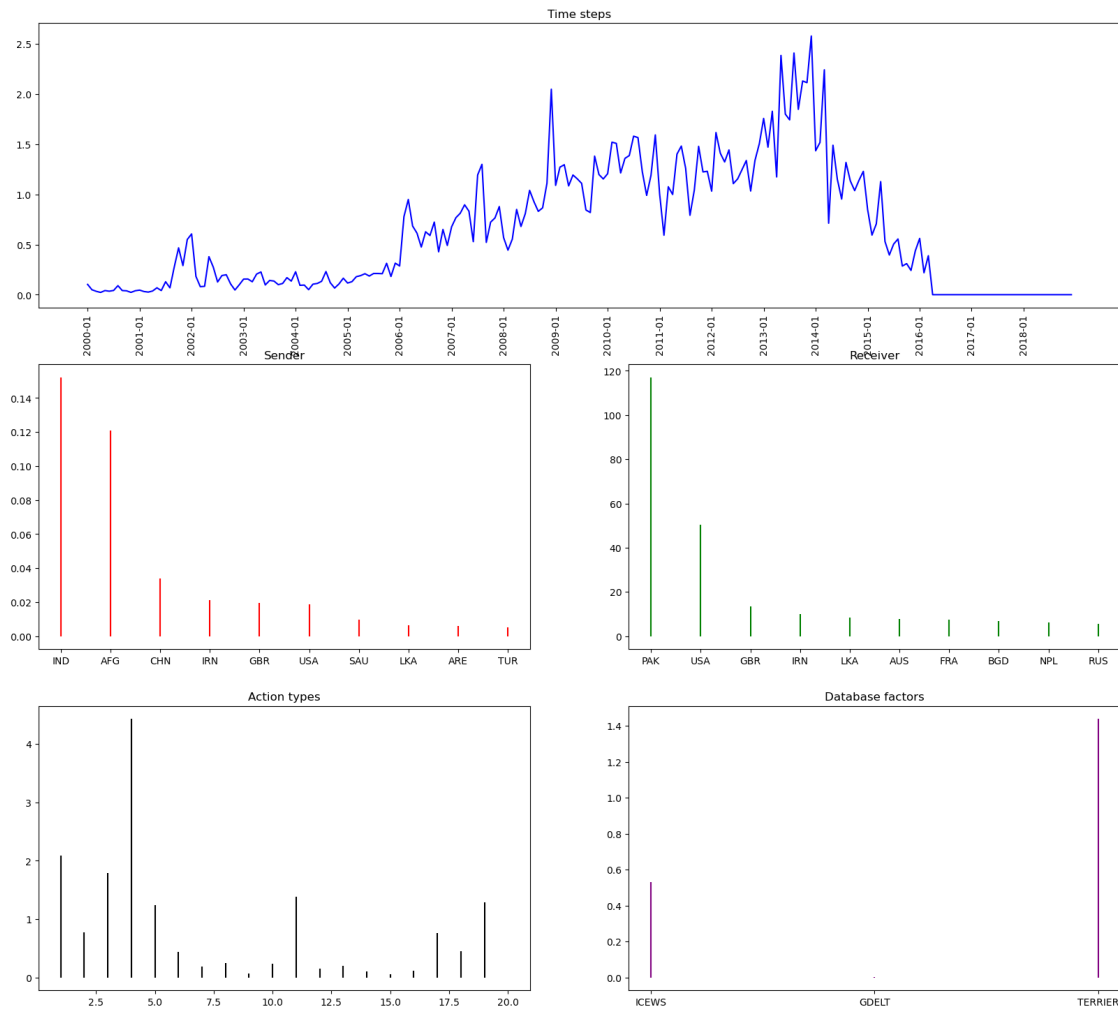


Component 57





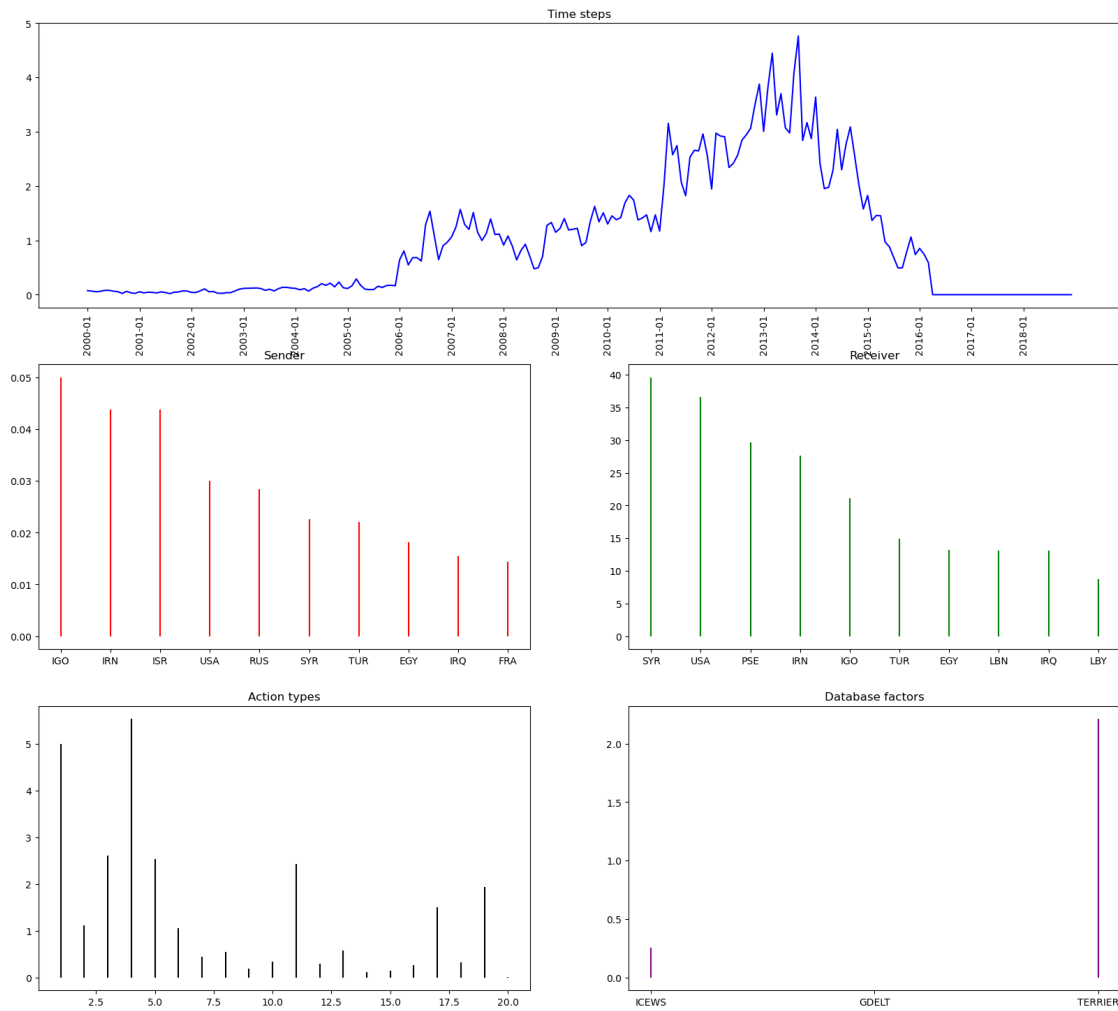
Component 13

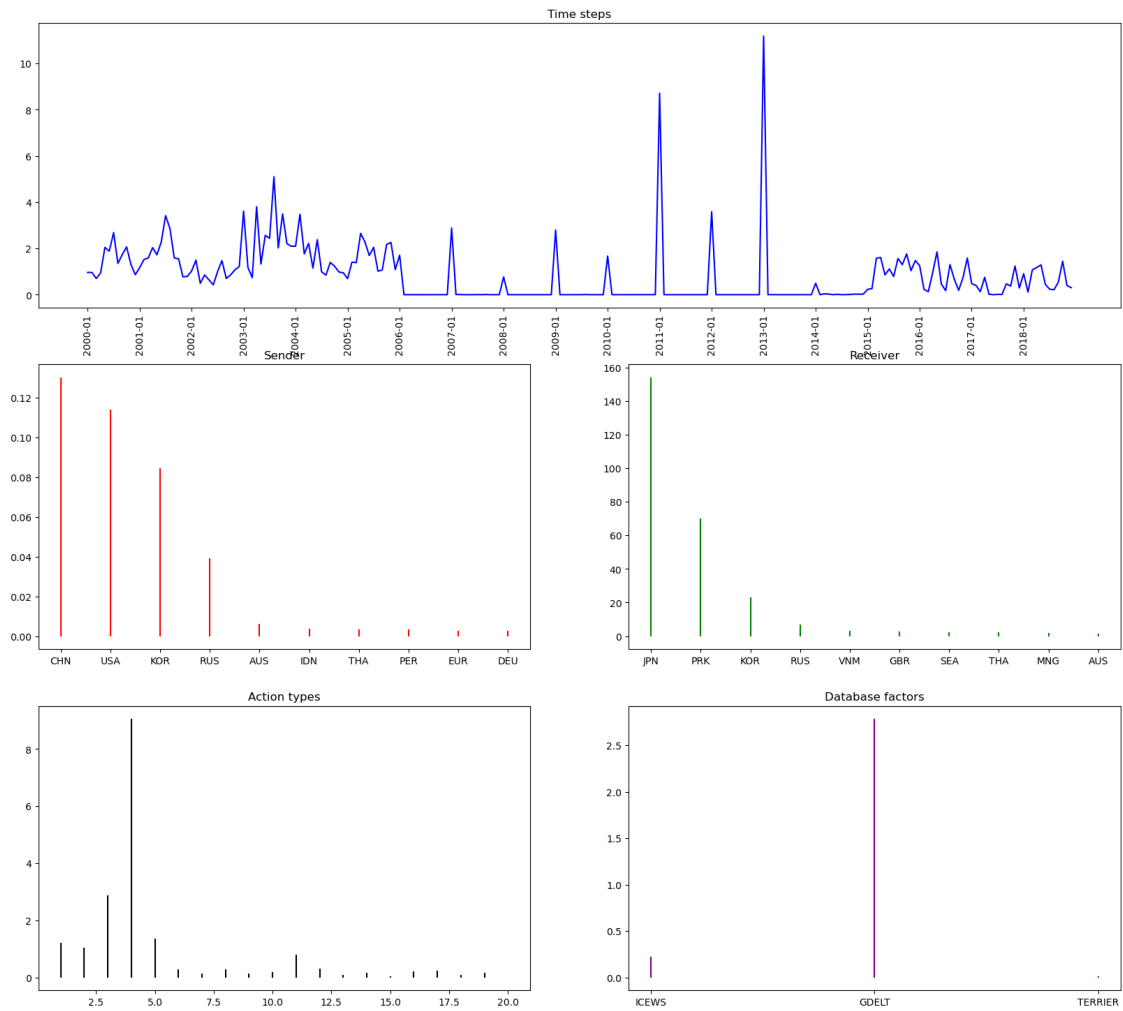


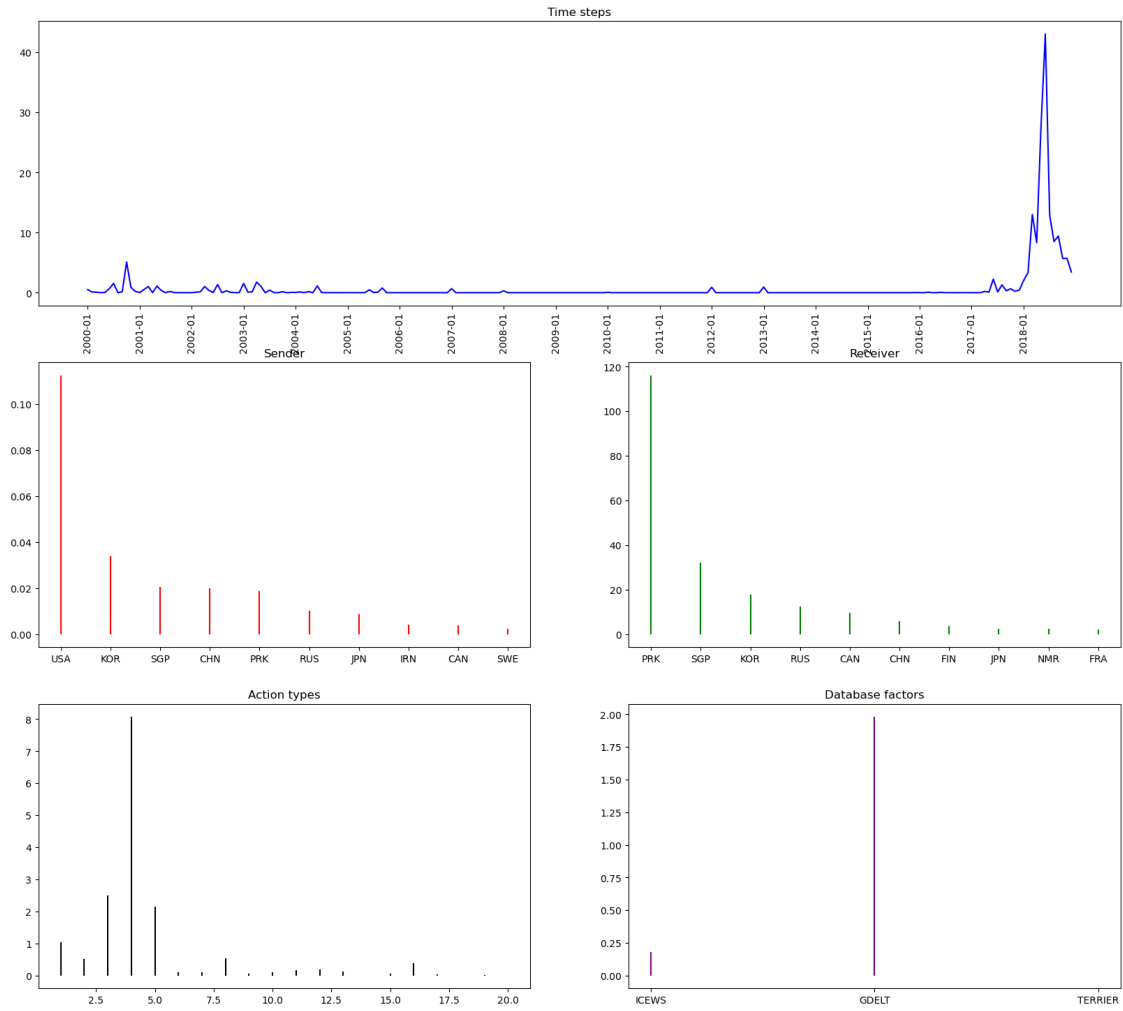
Component 81

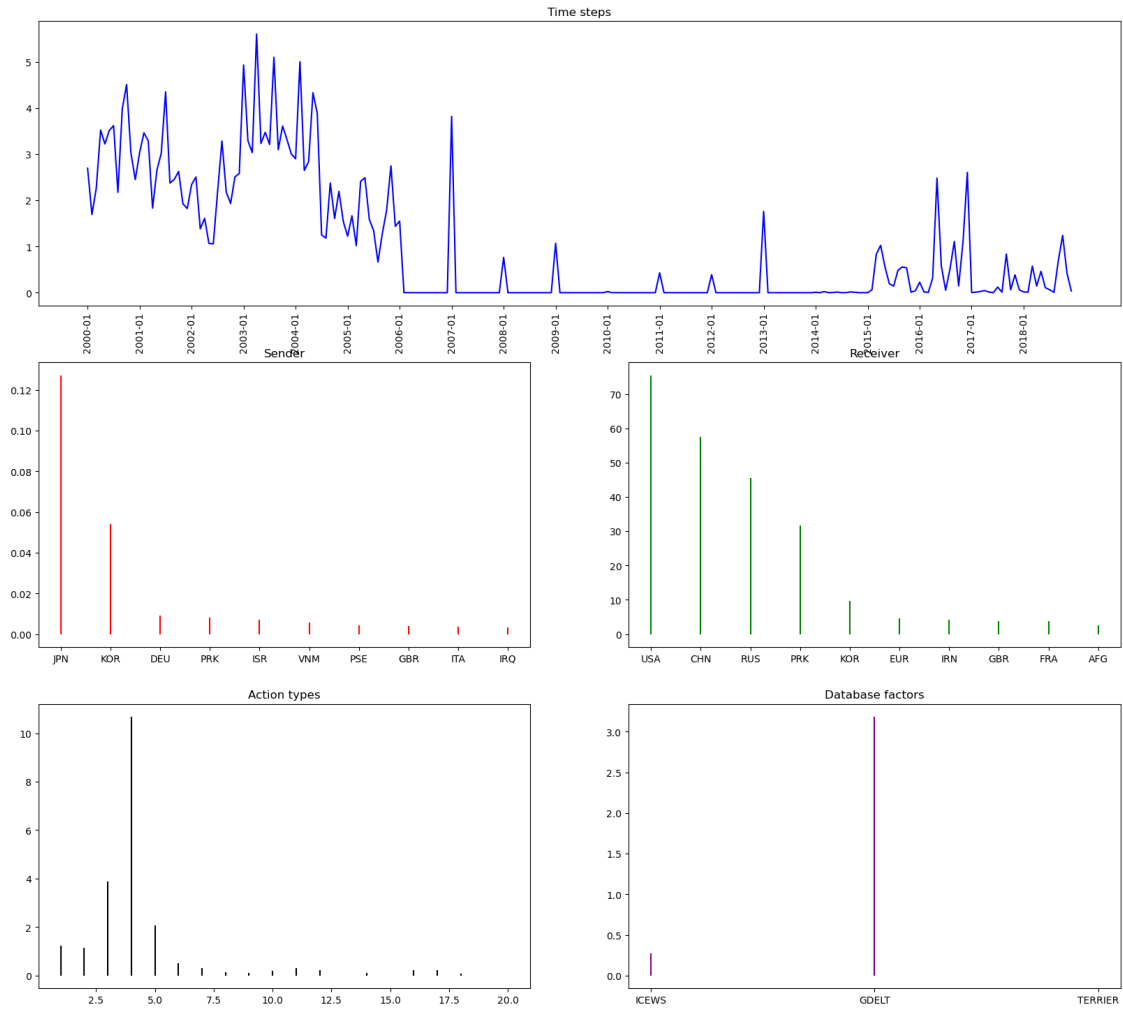


Component 14

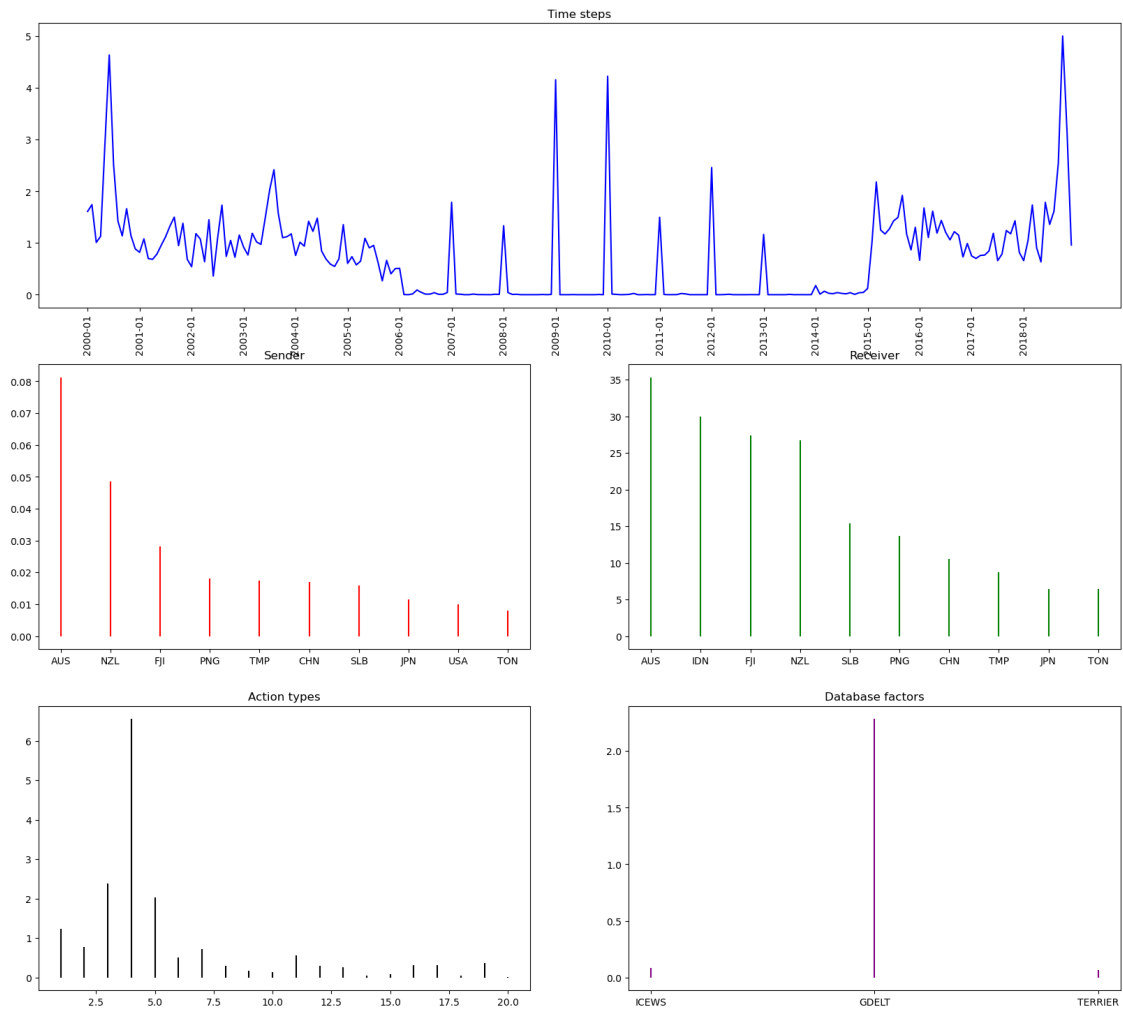


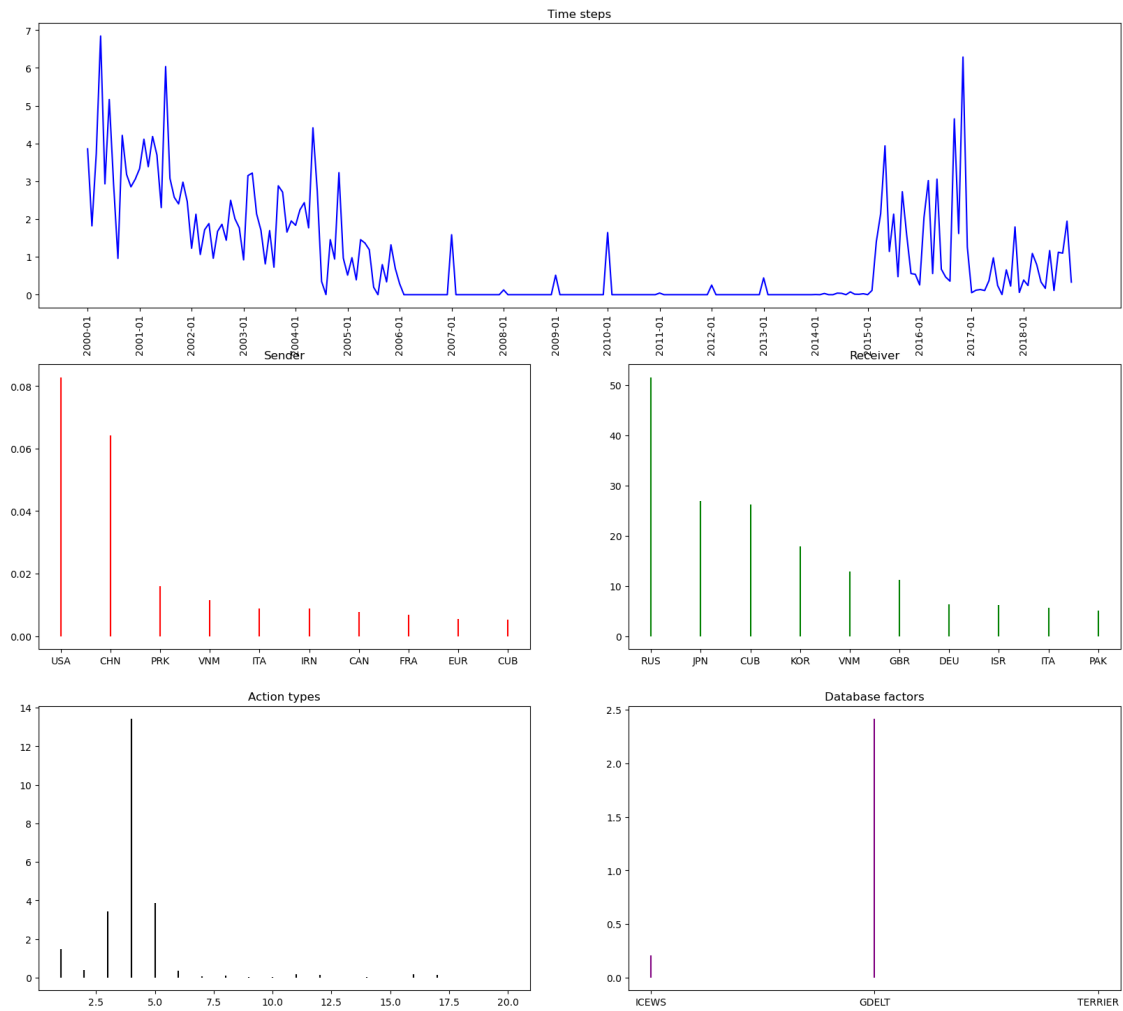




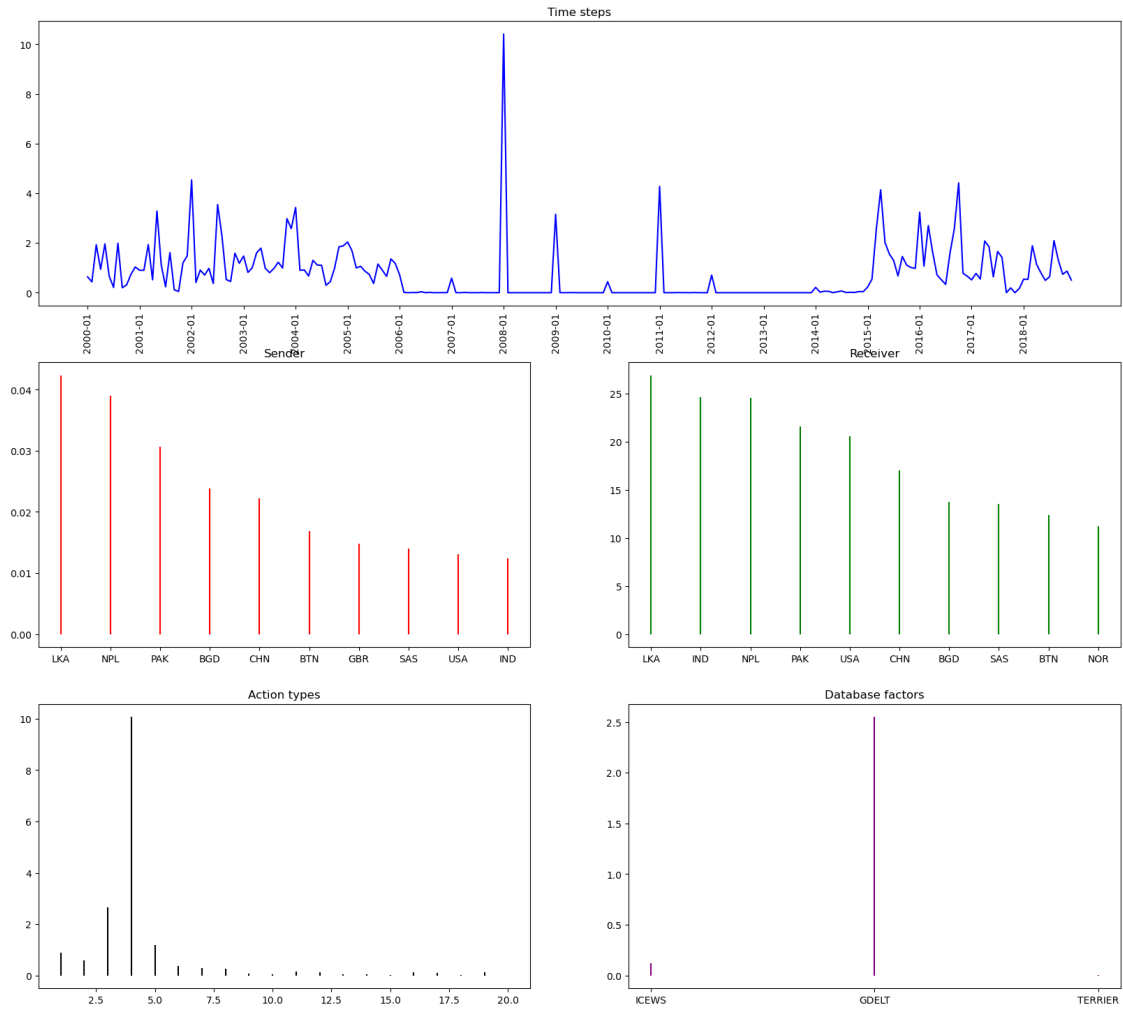


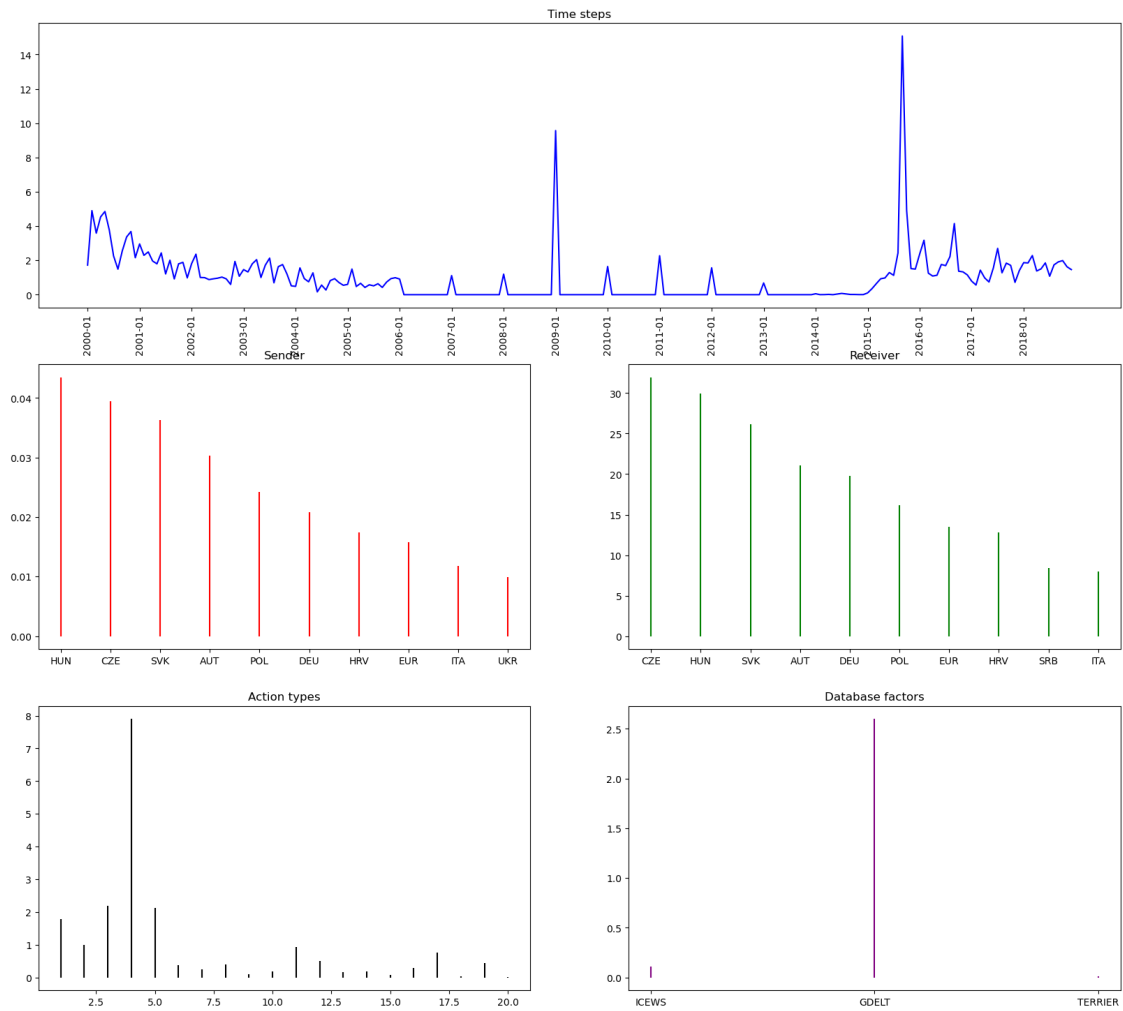
Component 20



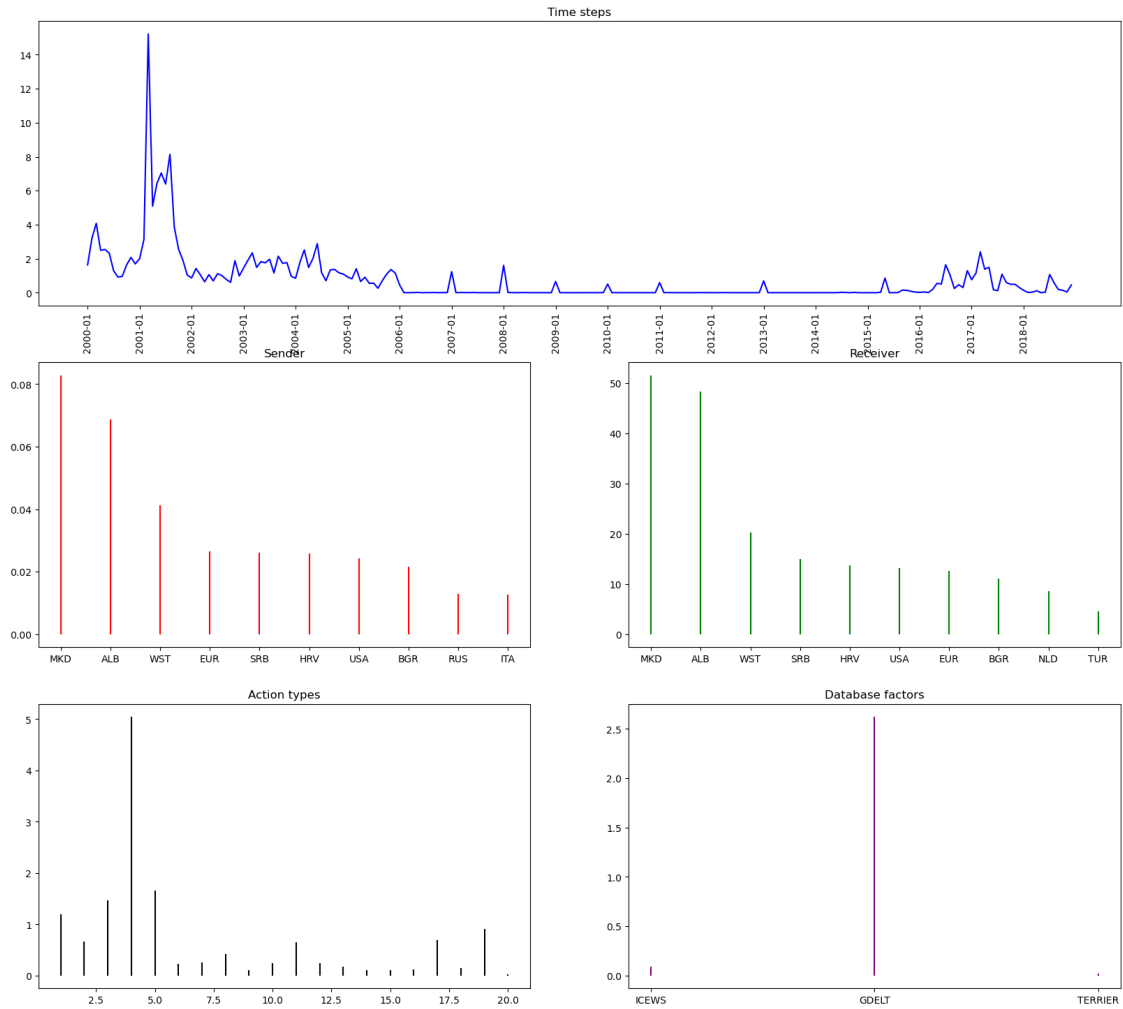


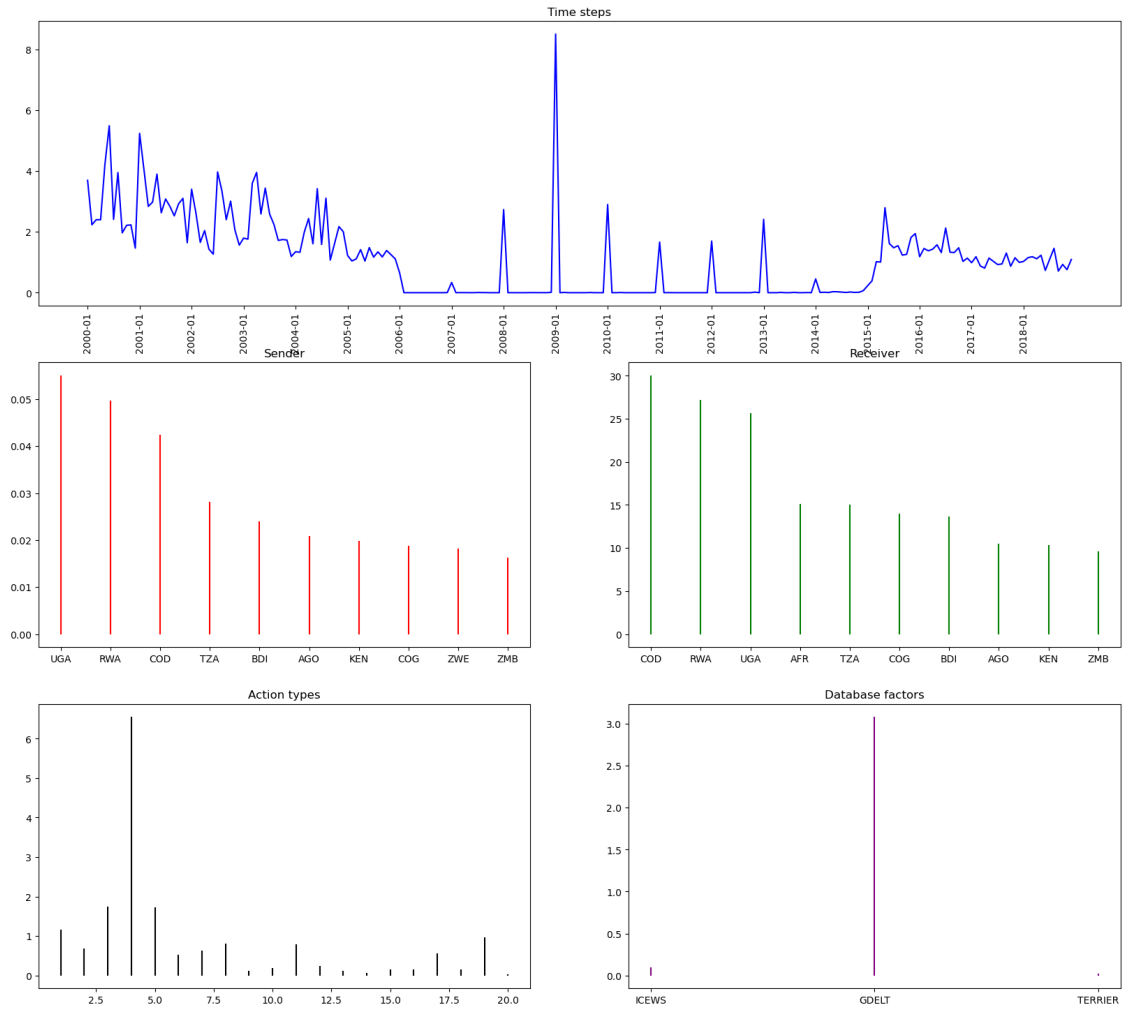
Component 27

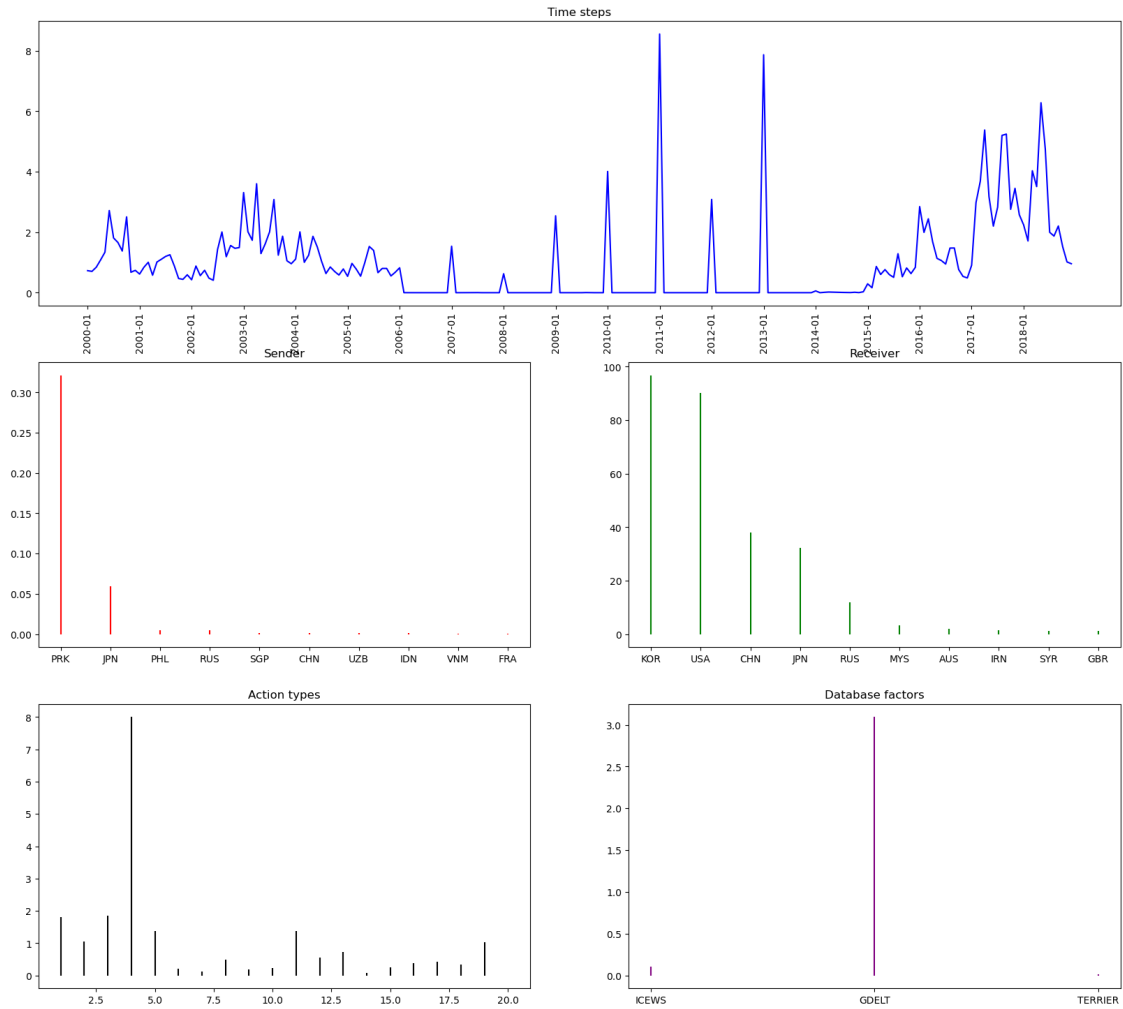


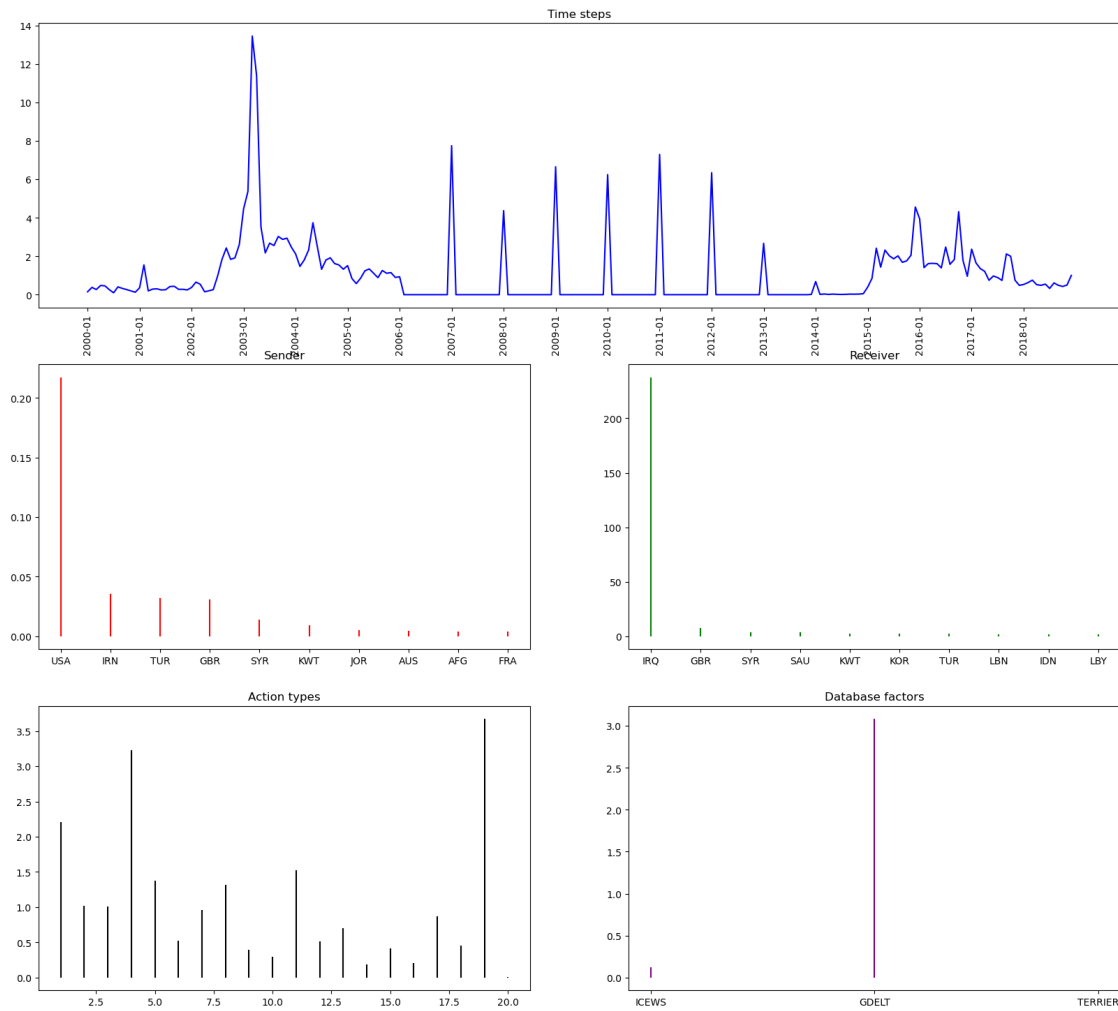


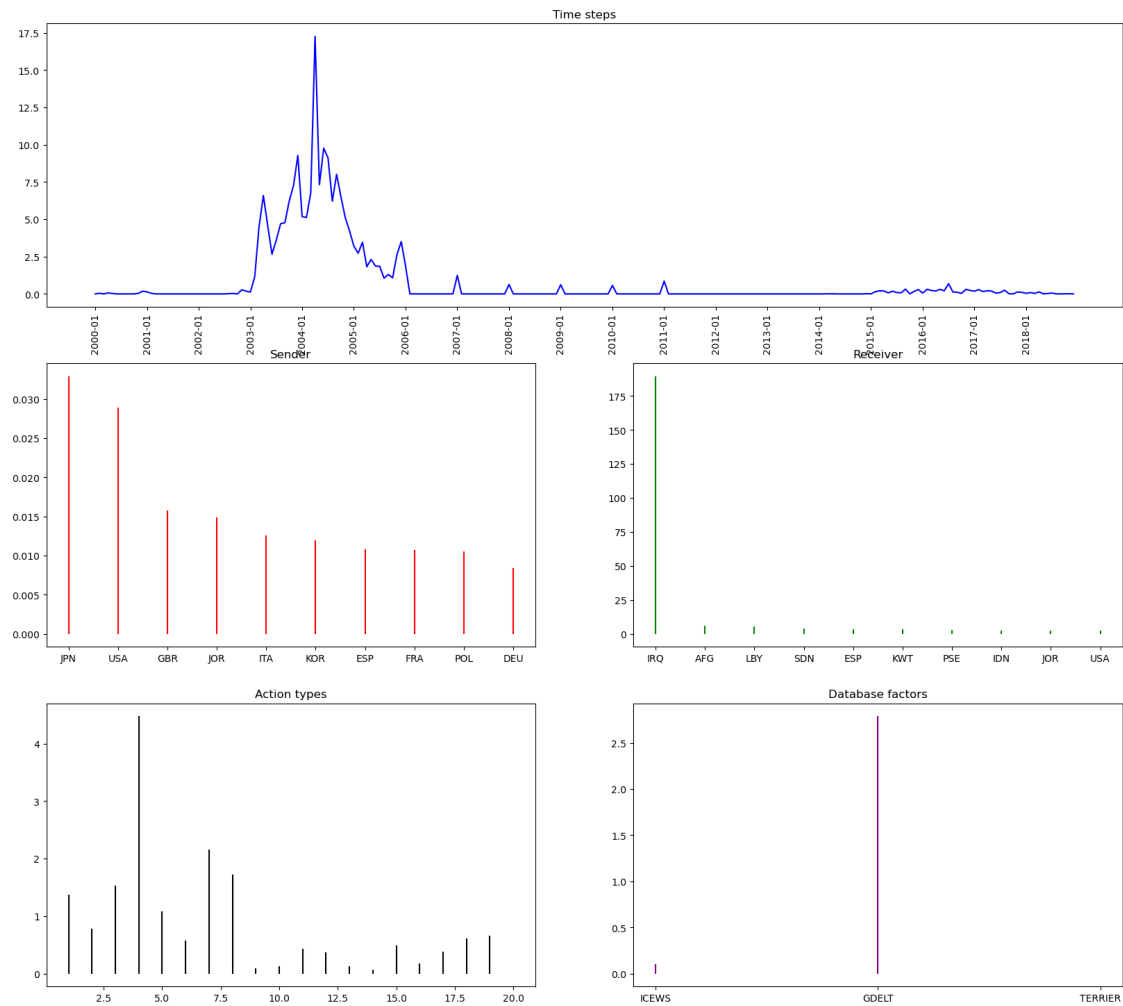
Component 34



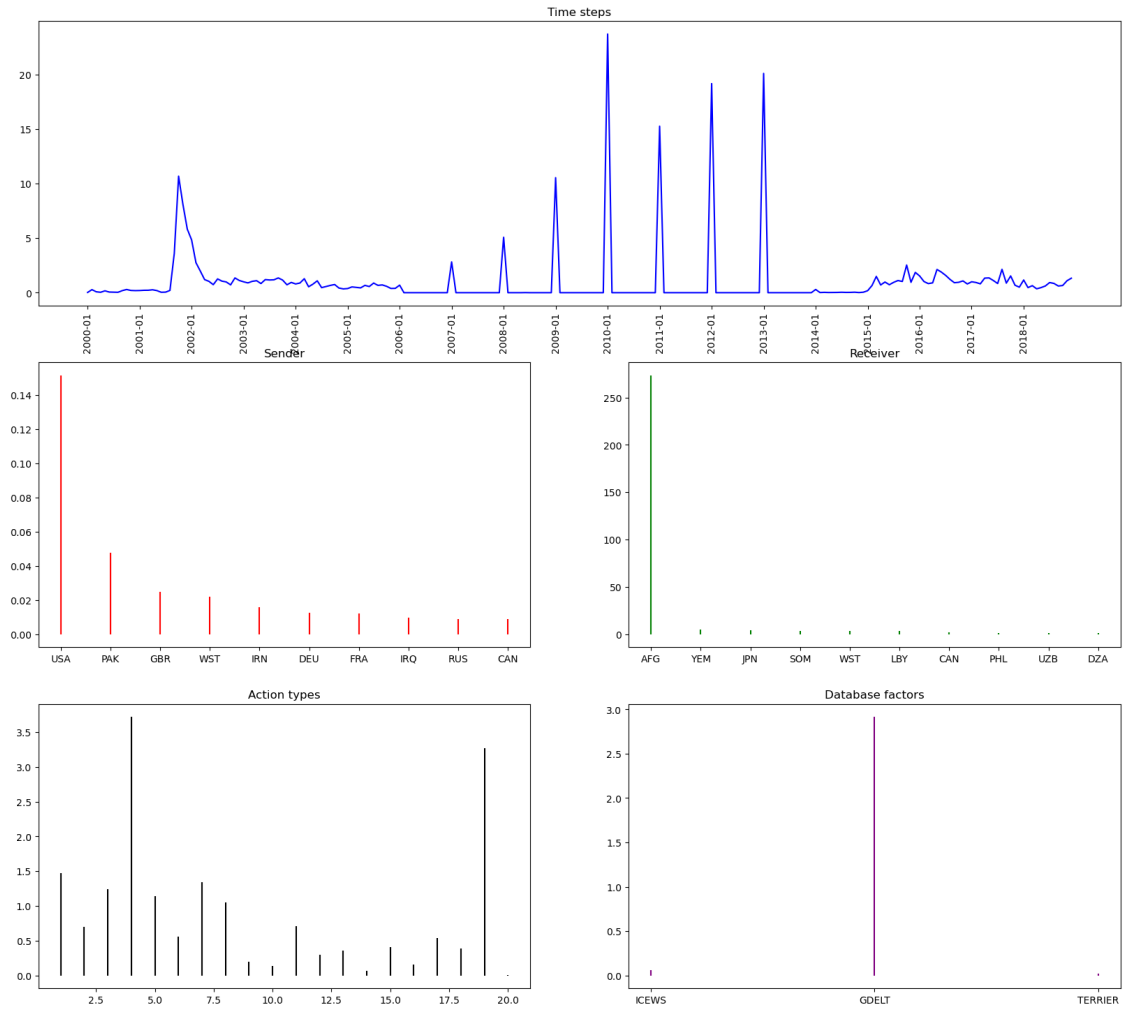


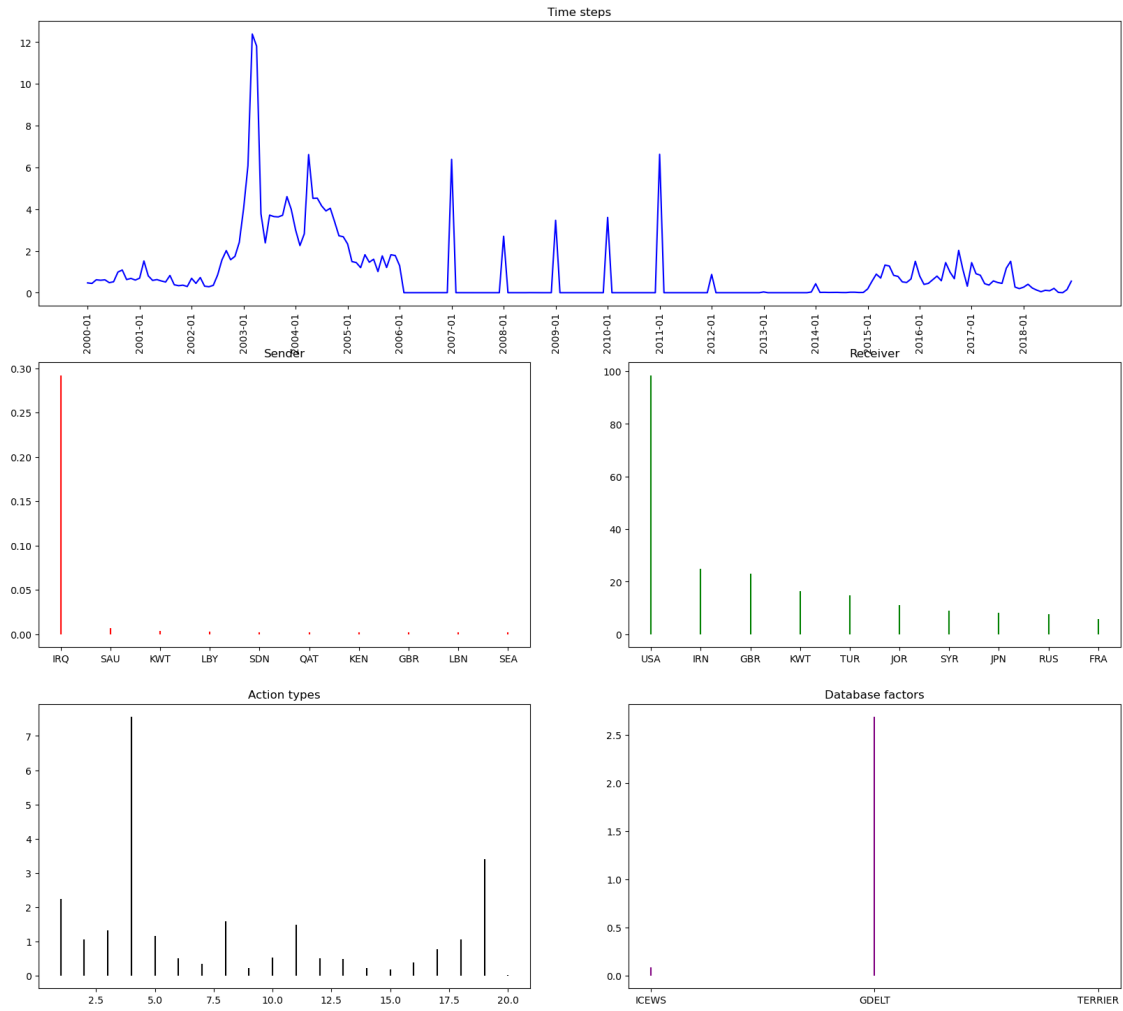


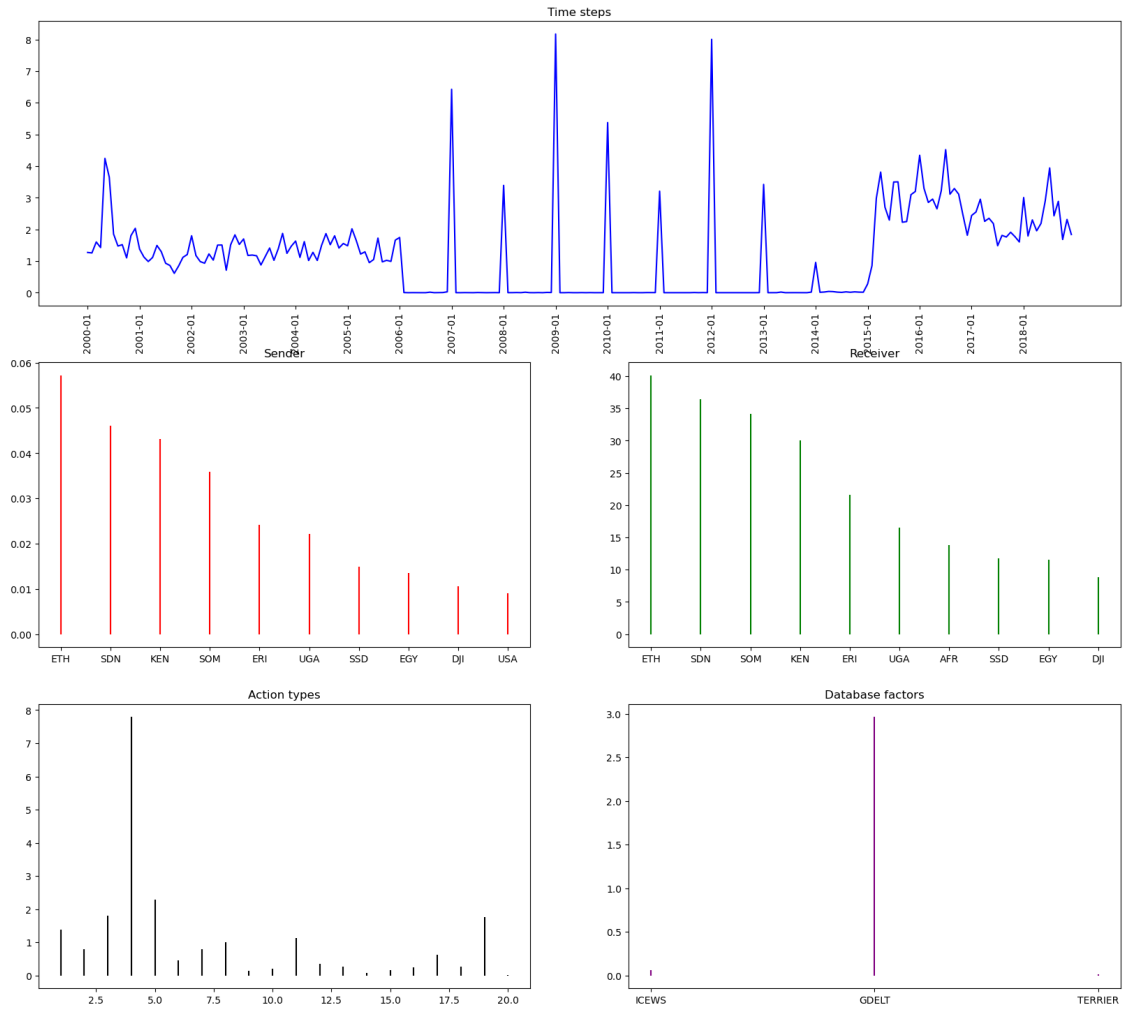




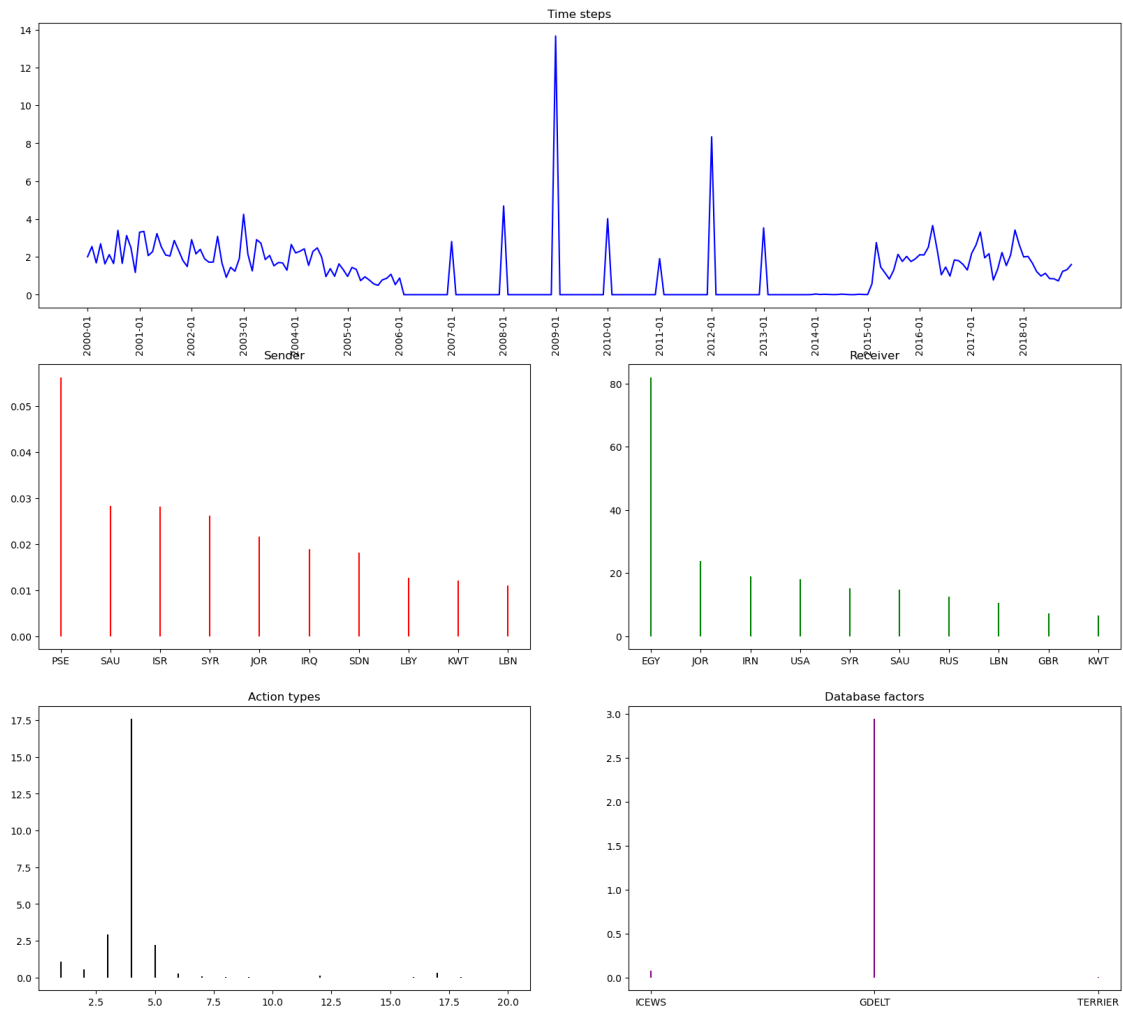
Component 5

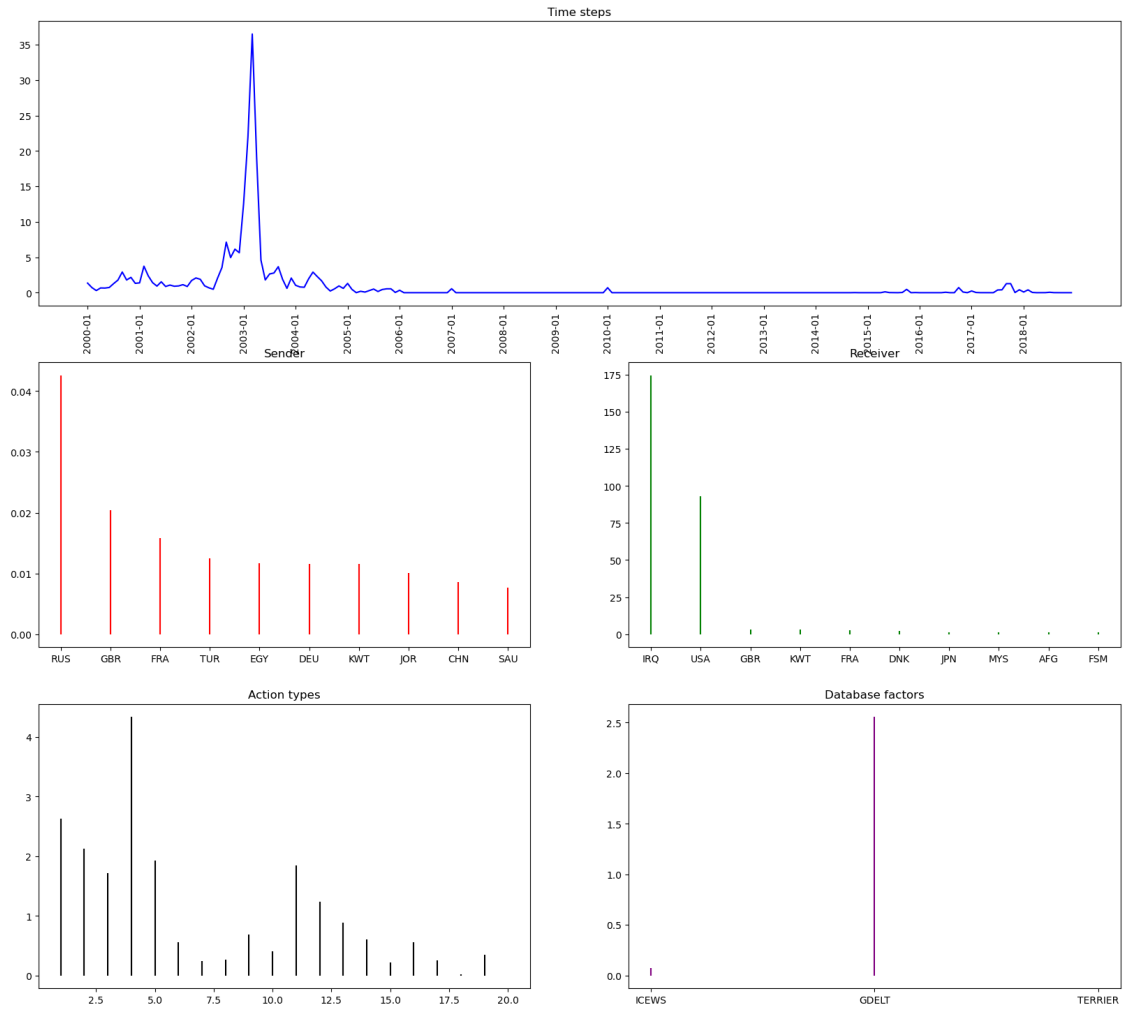


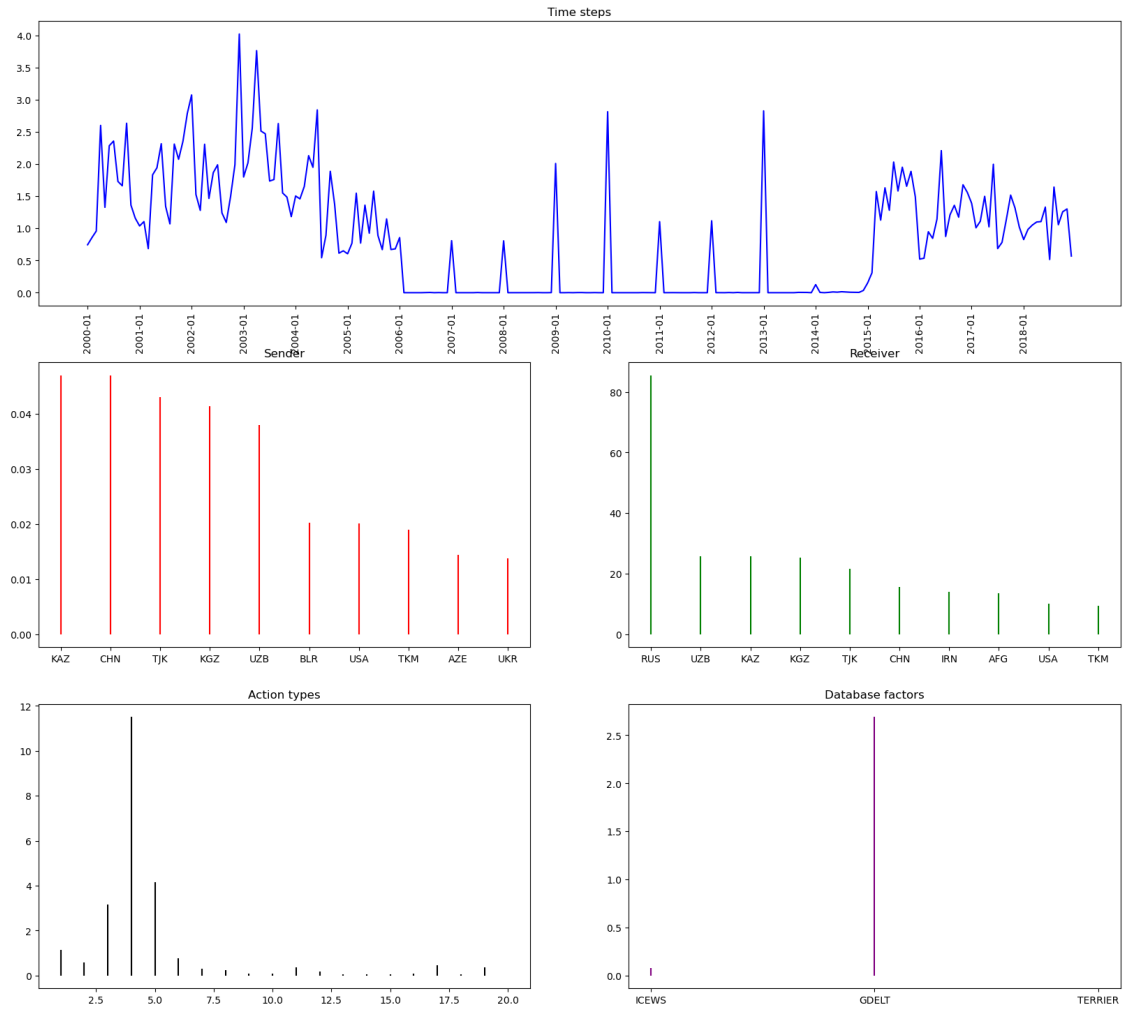


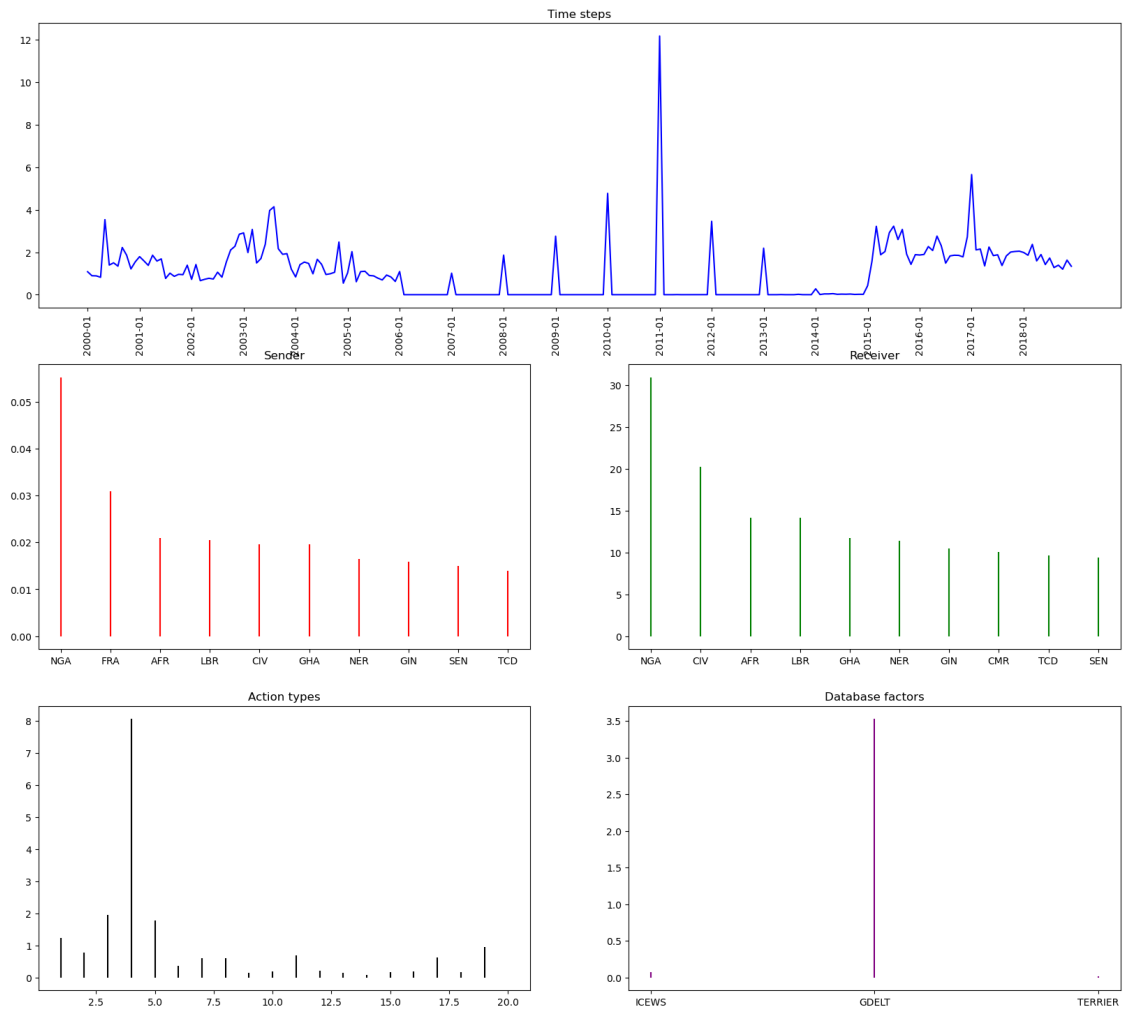


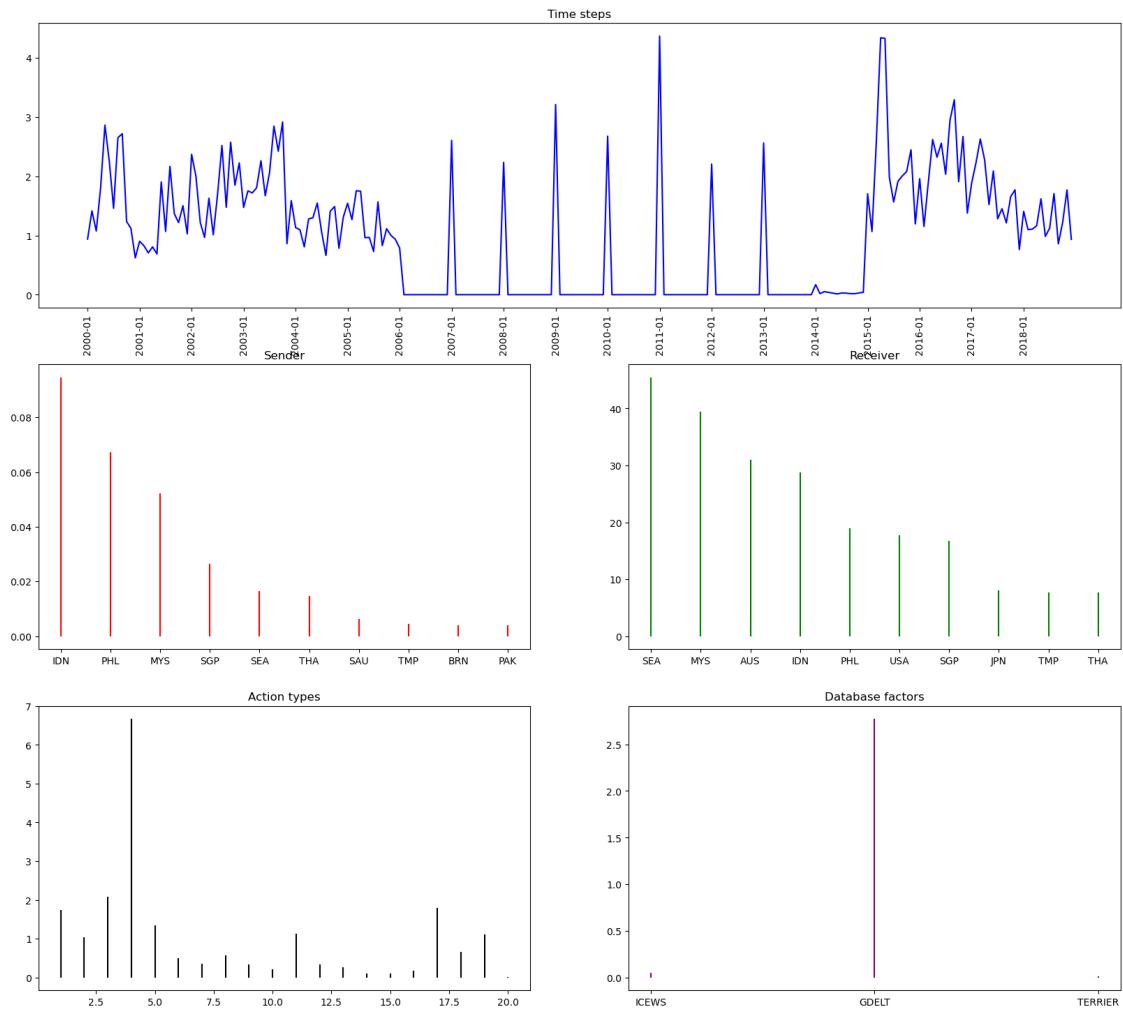
Component 43



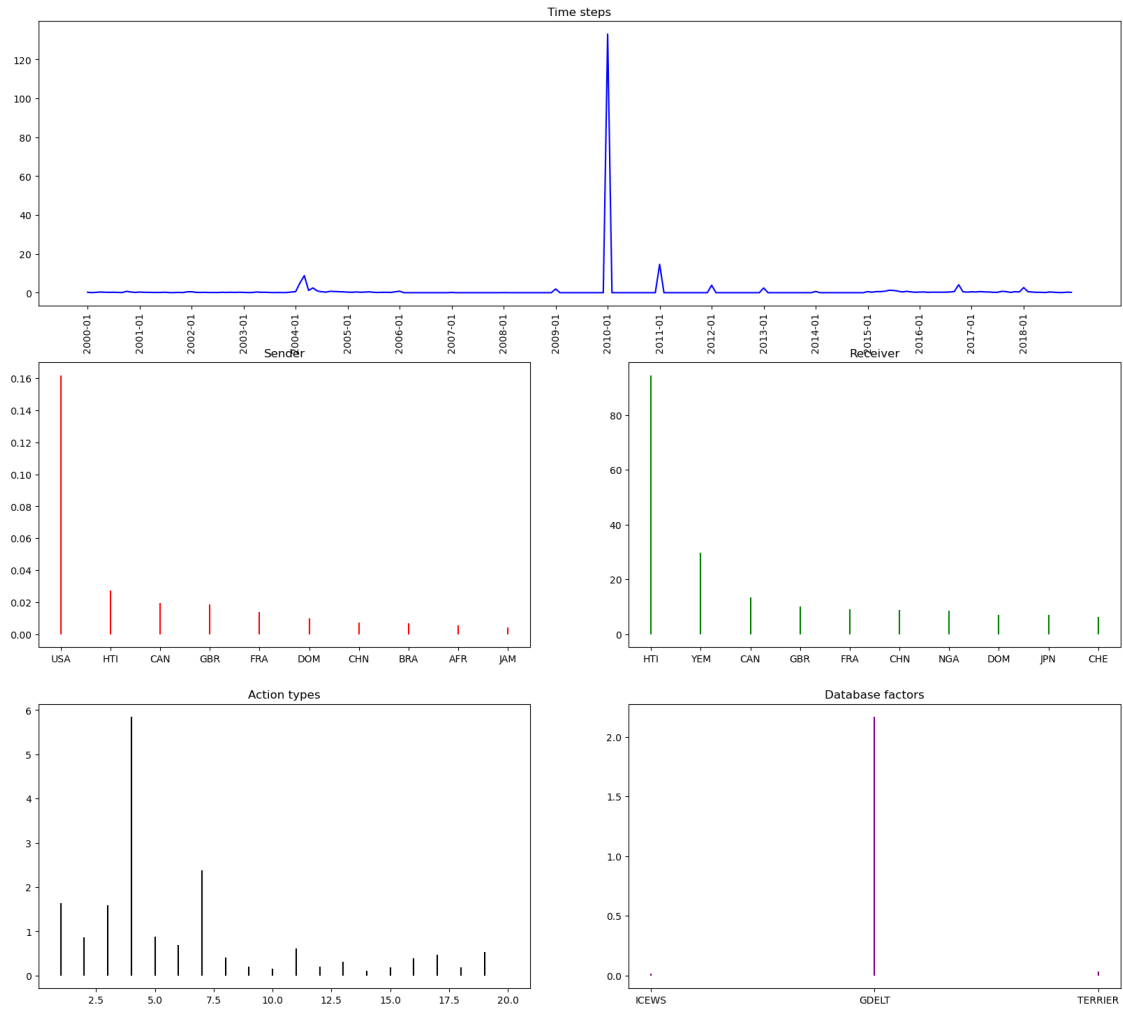




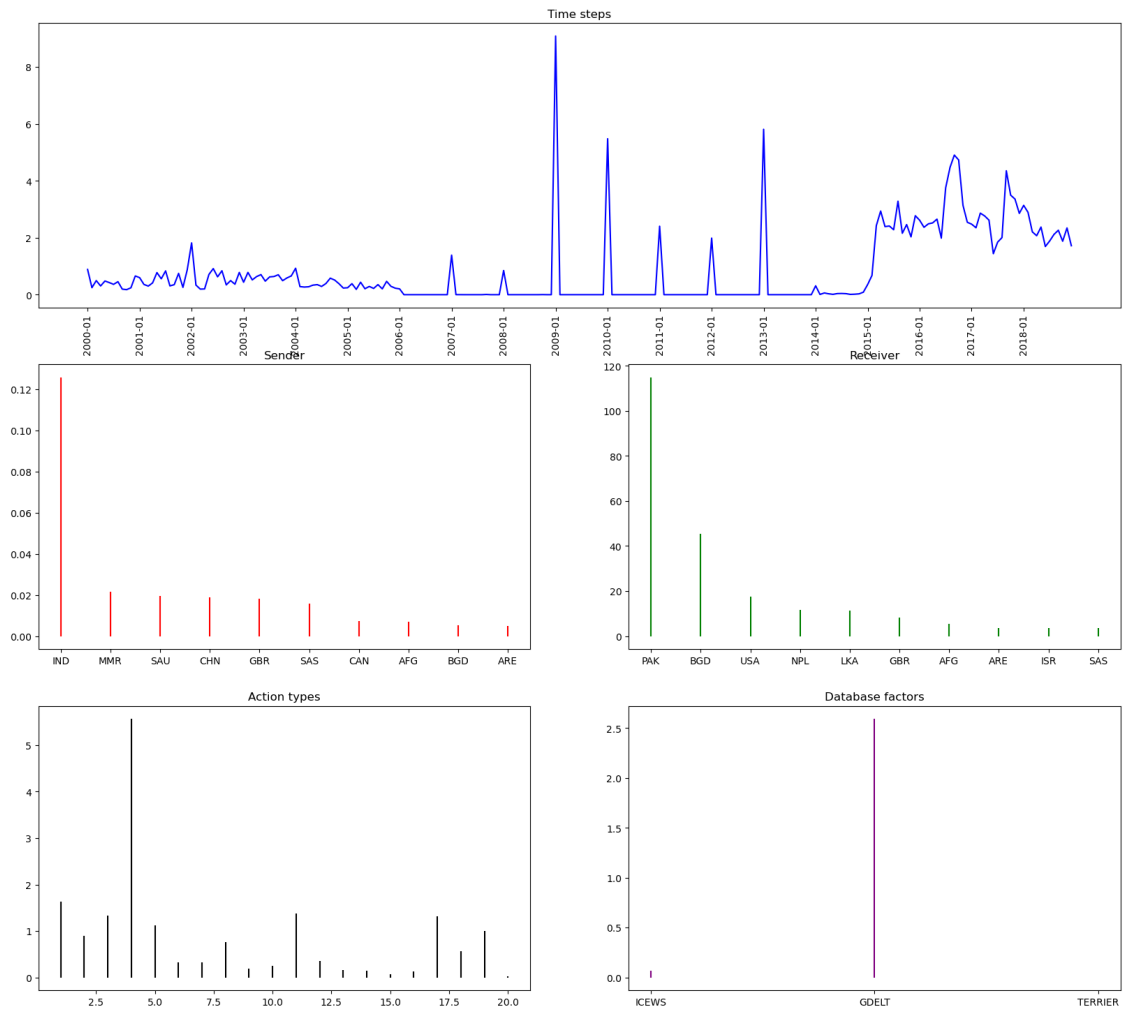


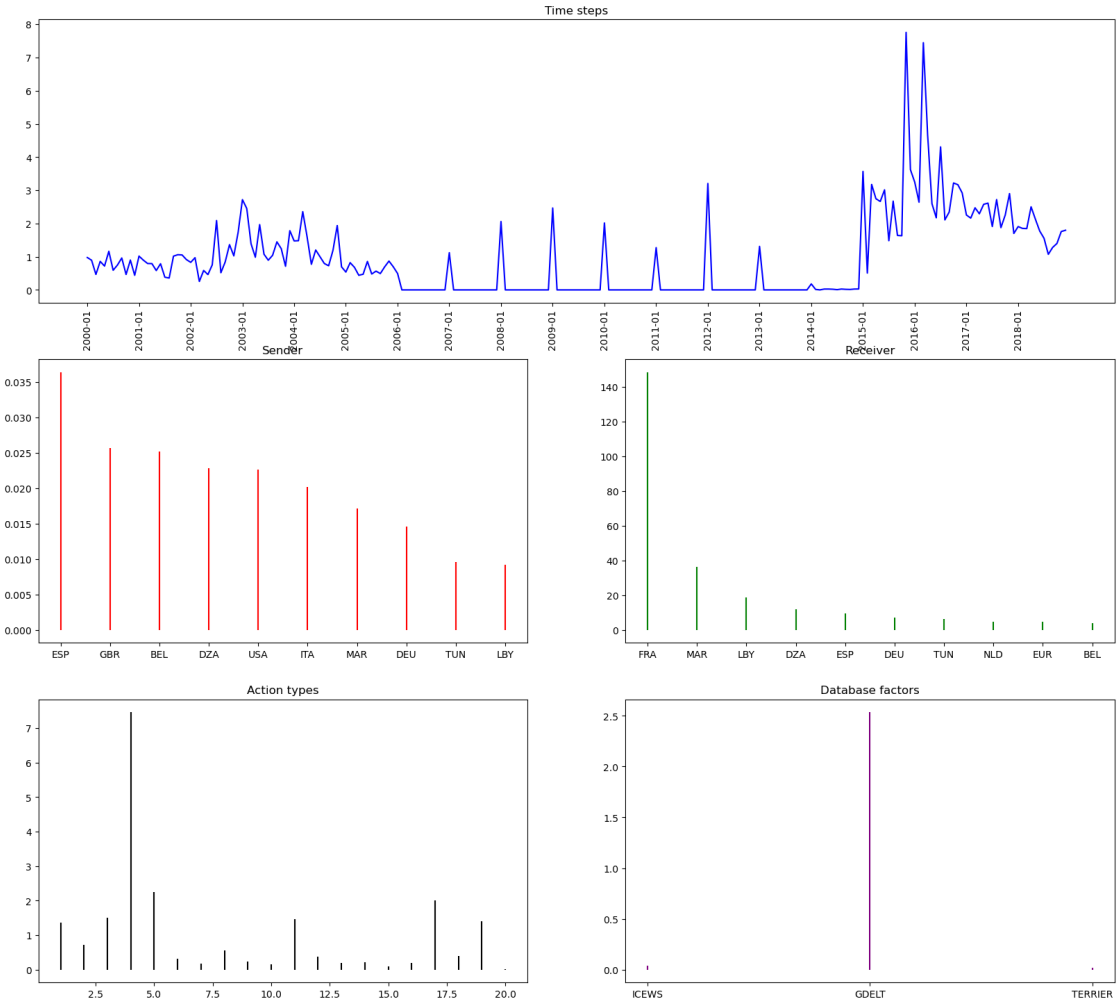


Component 0

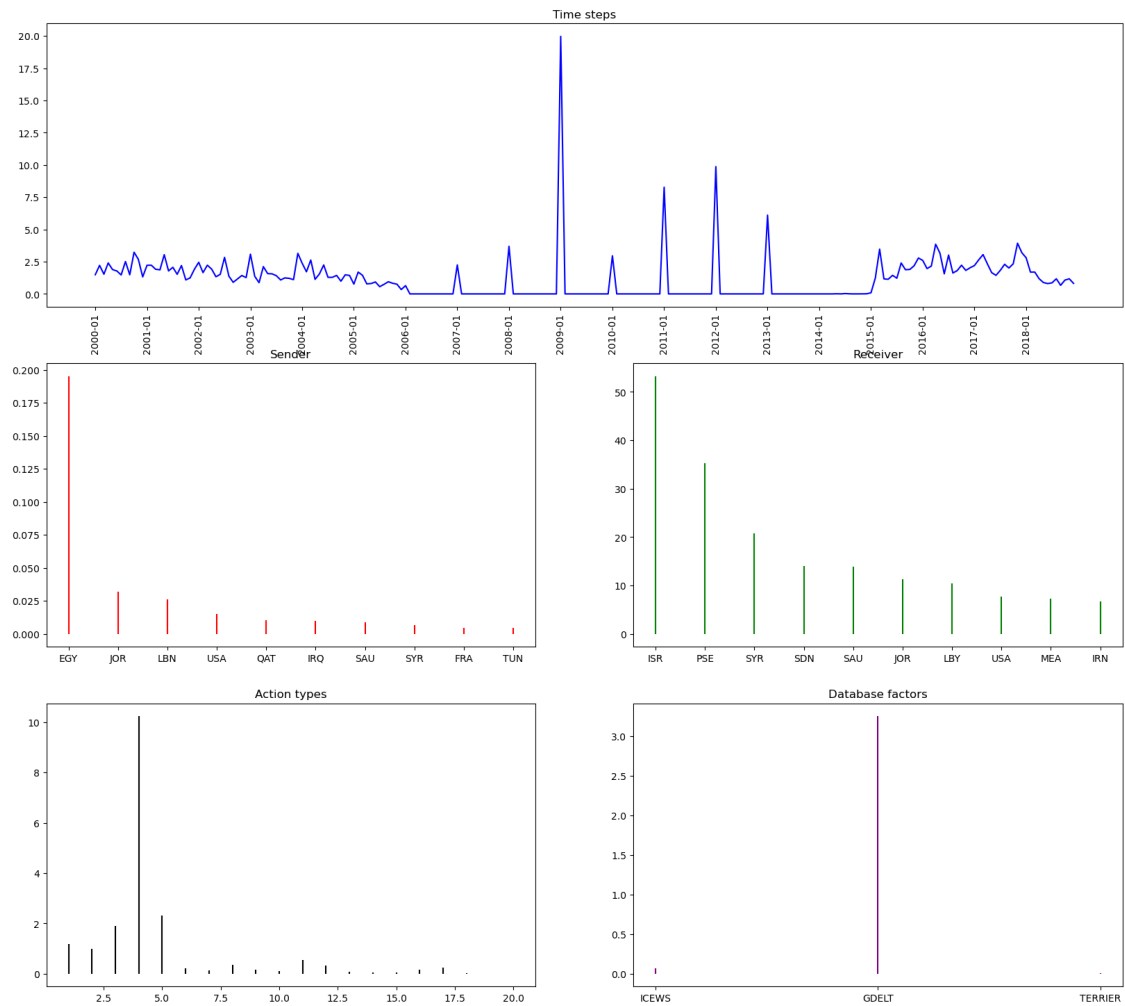


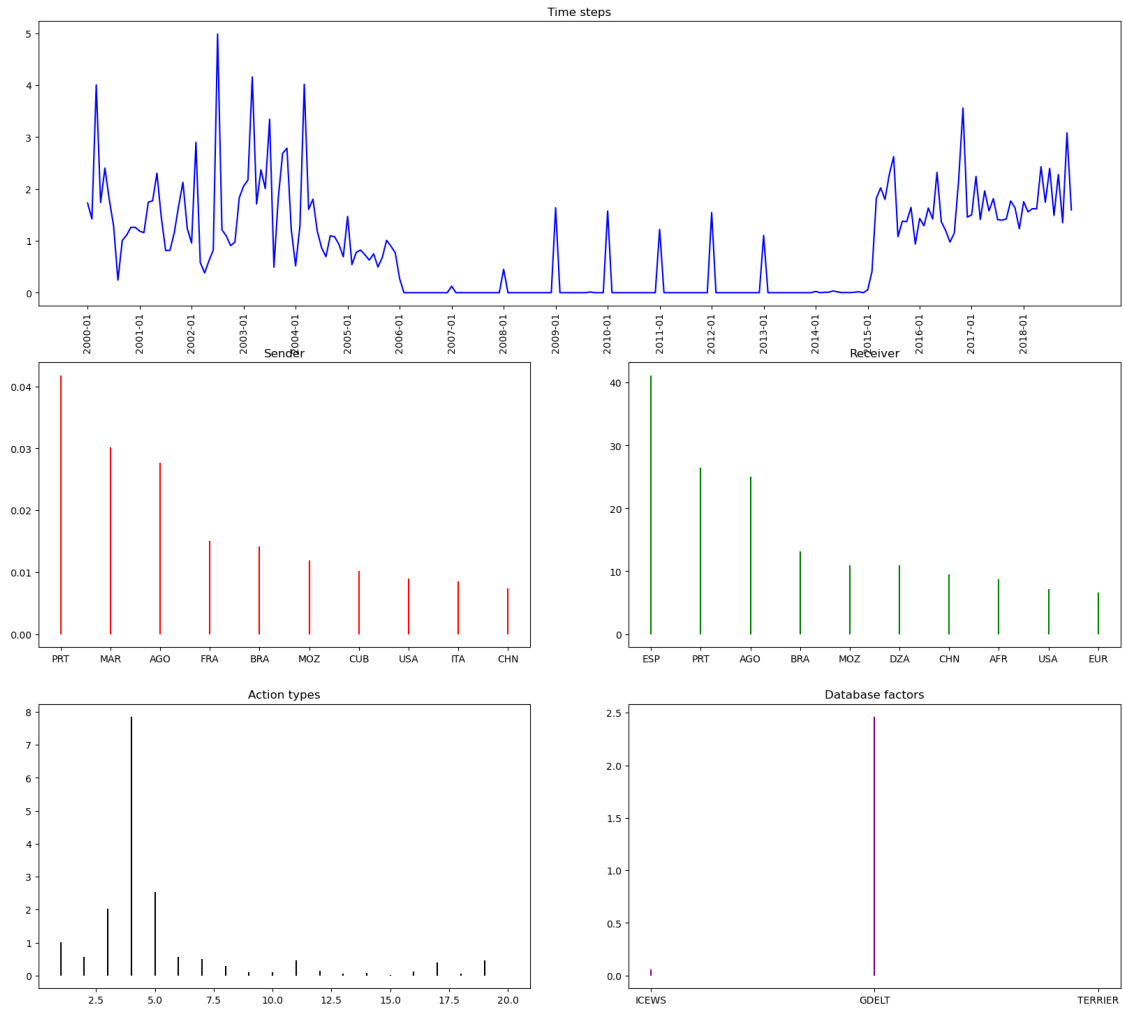
Component 23



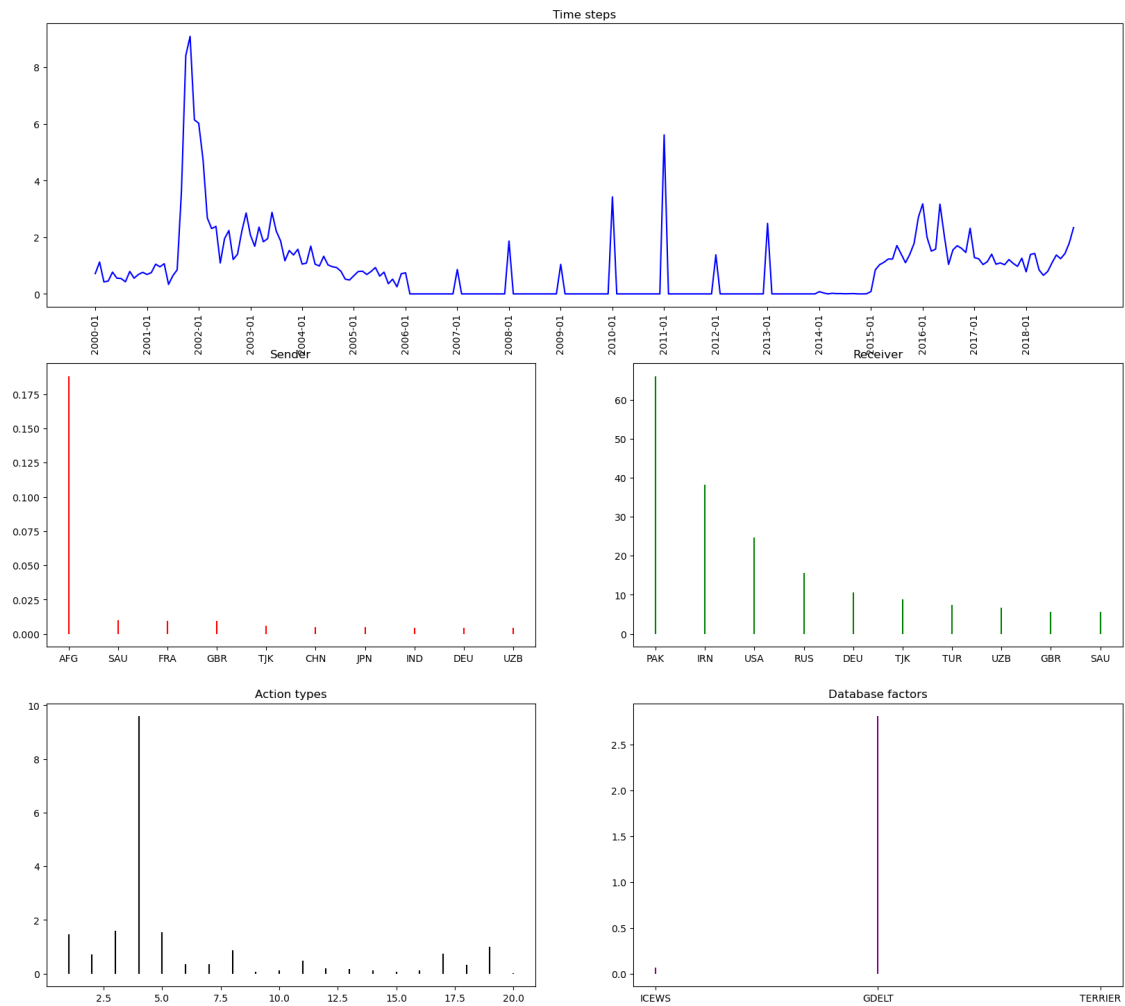


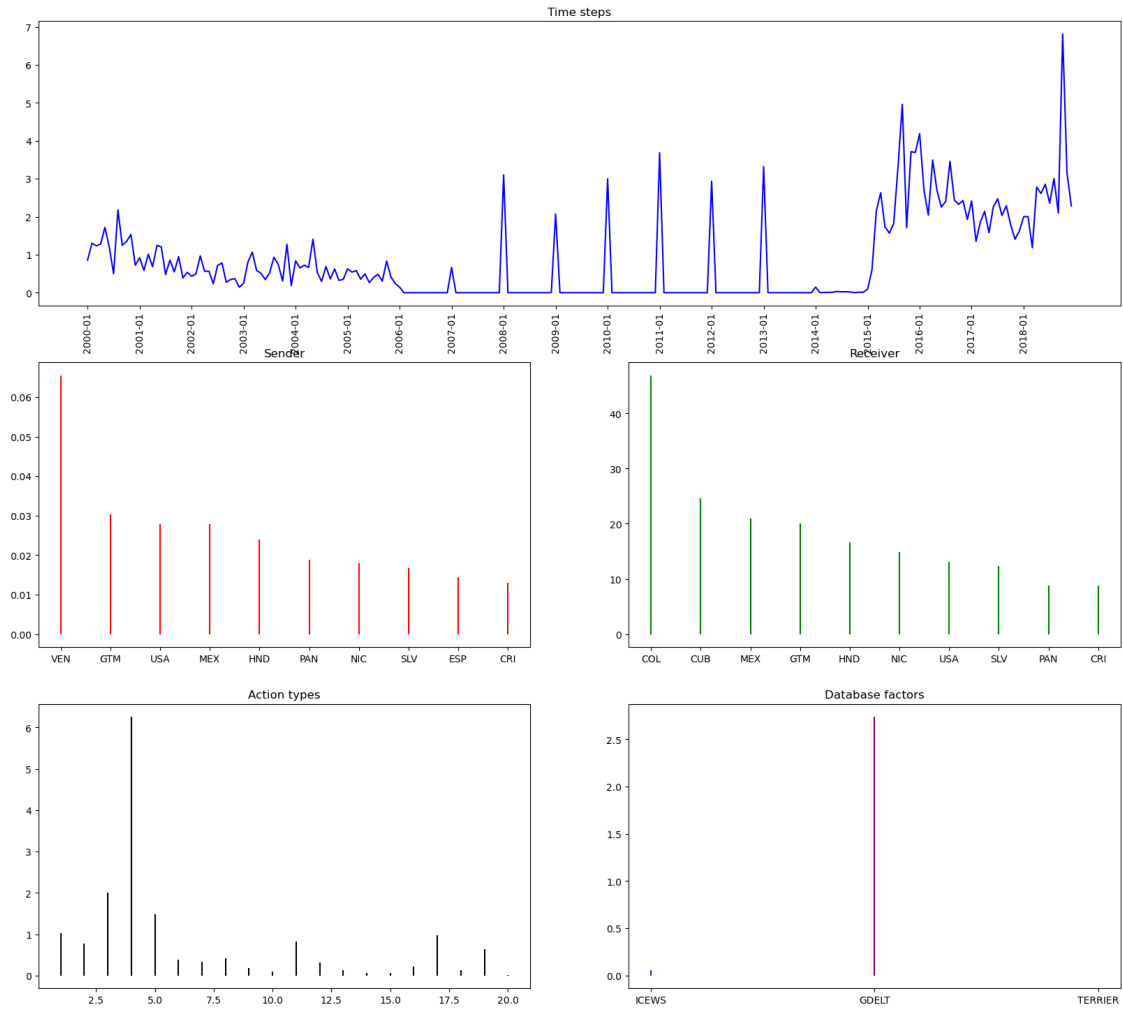
Component 90



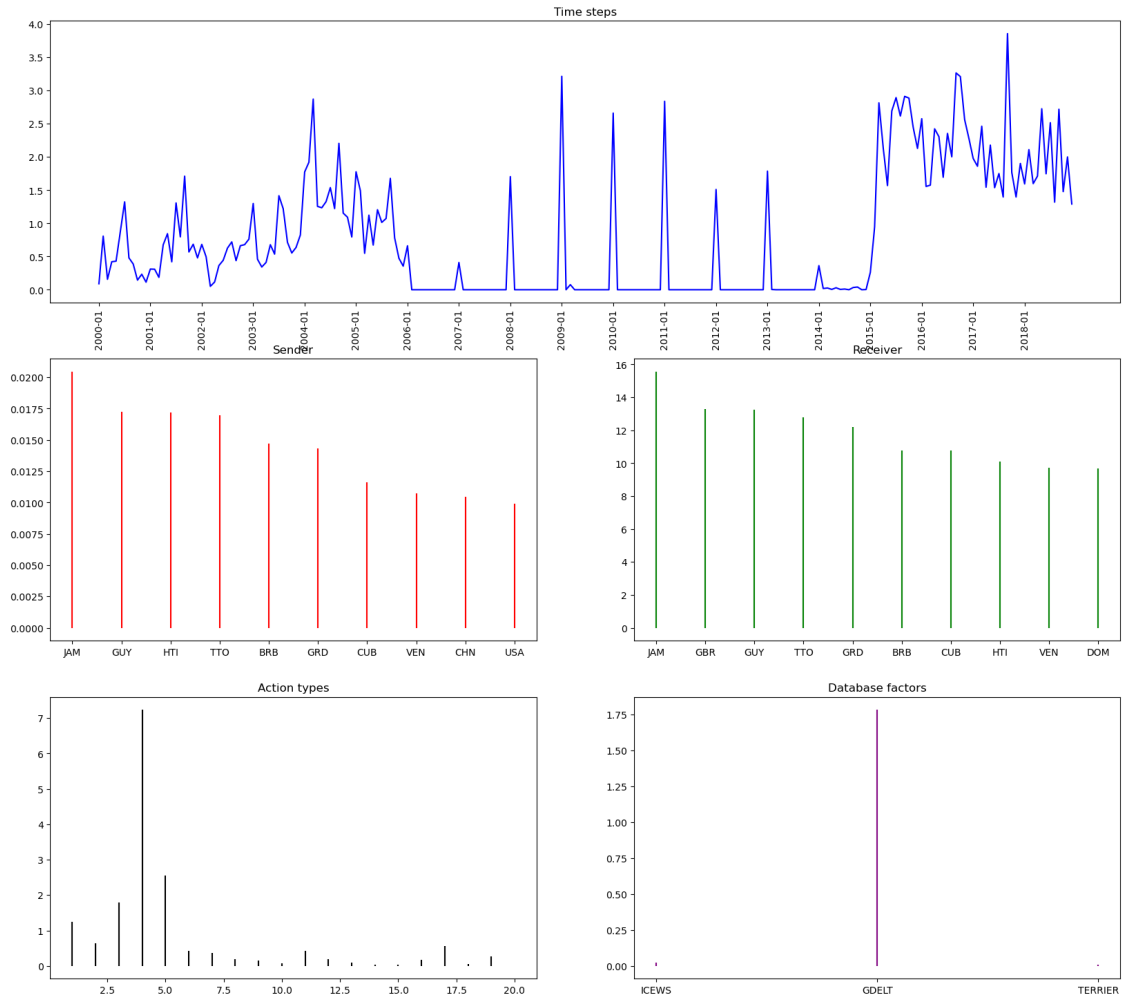


Component 17

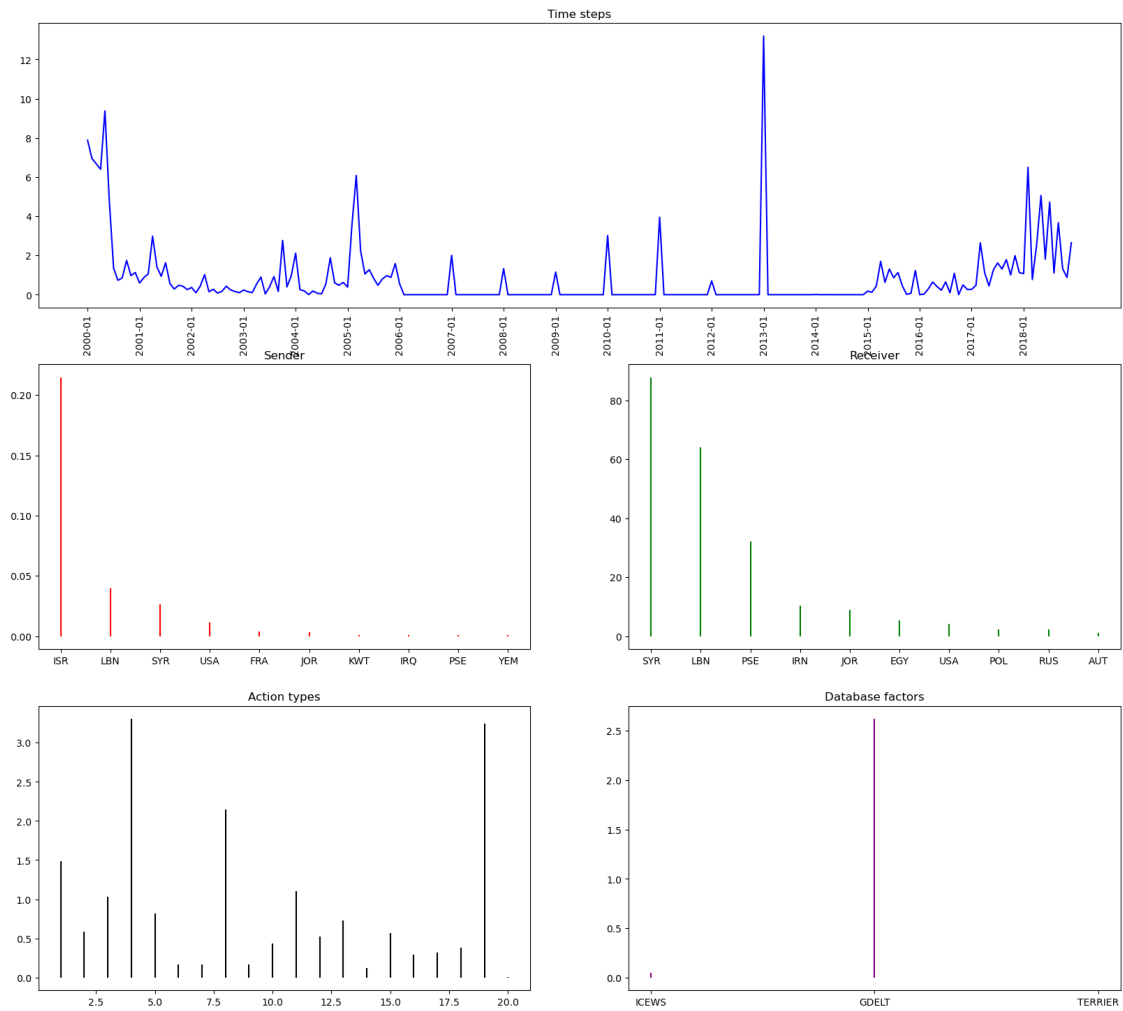


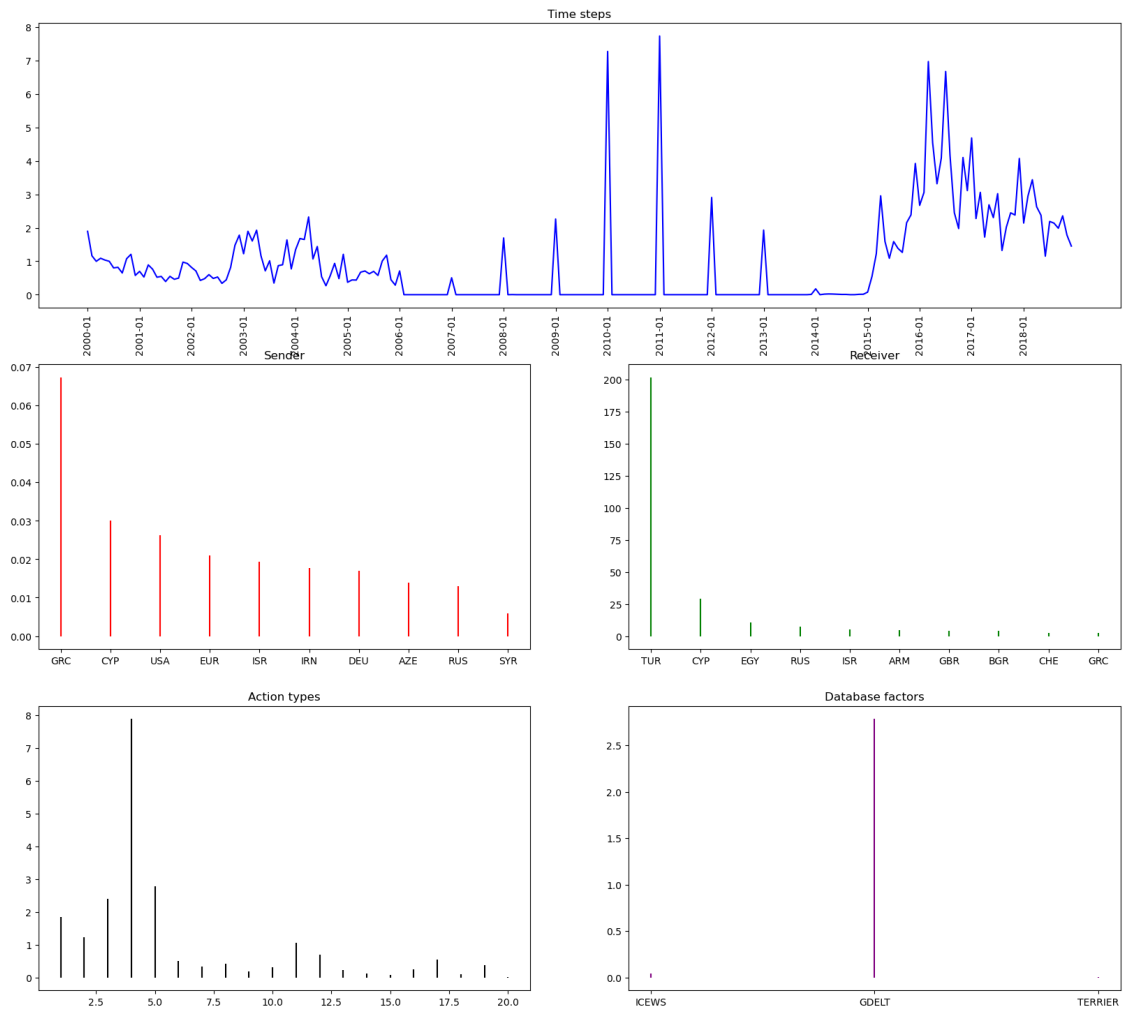


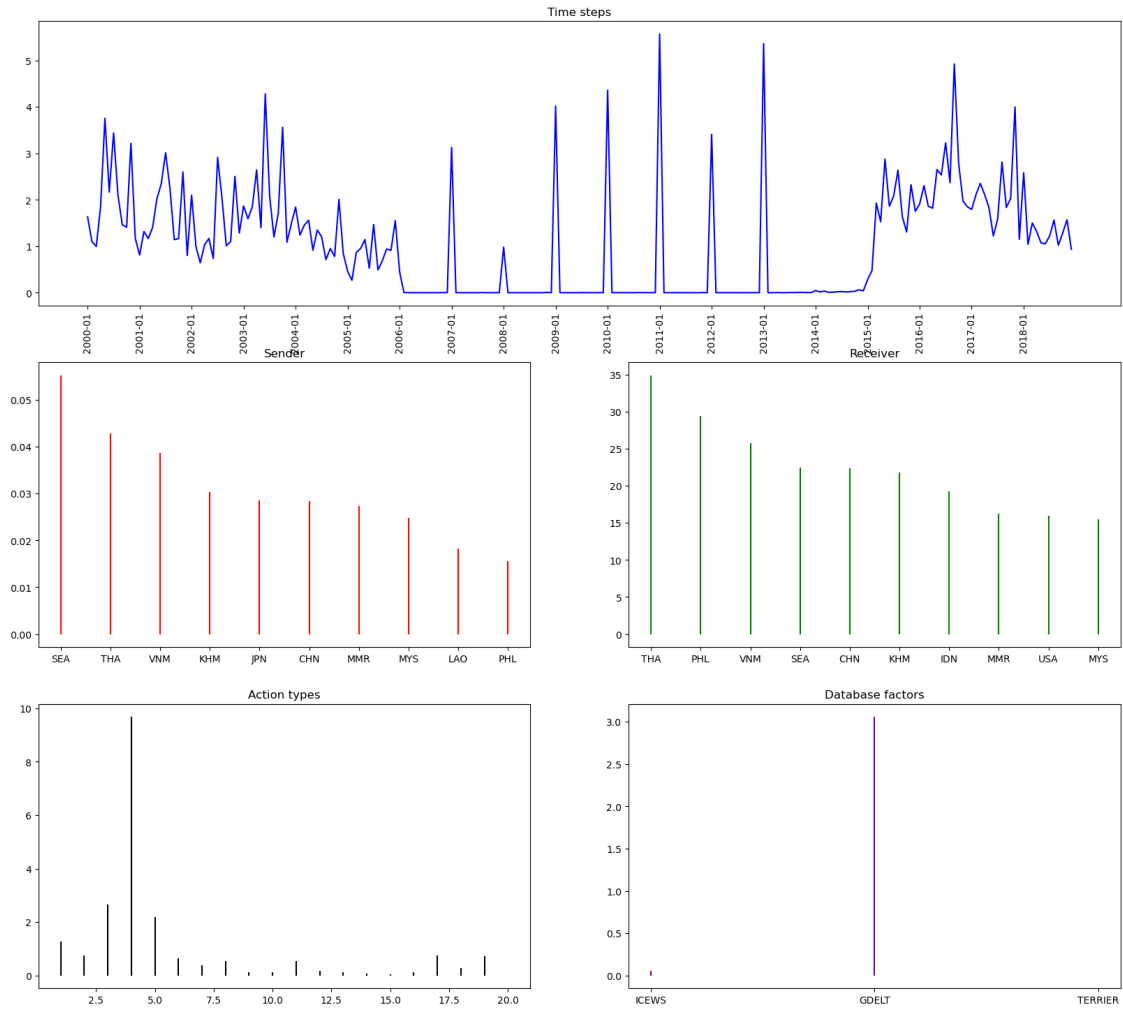
Component 58

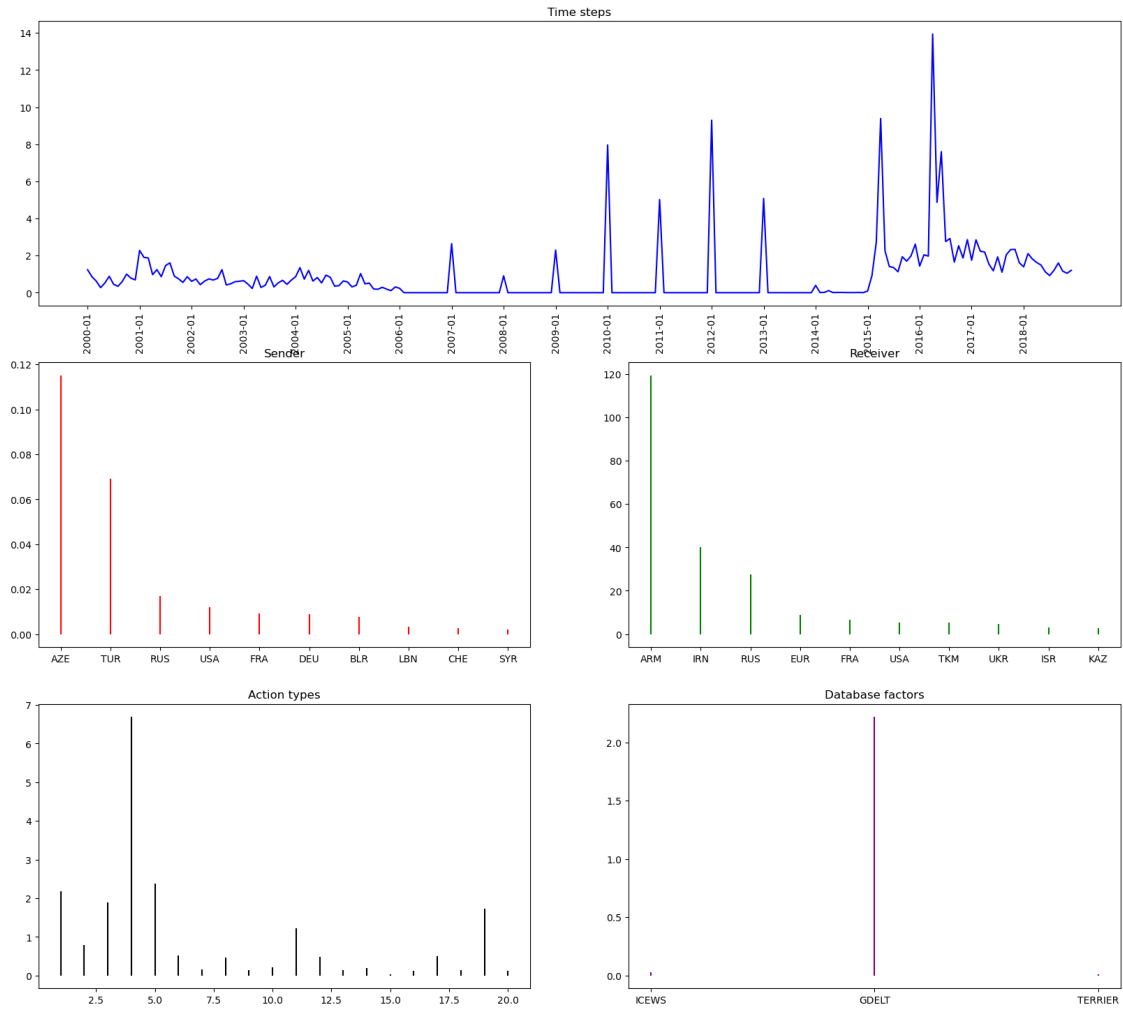


Component 12

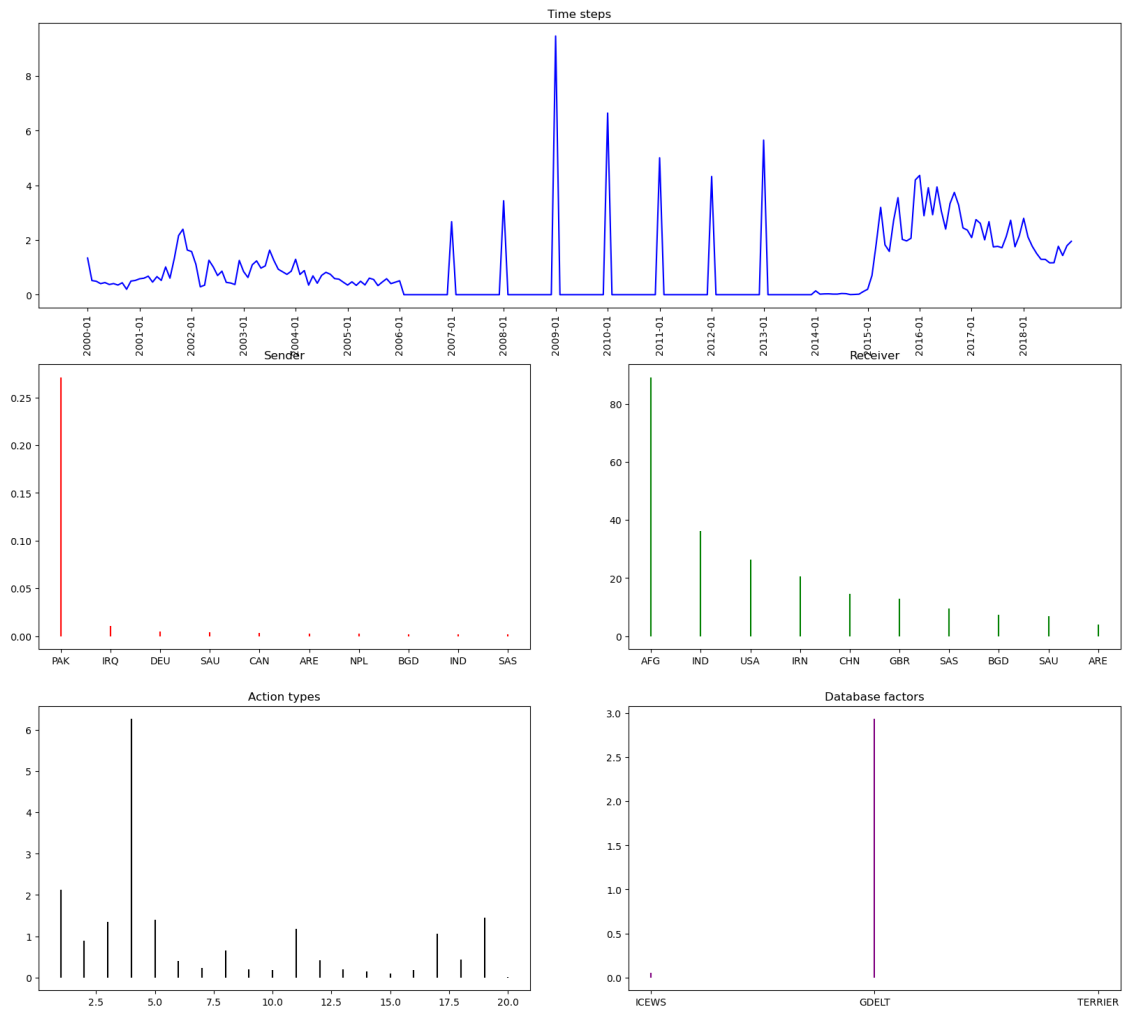




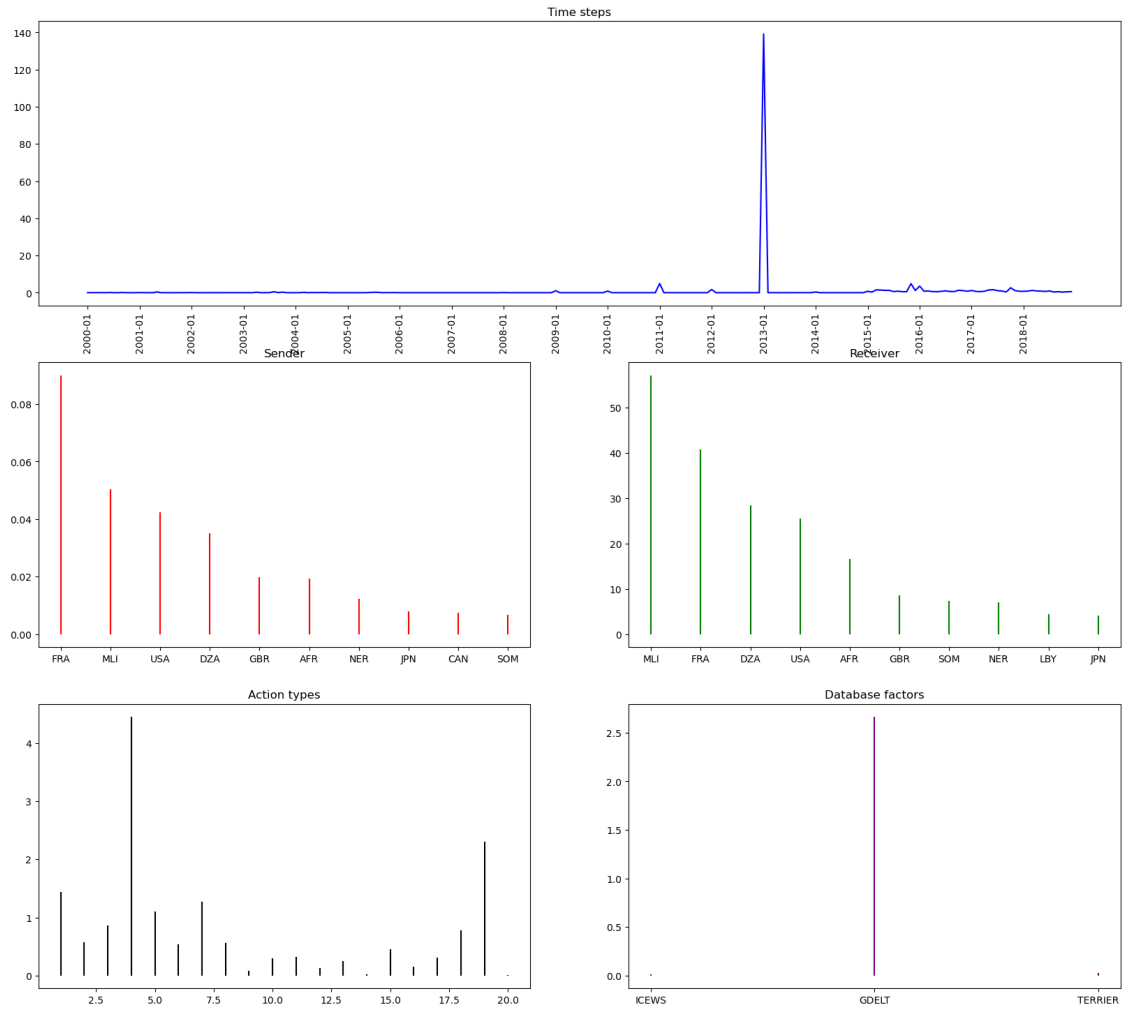


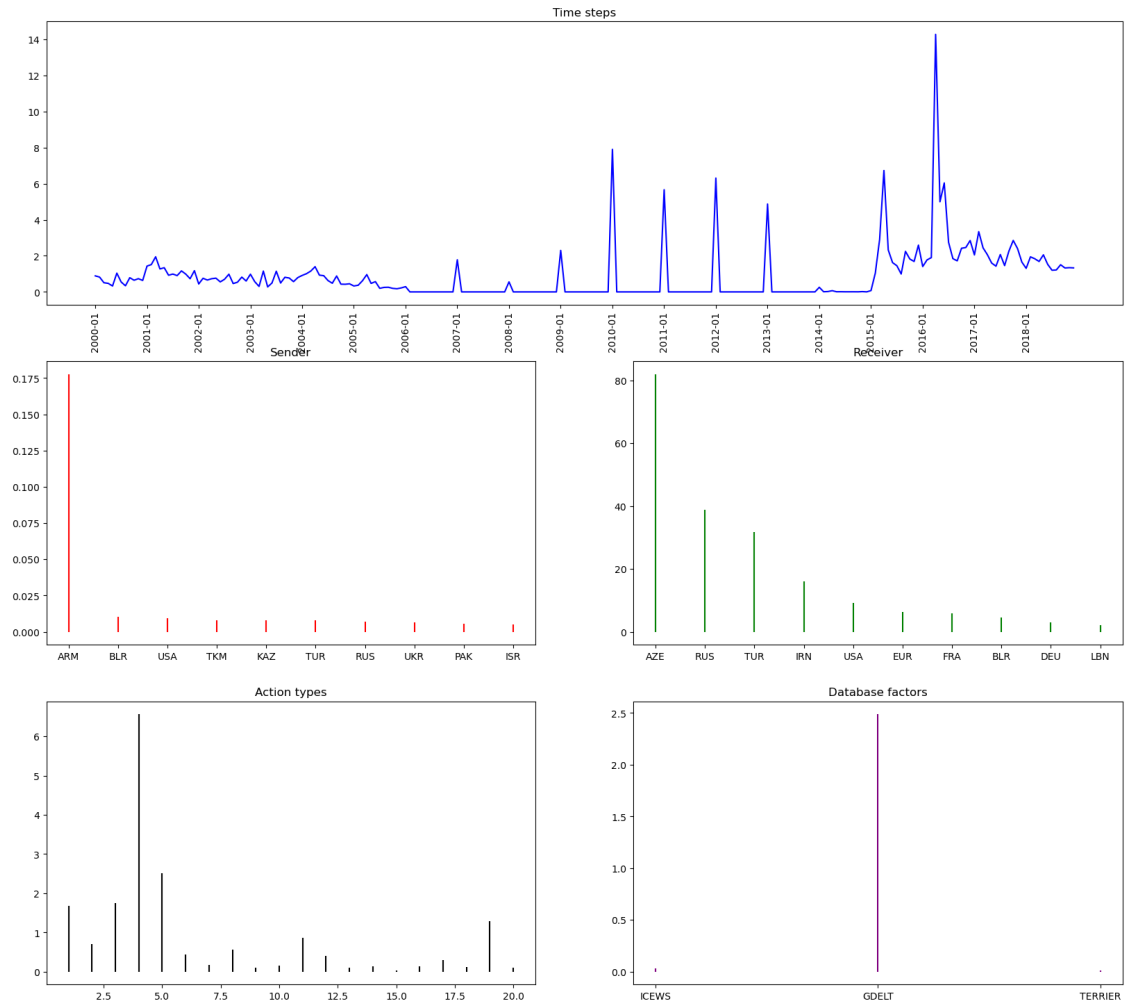


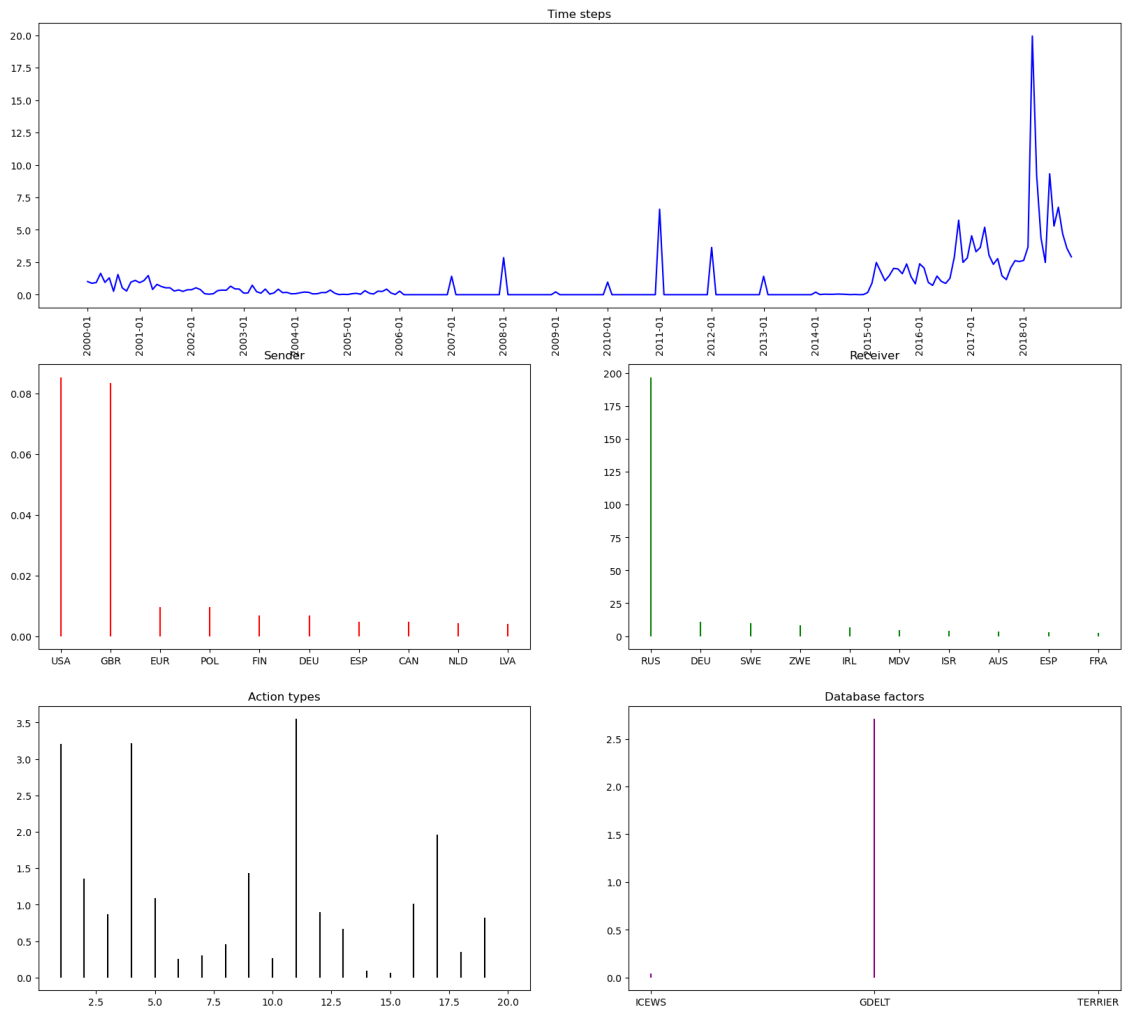
Component 35

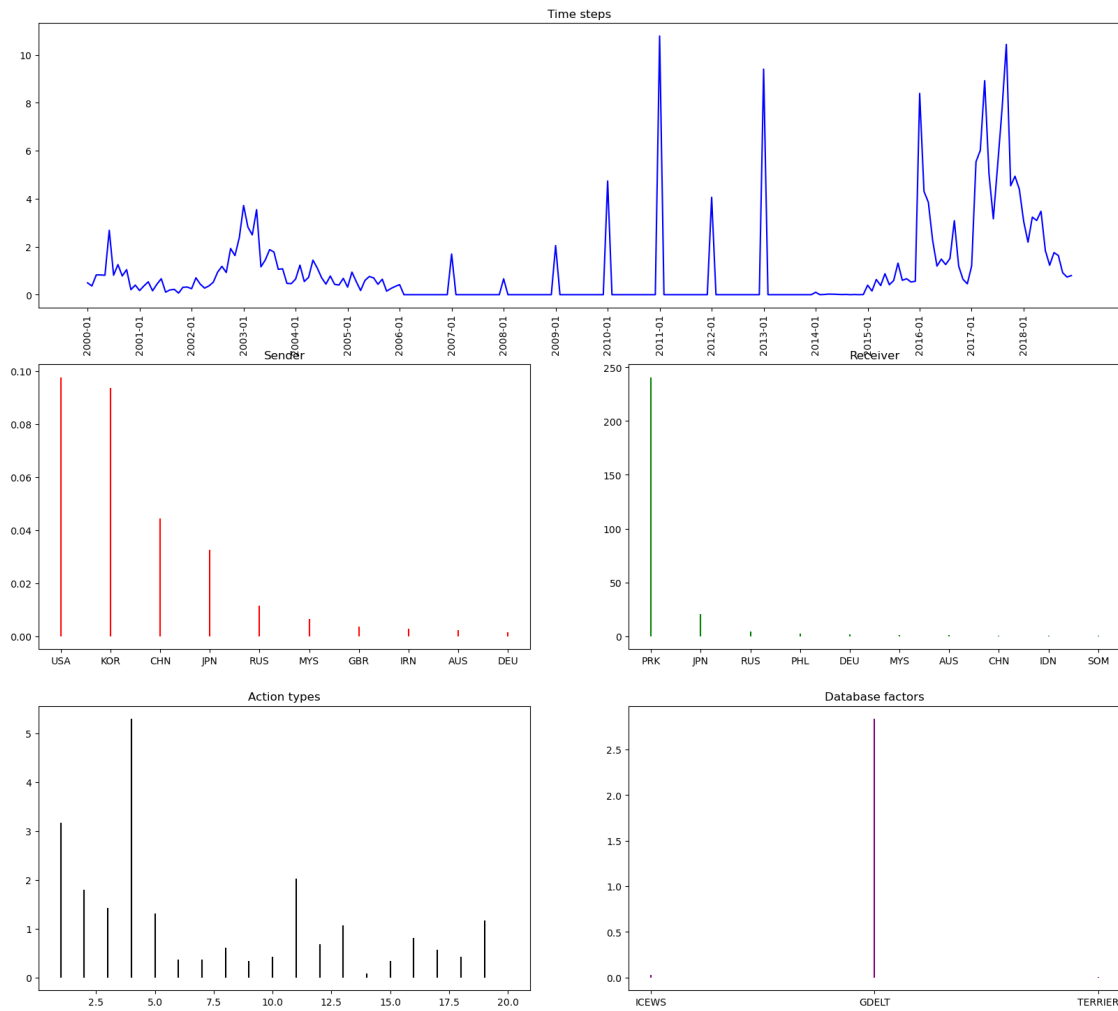


Component 22

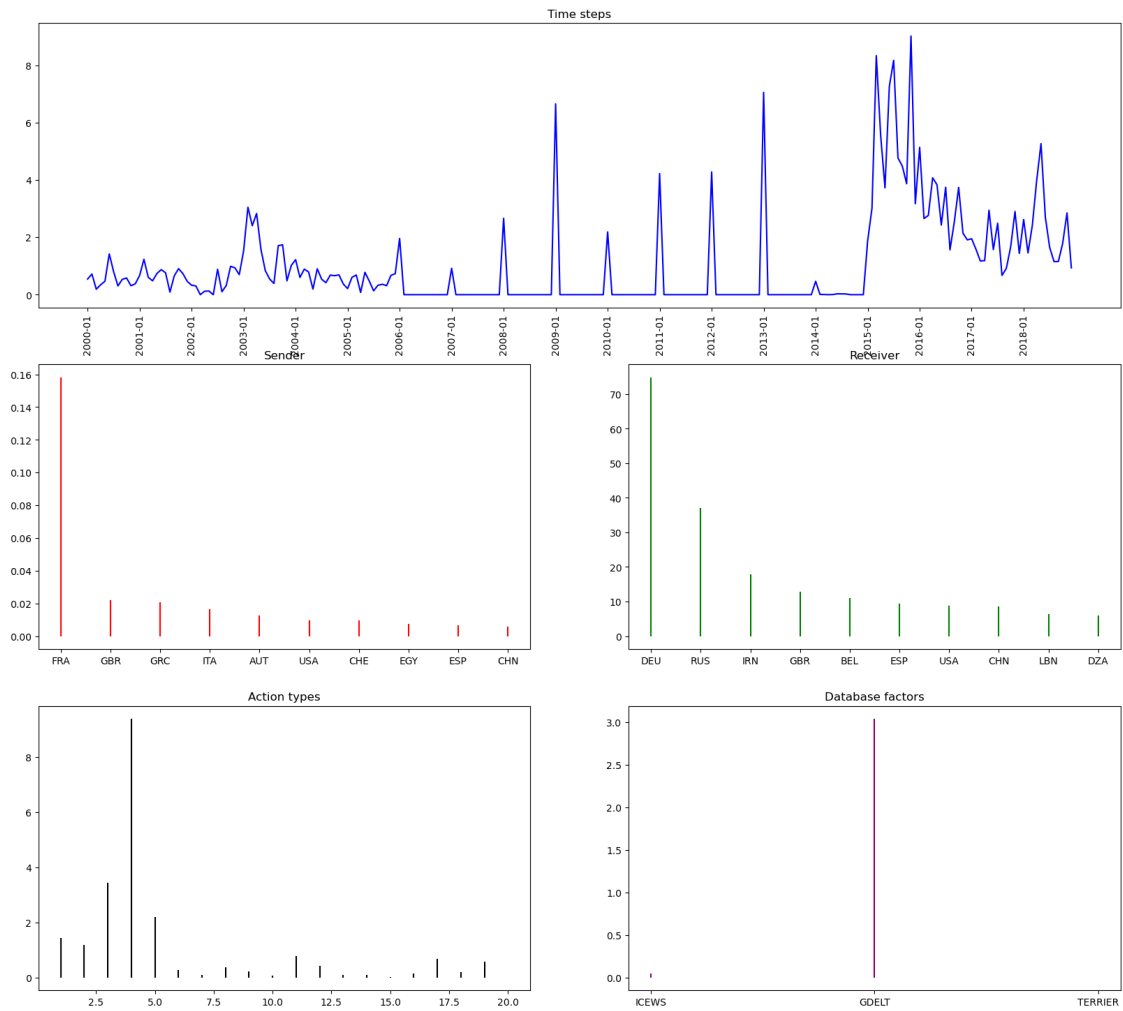




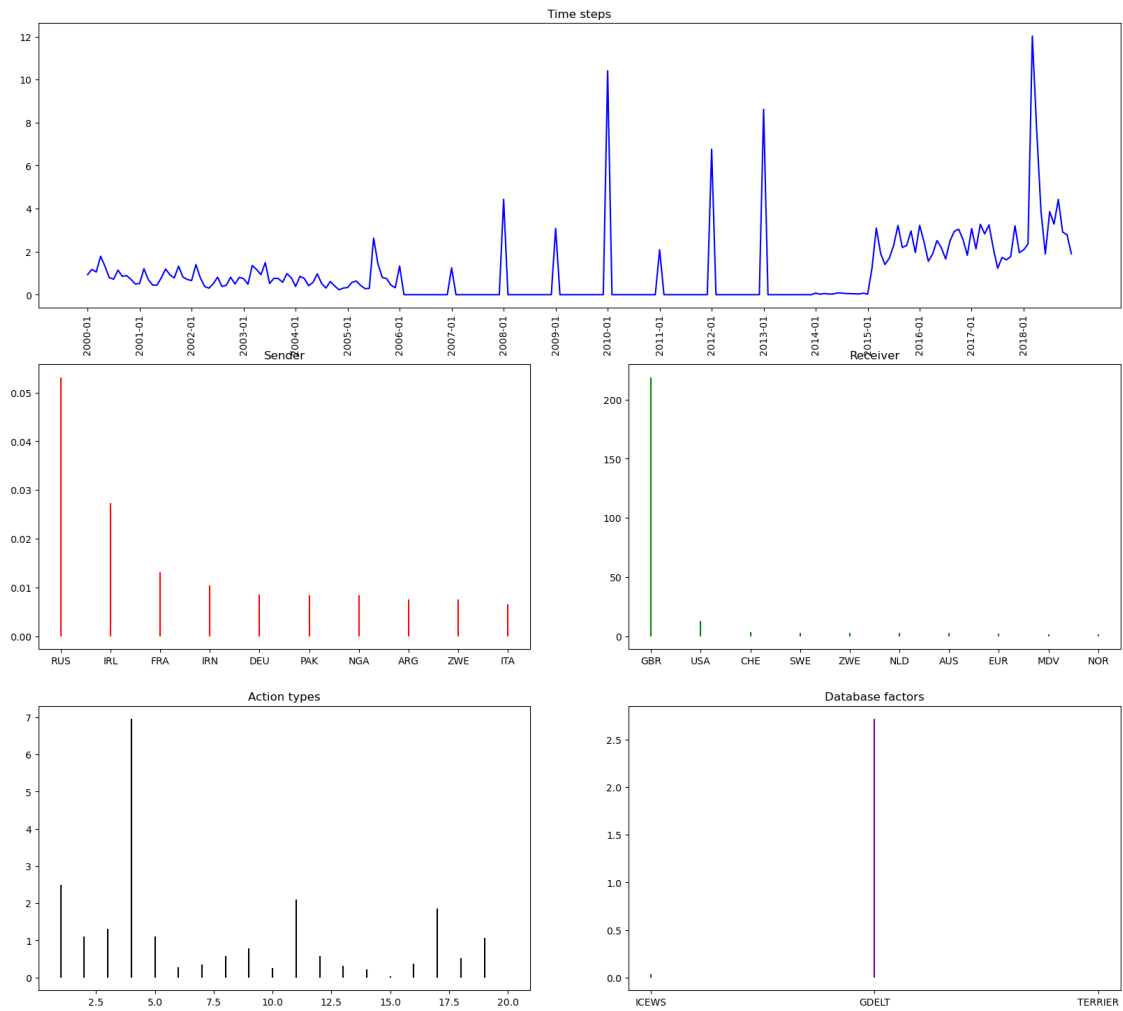


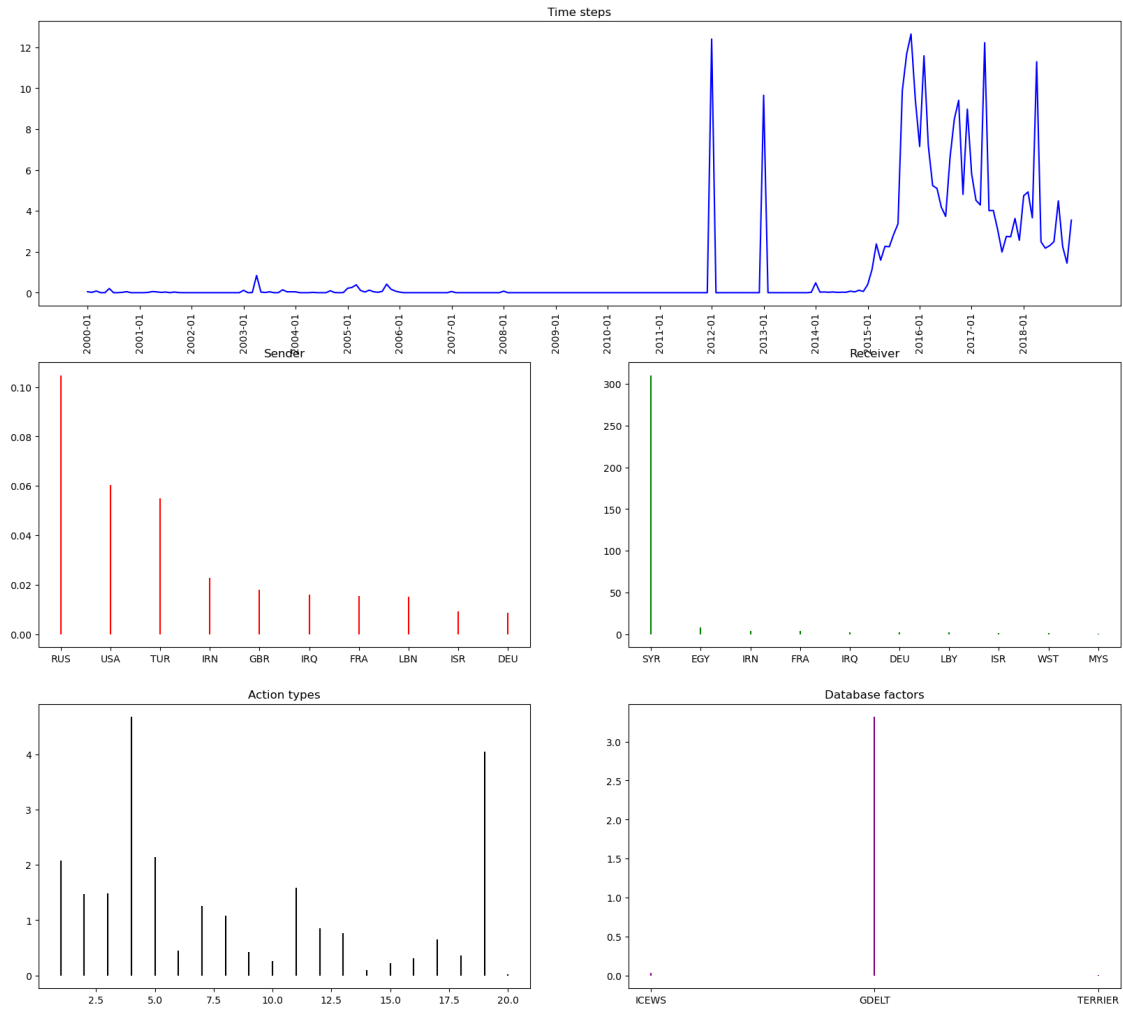


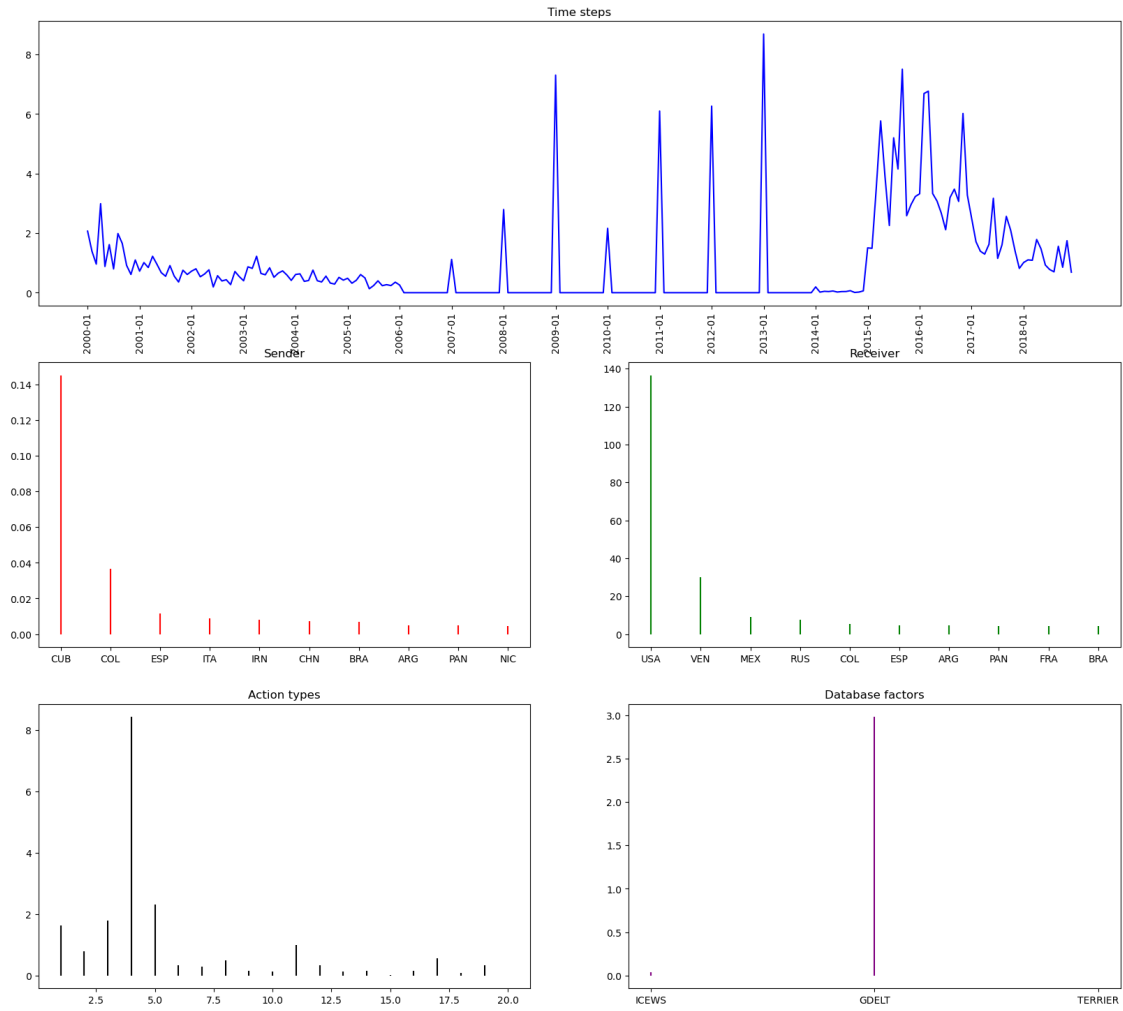
Component 60

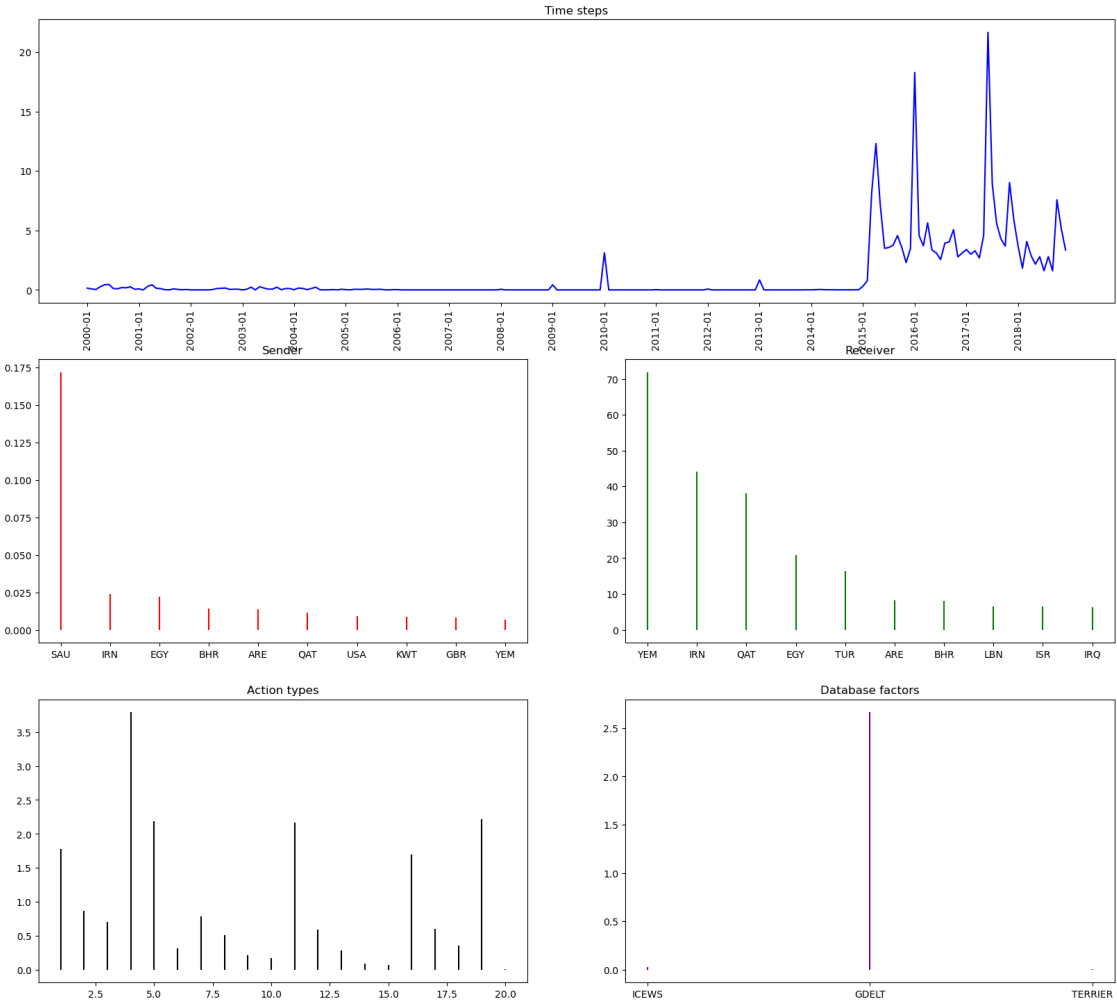


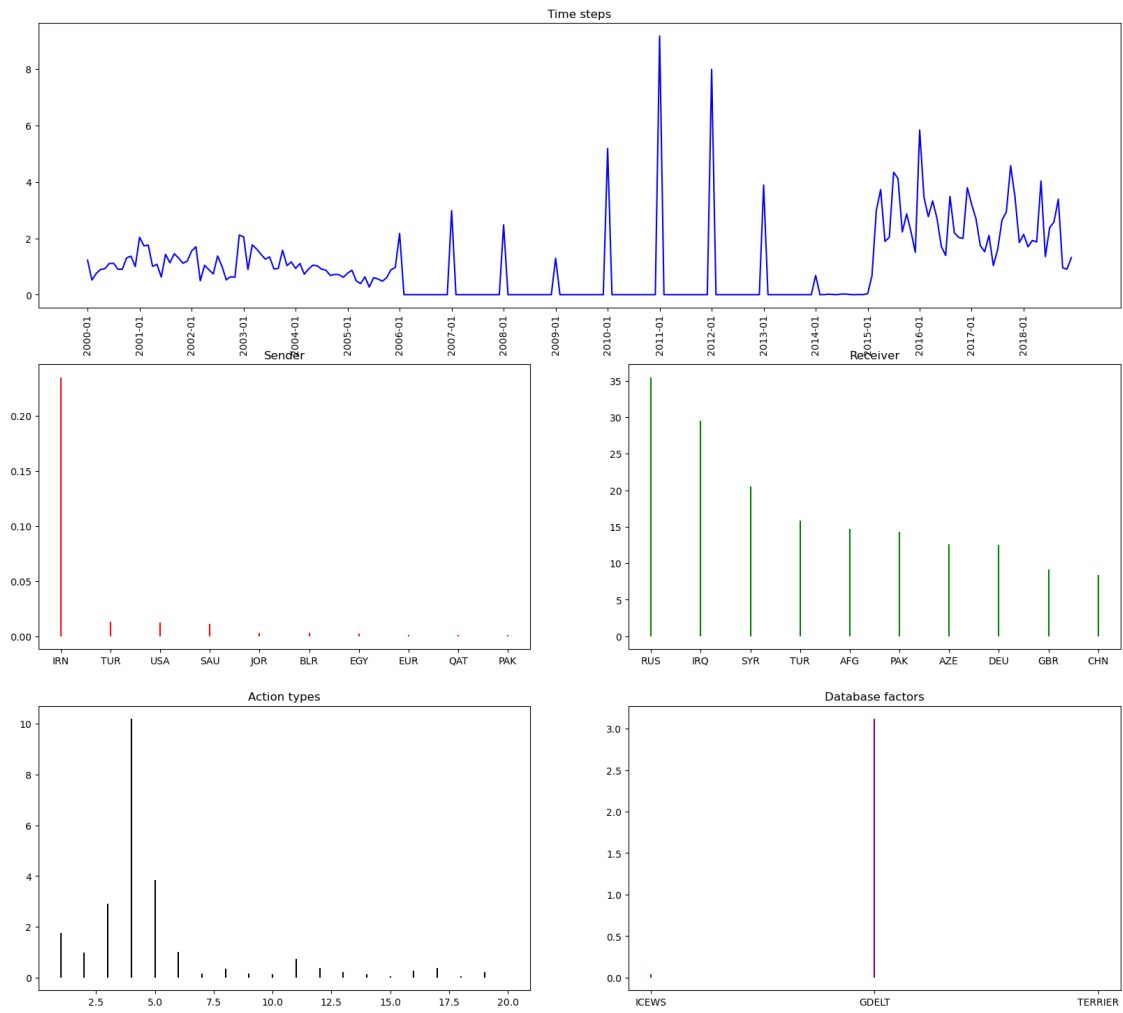
Component 3



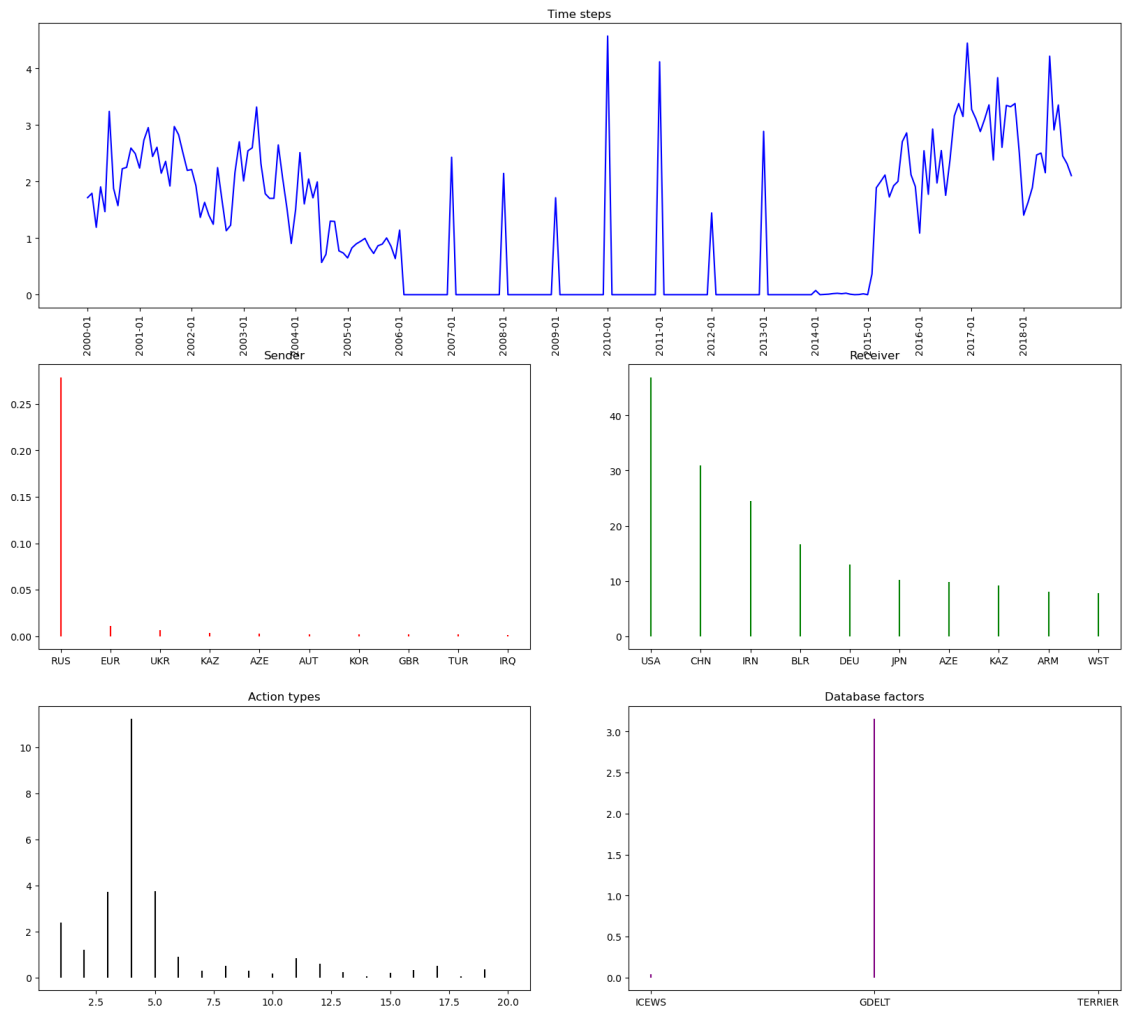


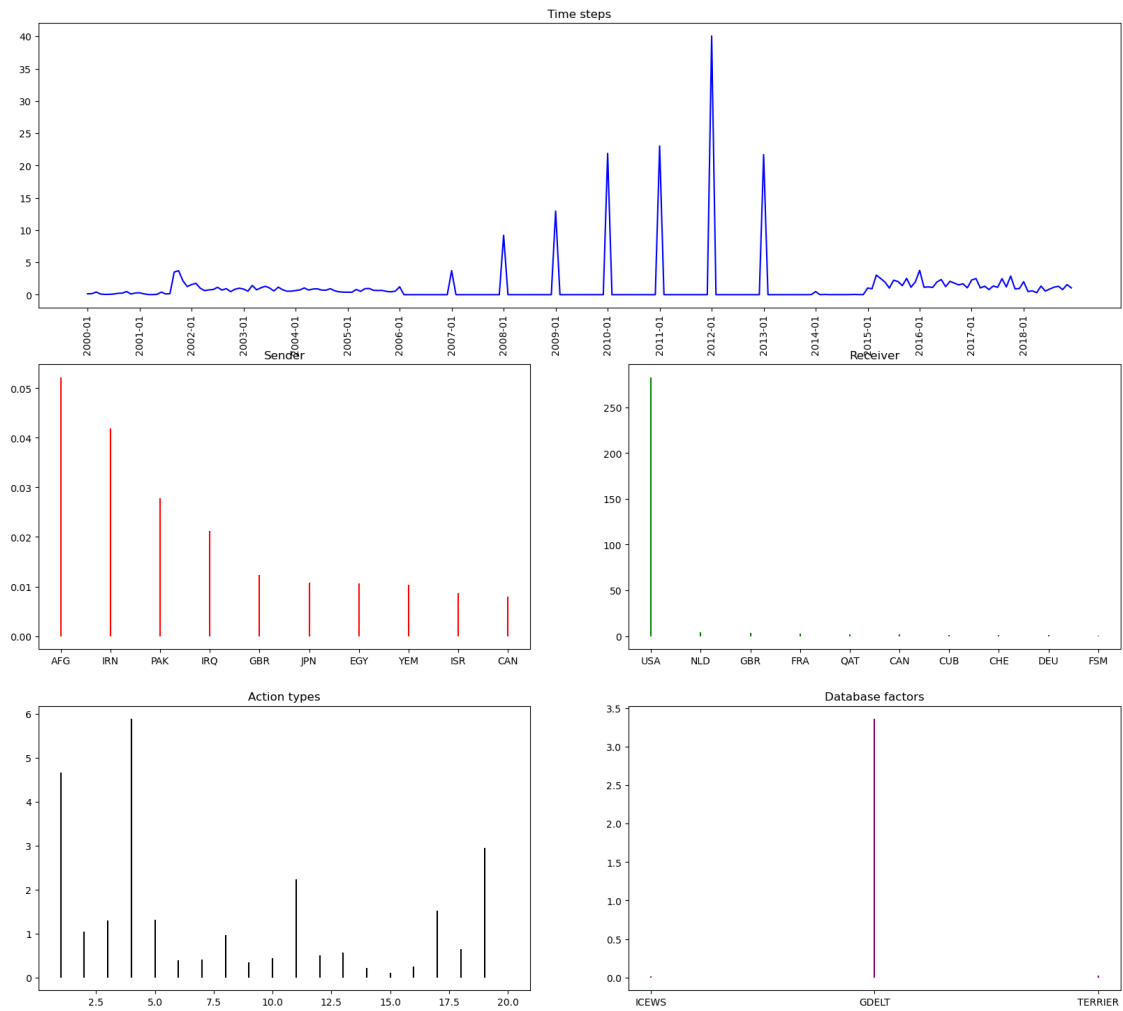




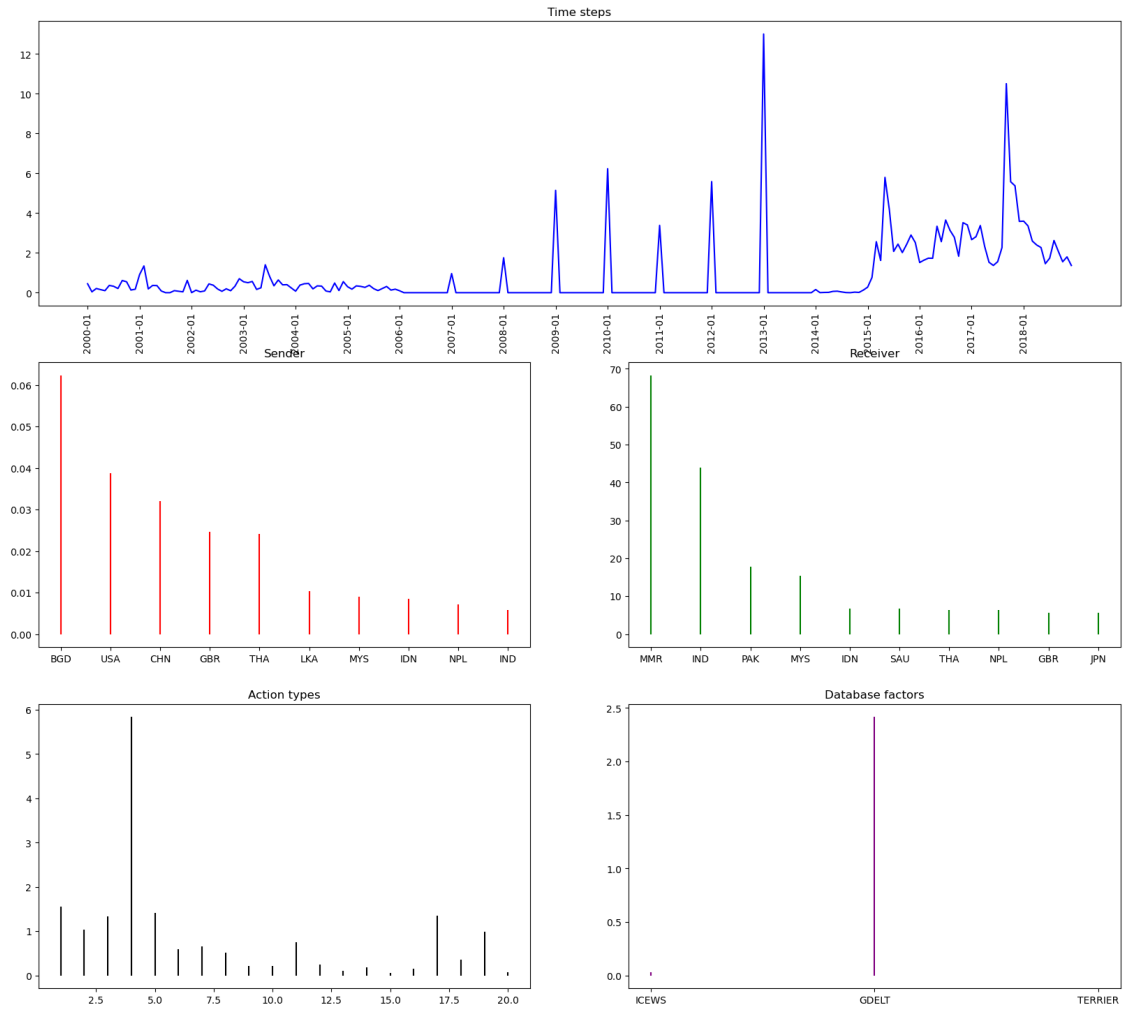


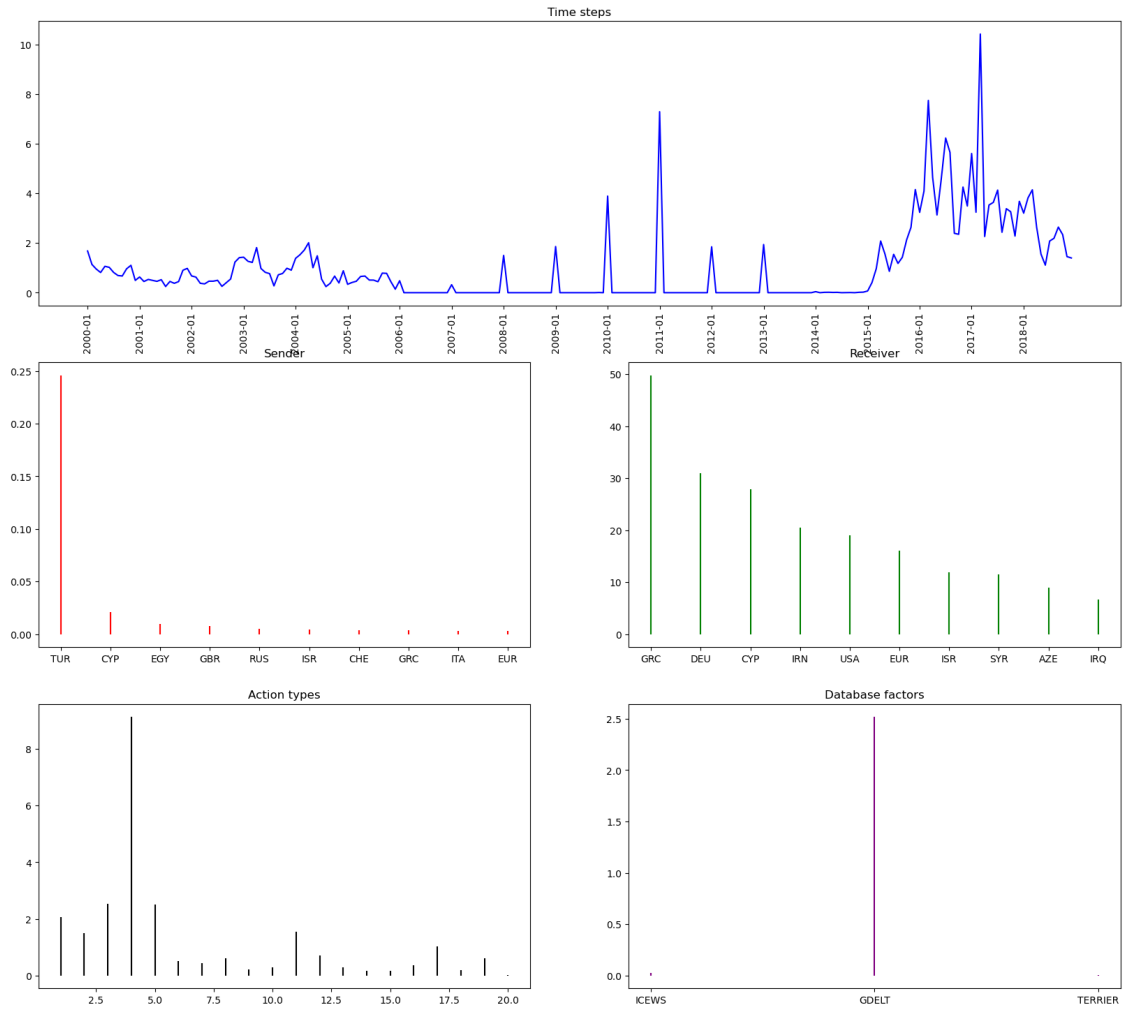
Component 9



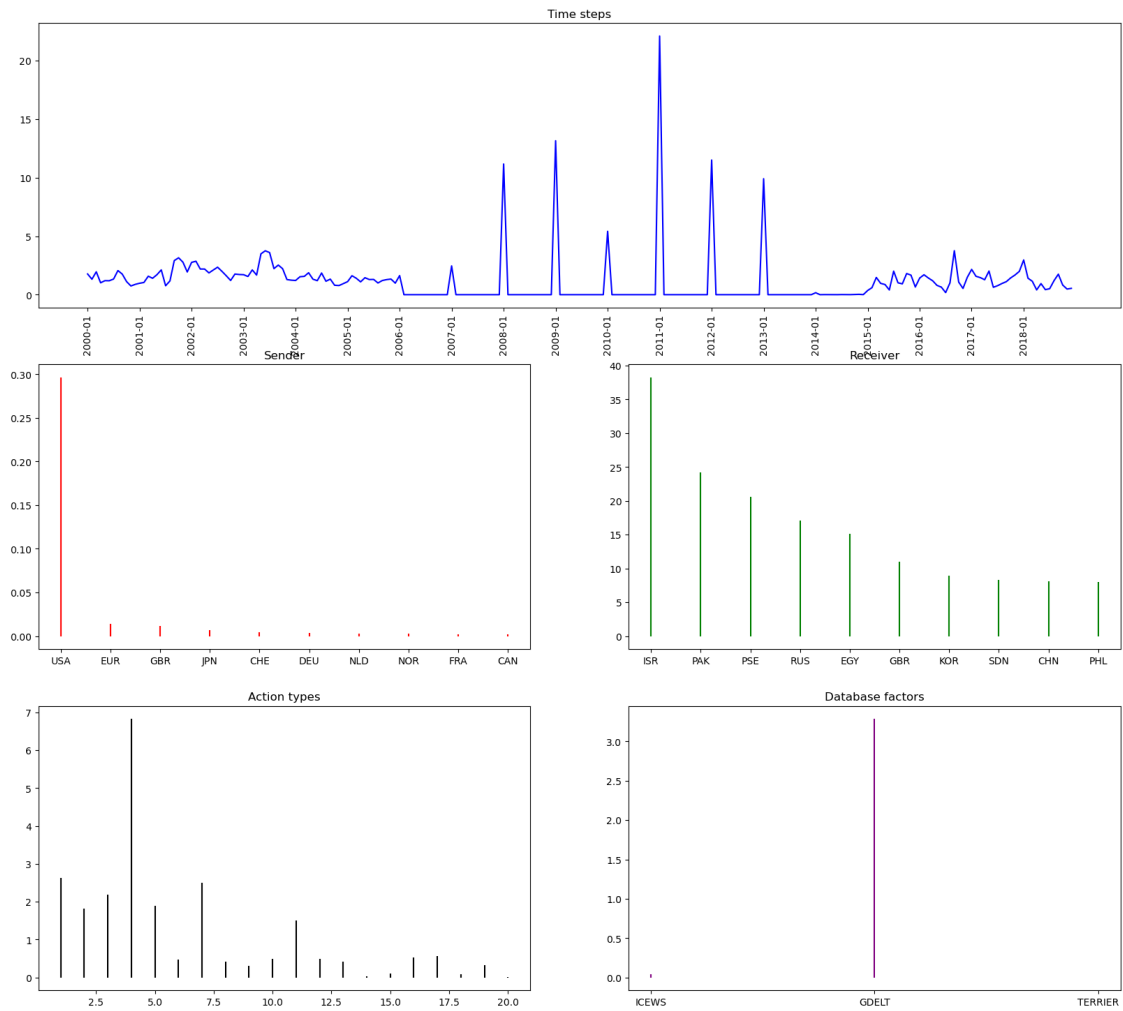


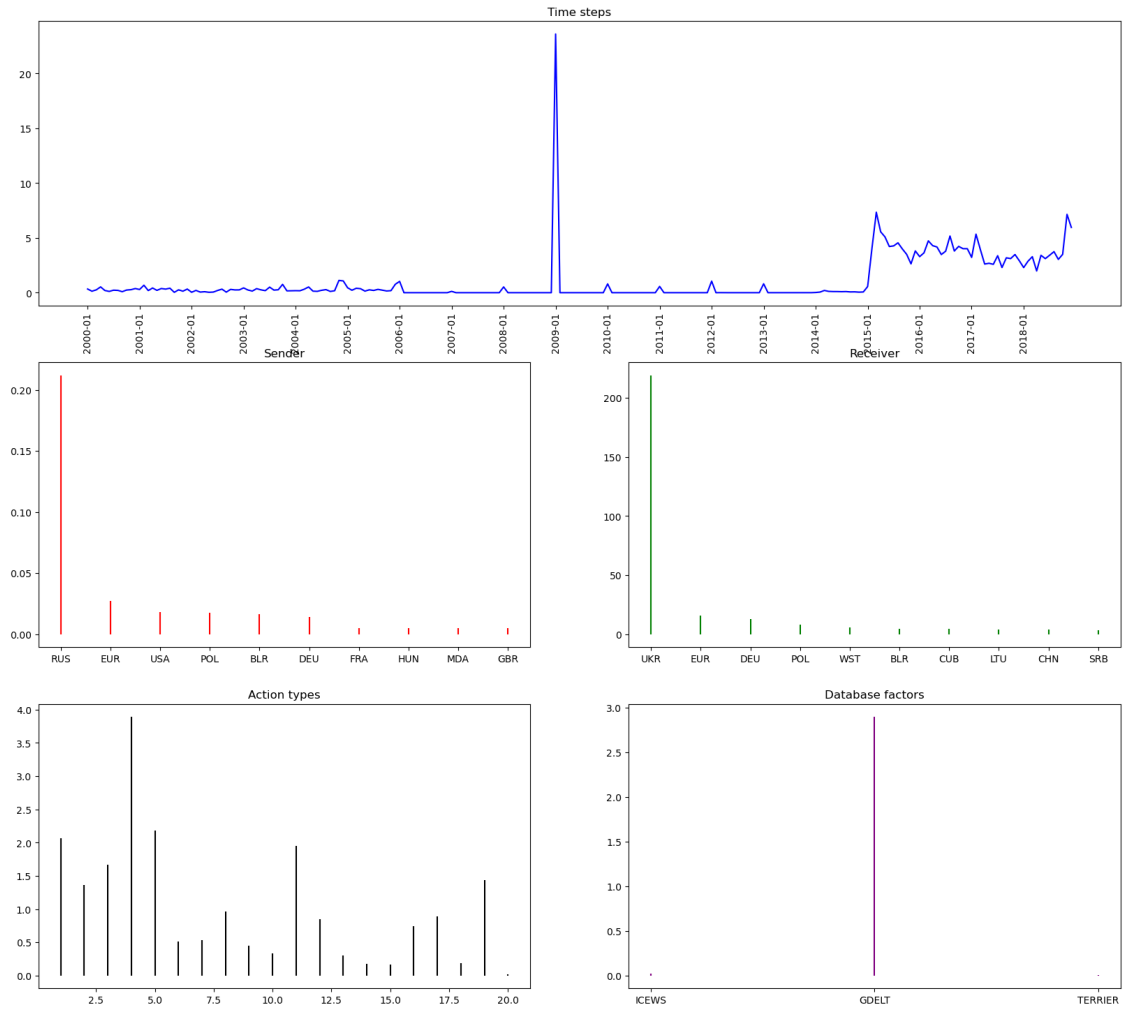
Component 2



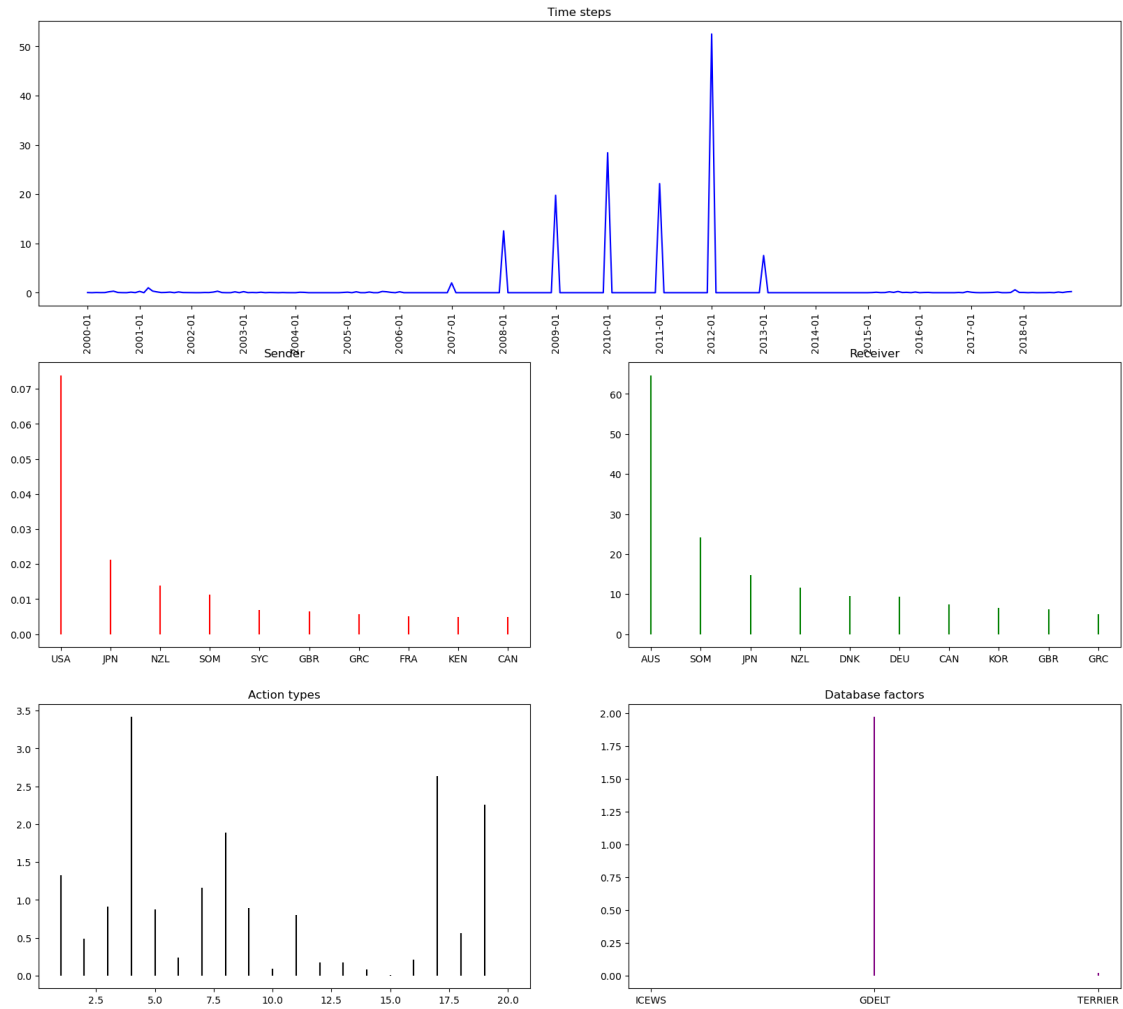


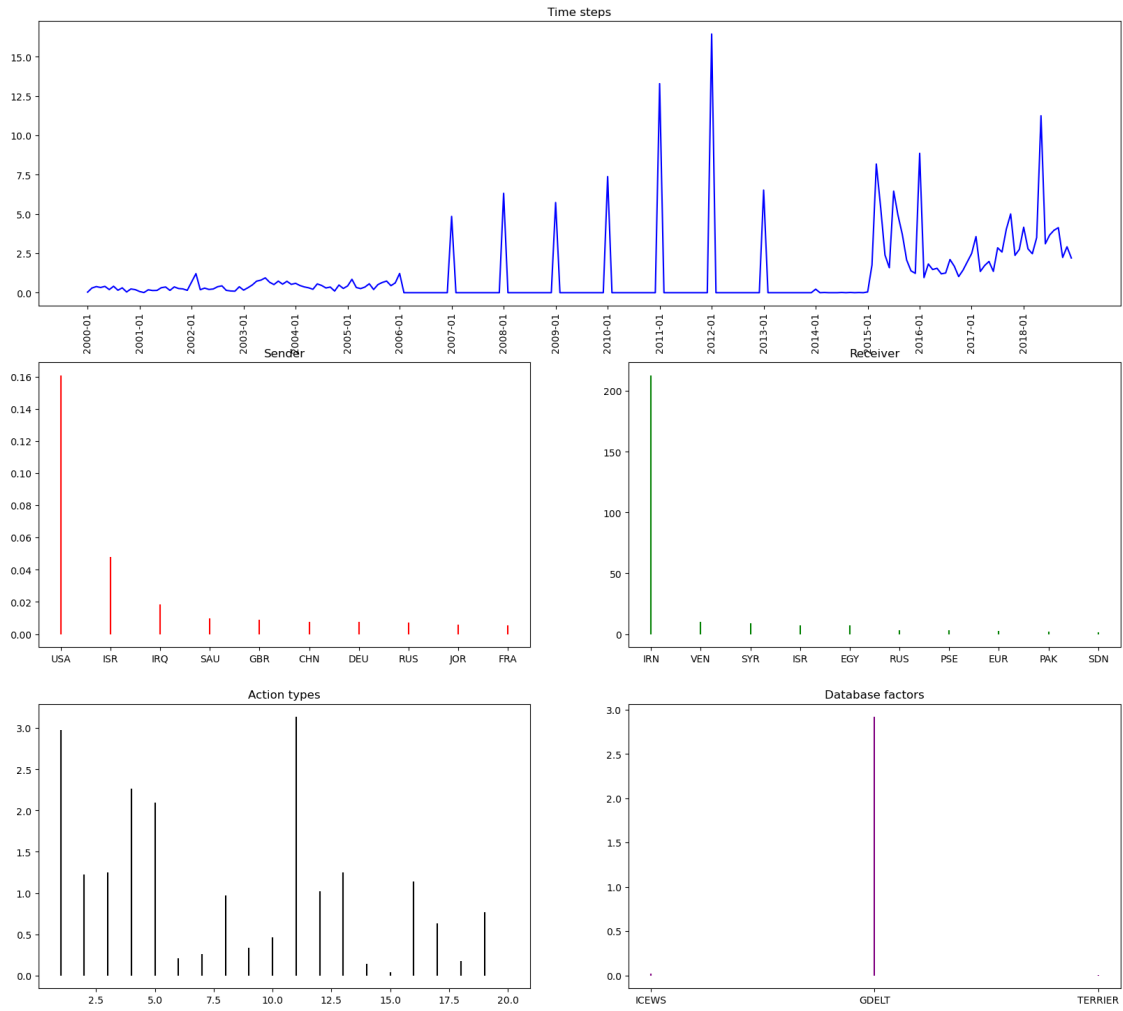
Component 11

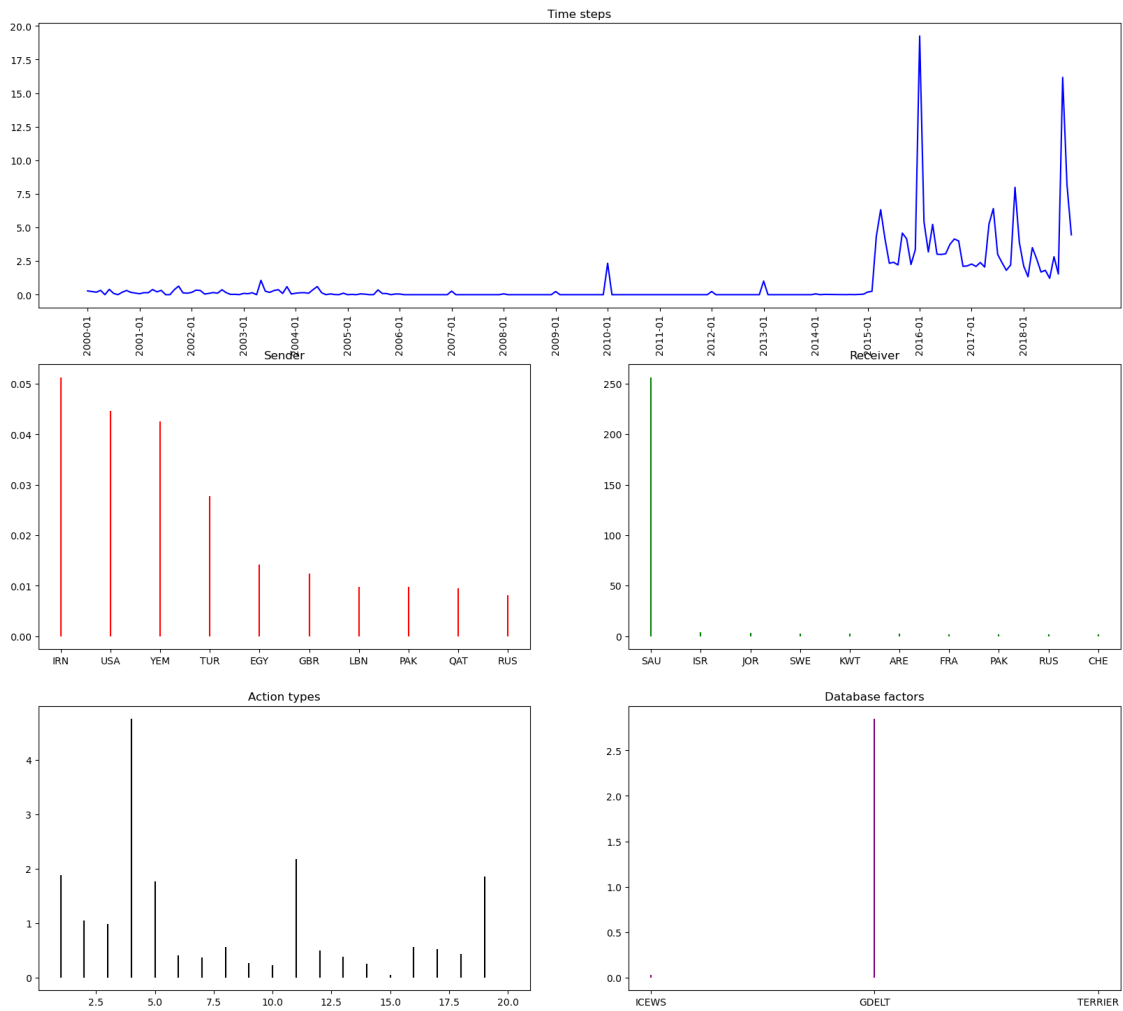


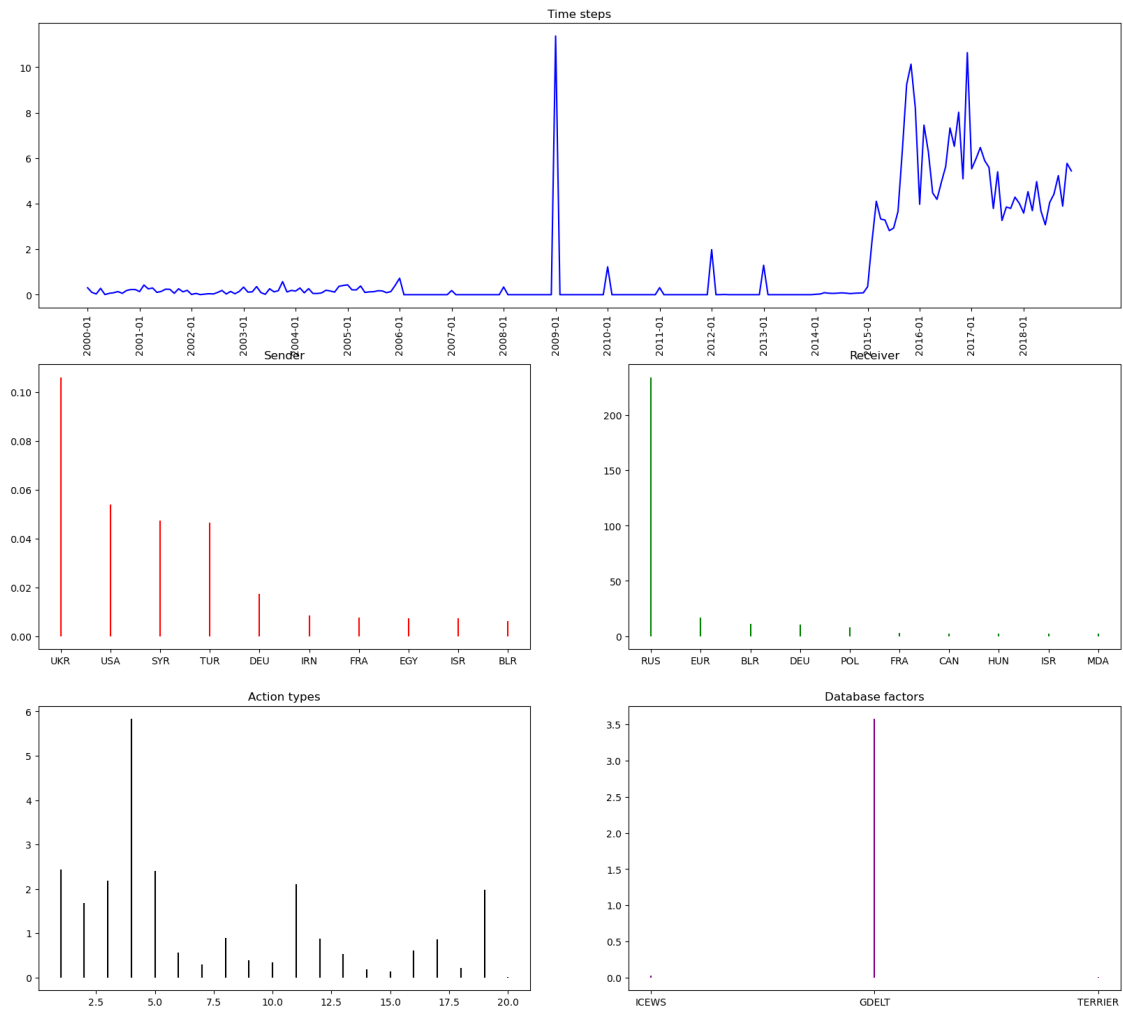


Component 21

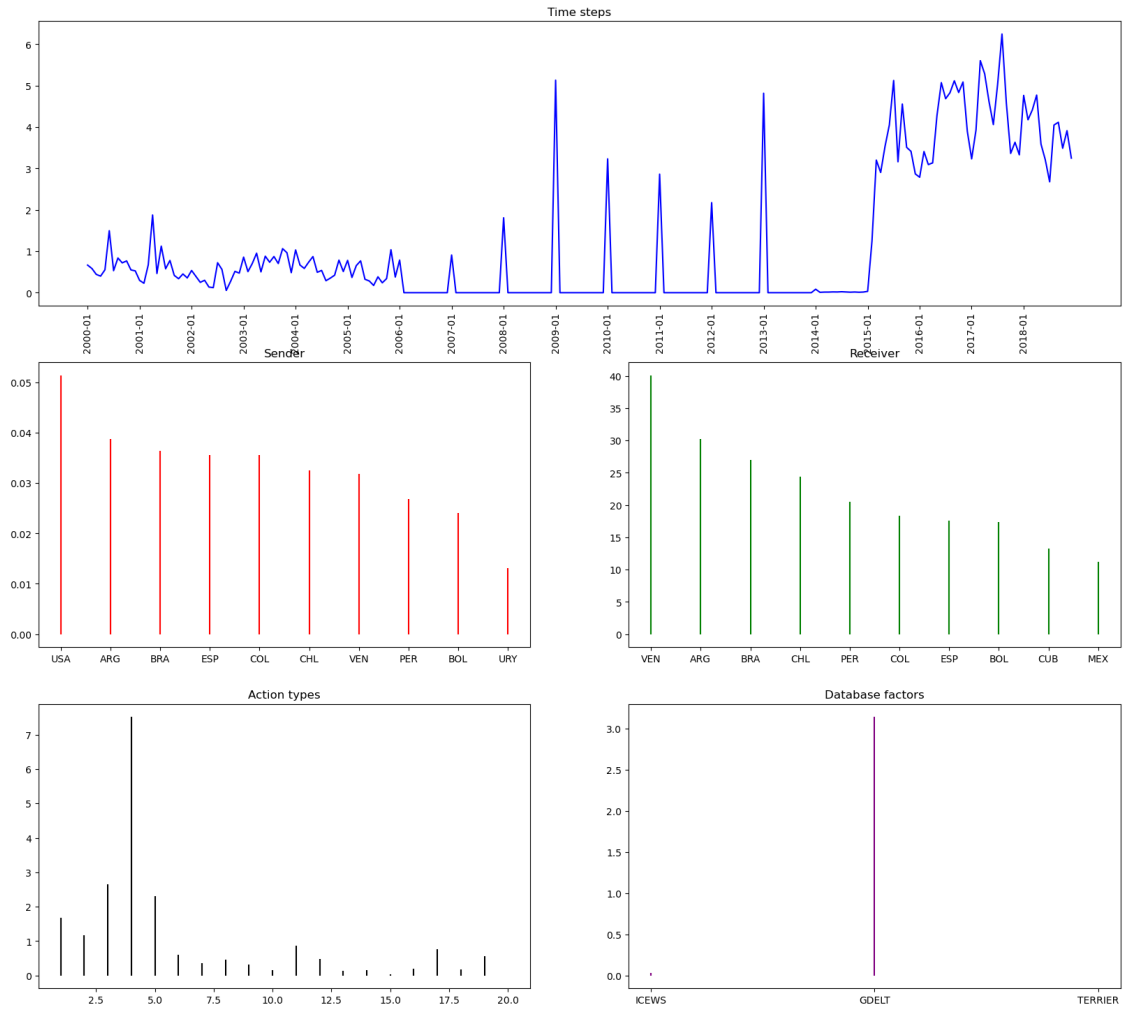


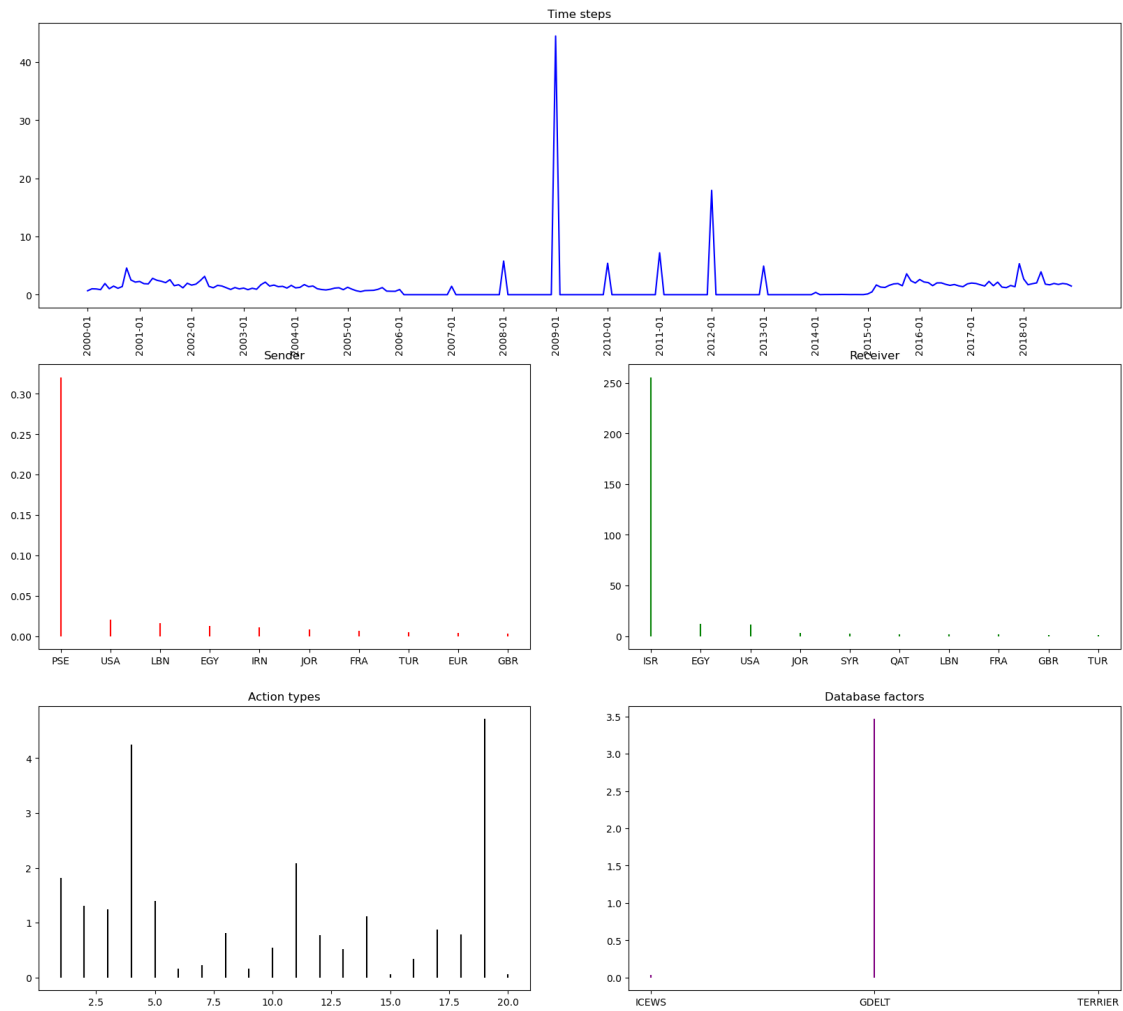


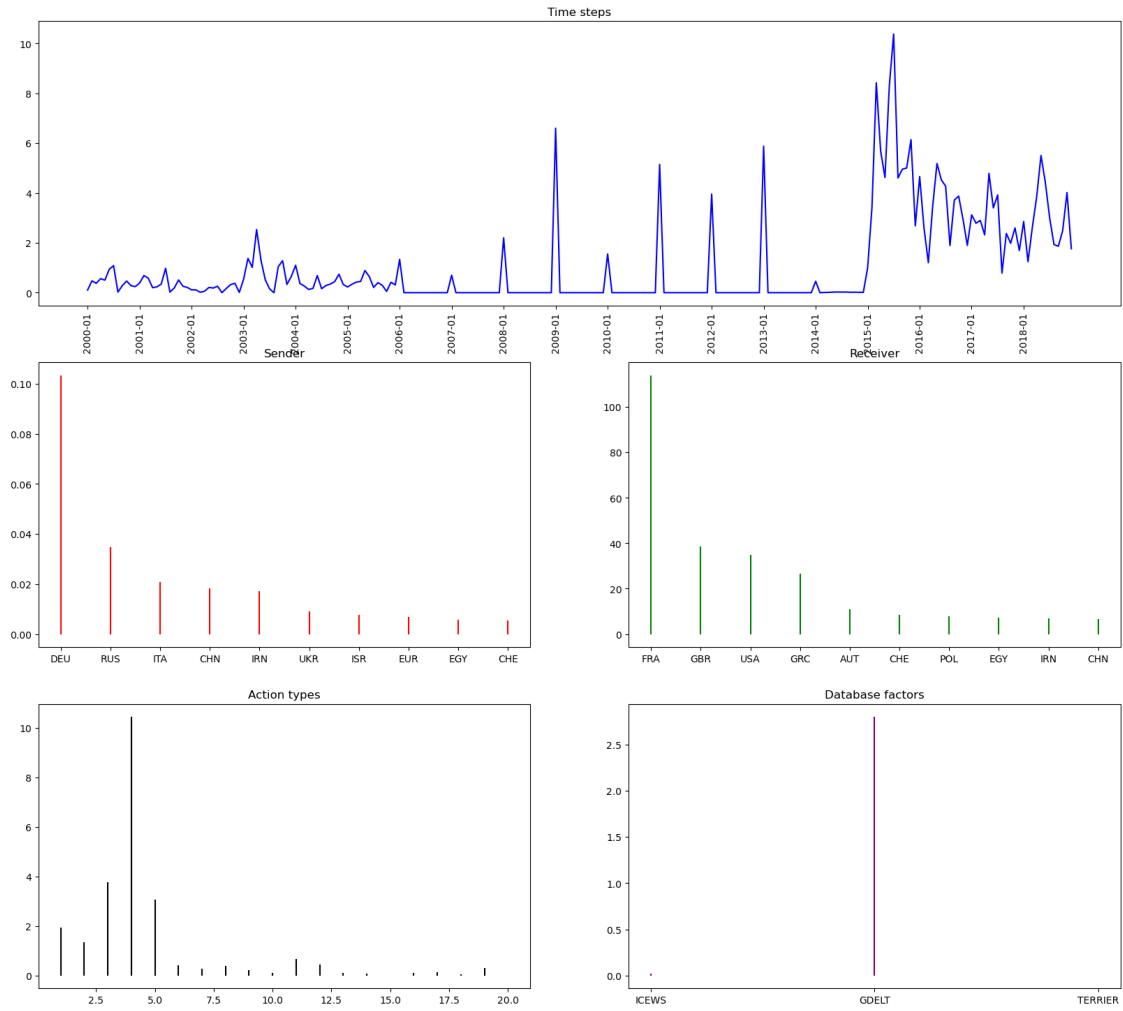


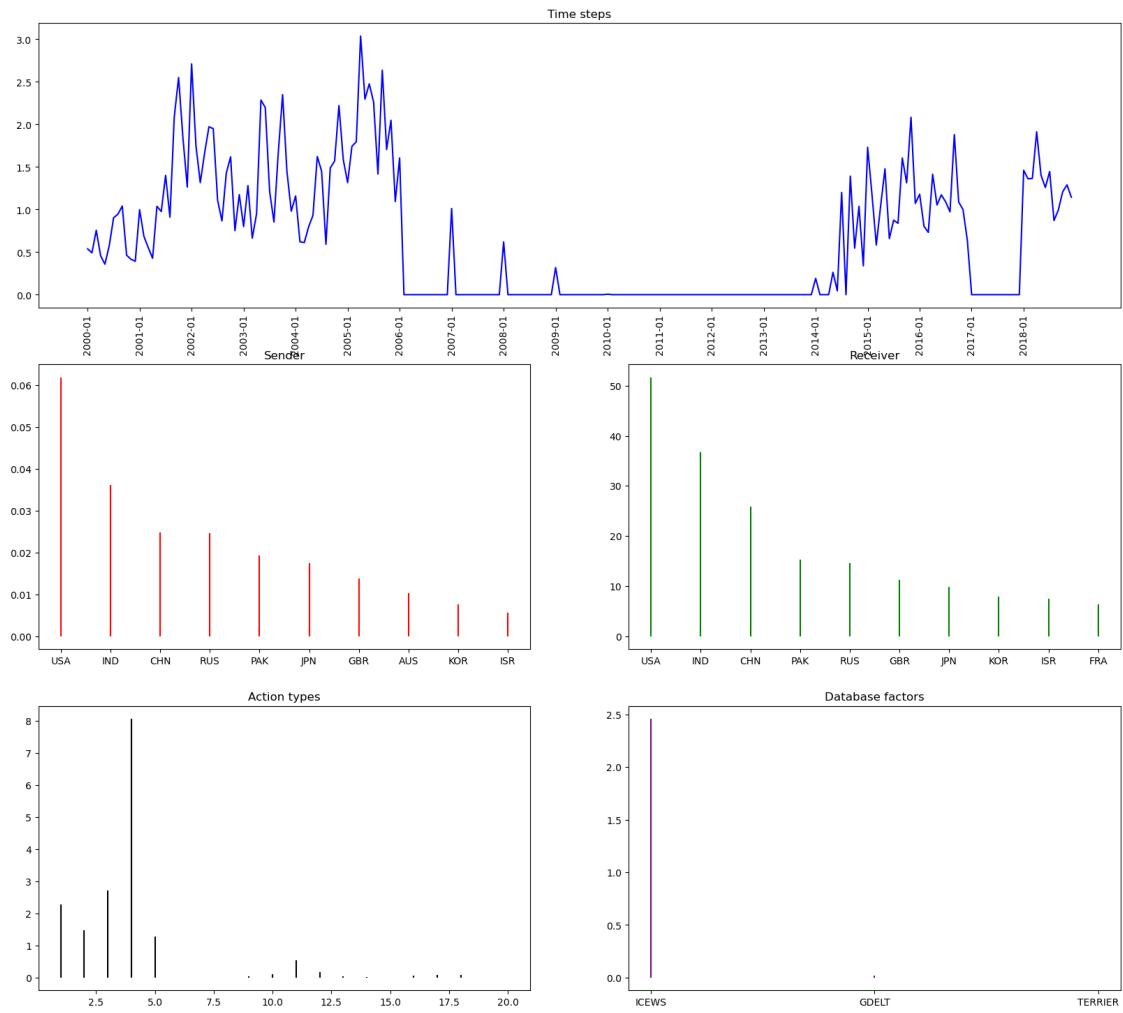


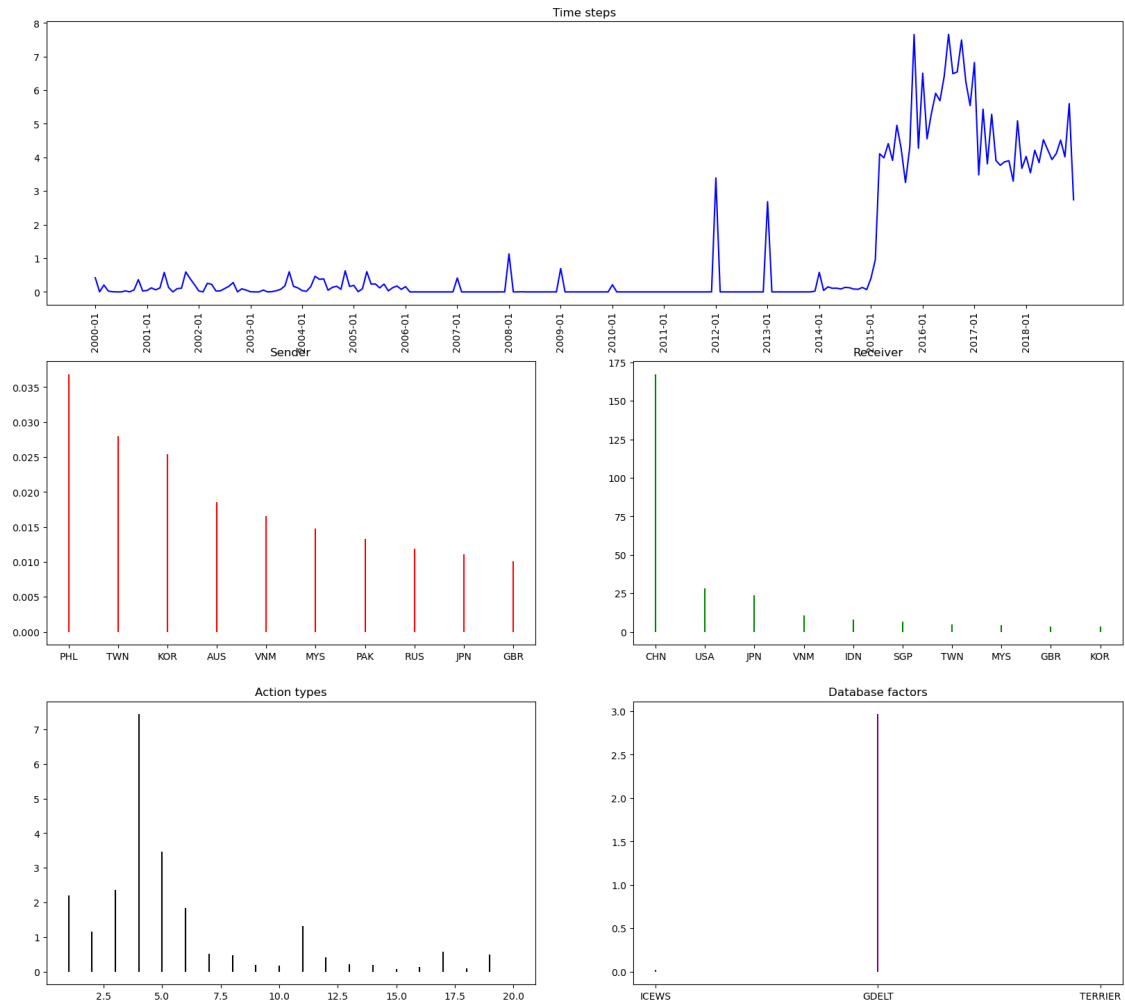
Component 1



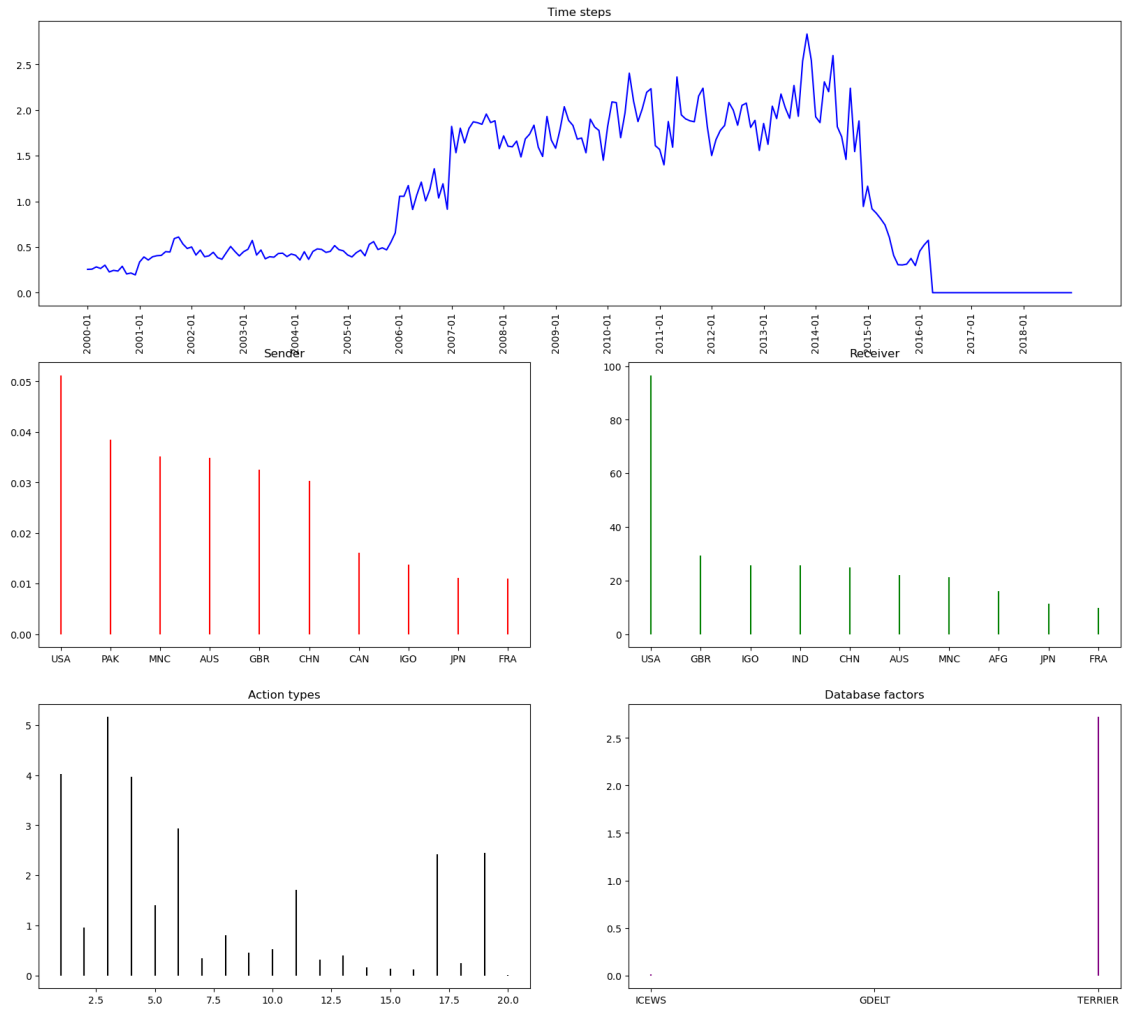


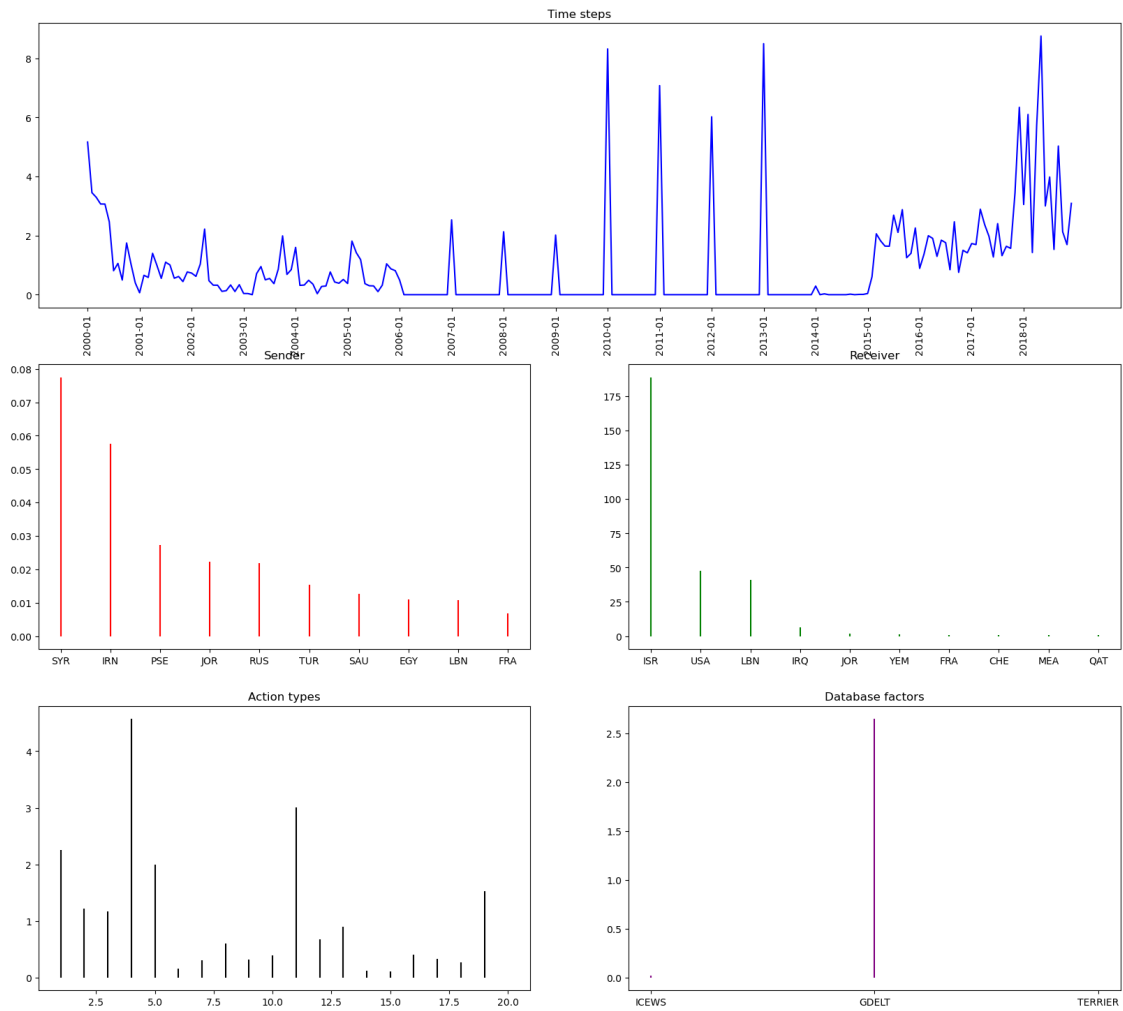




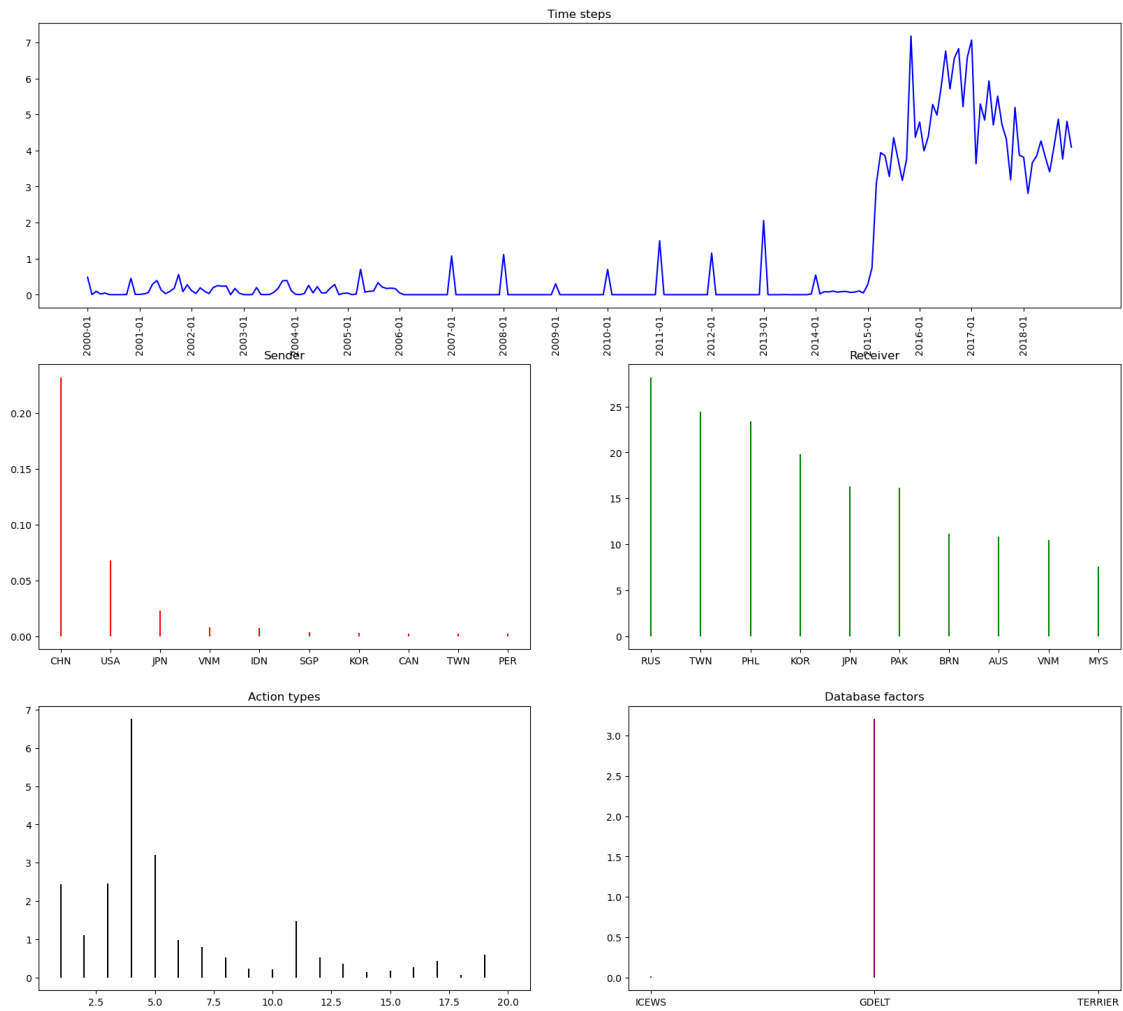


Component 4

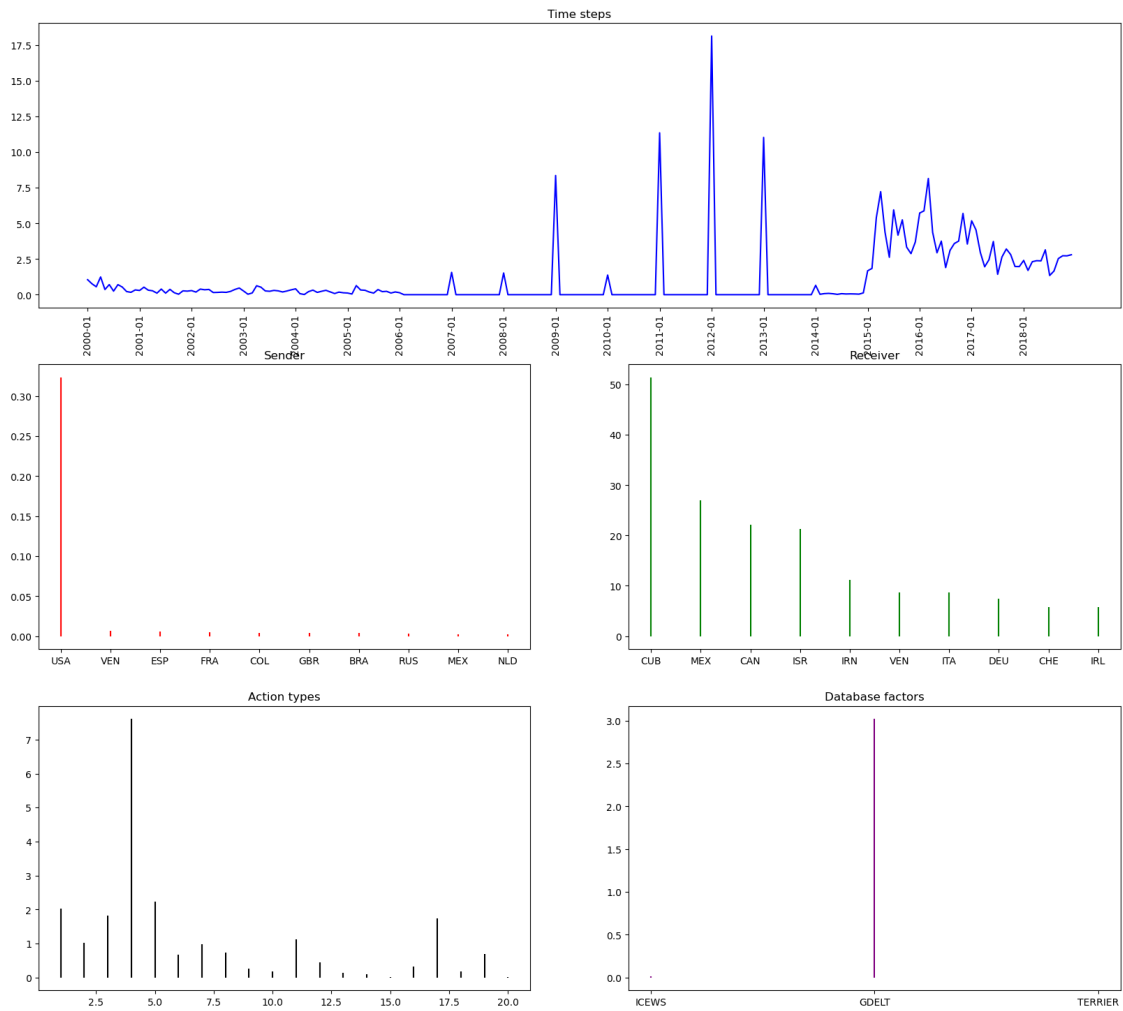


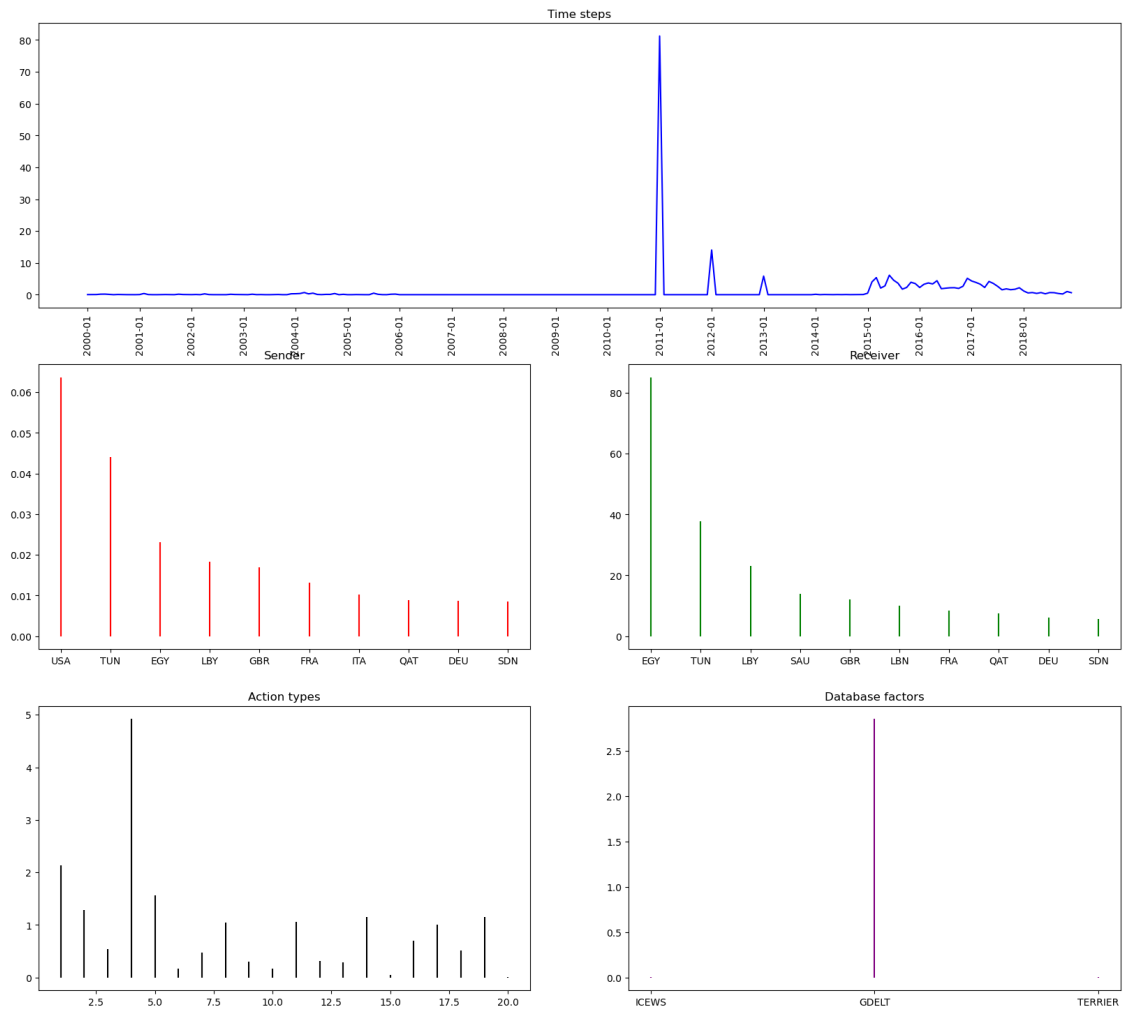


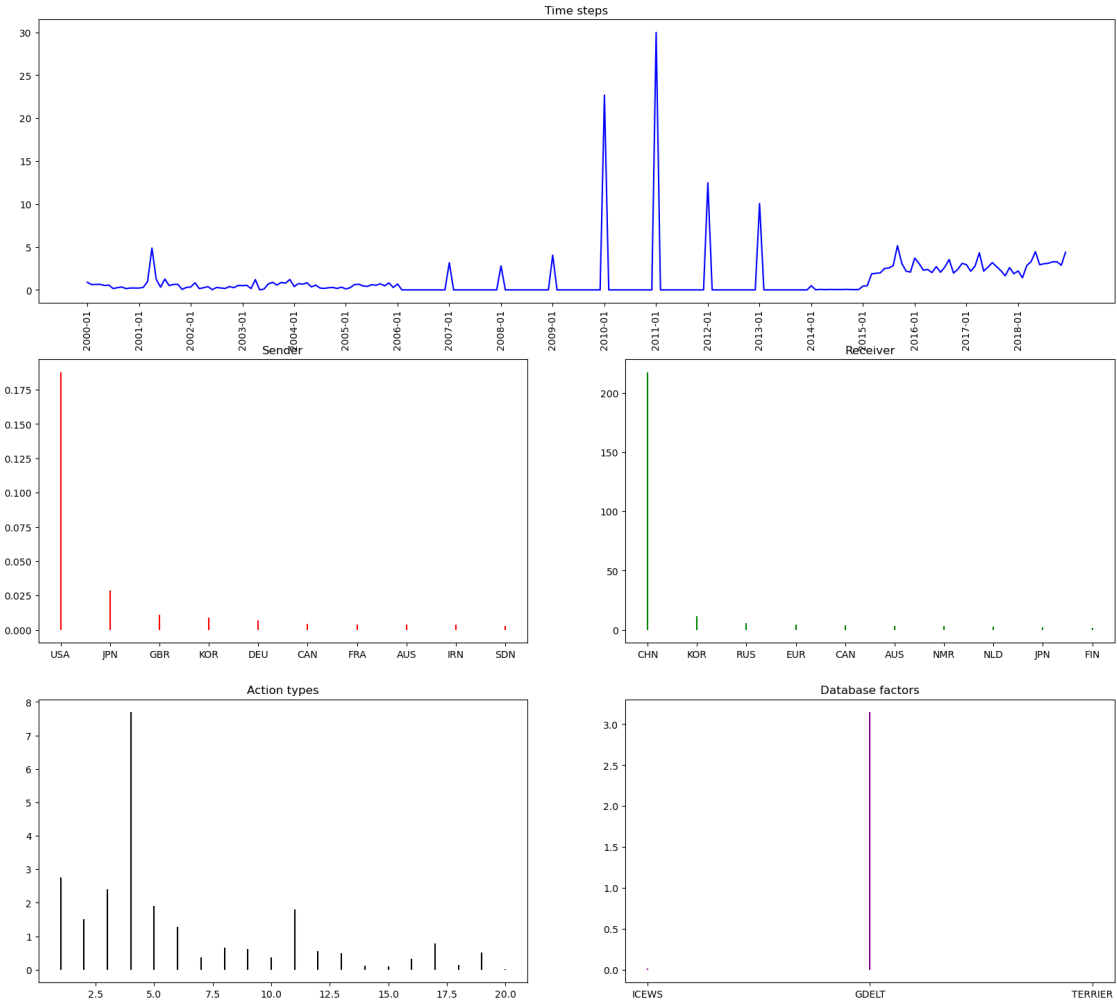
Component 6



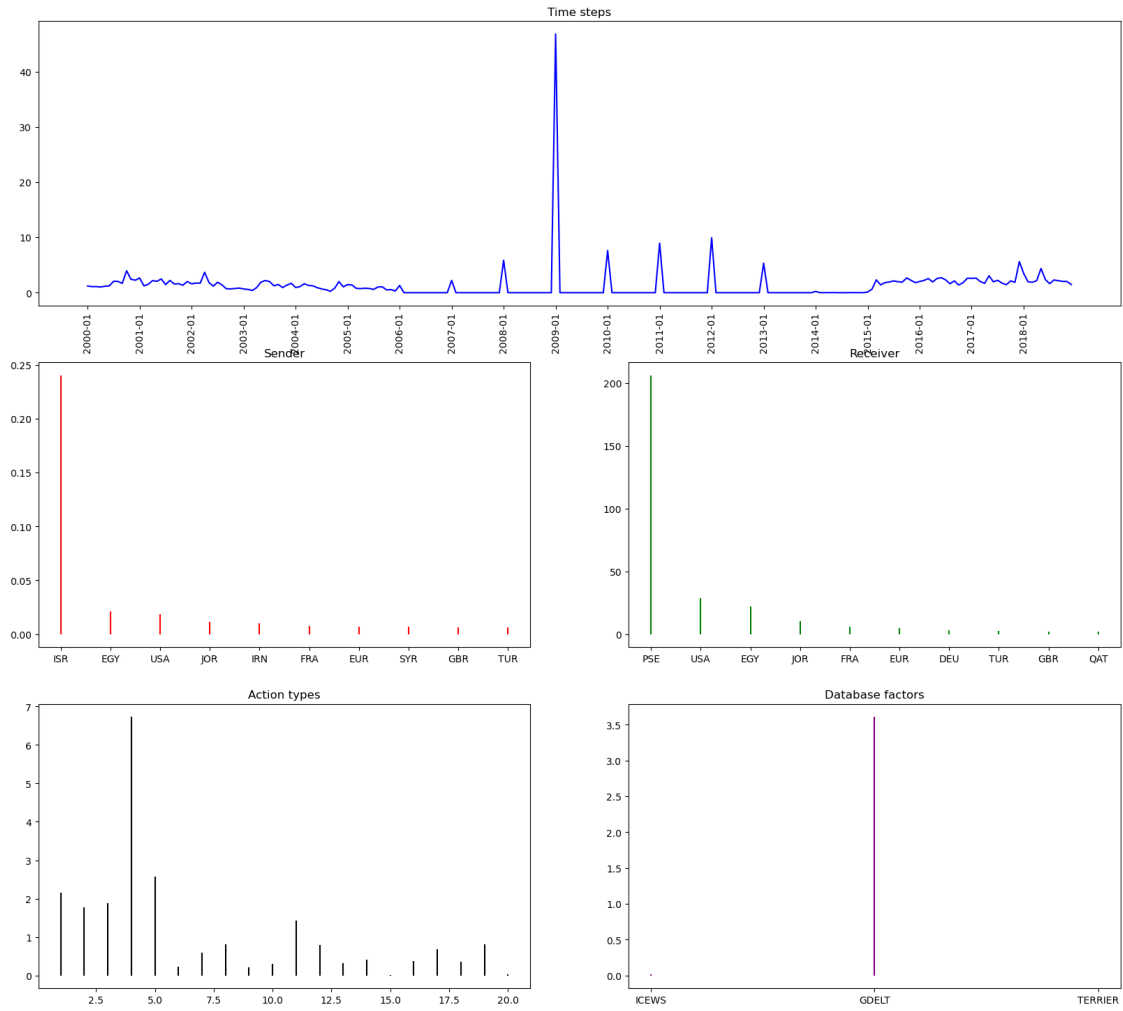
Component 66

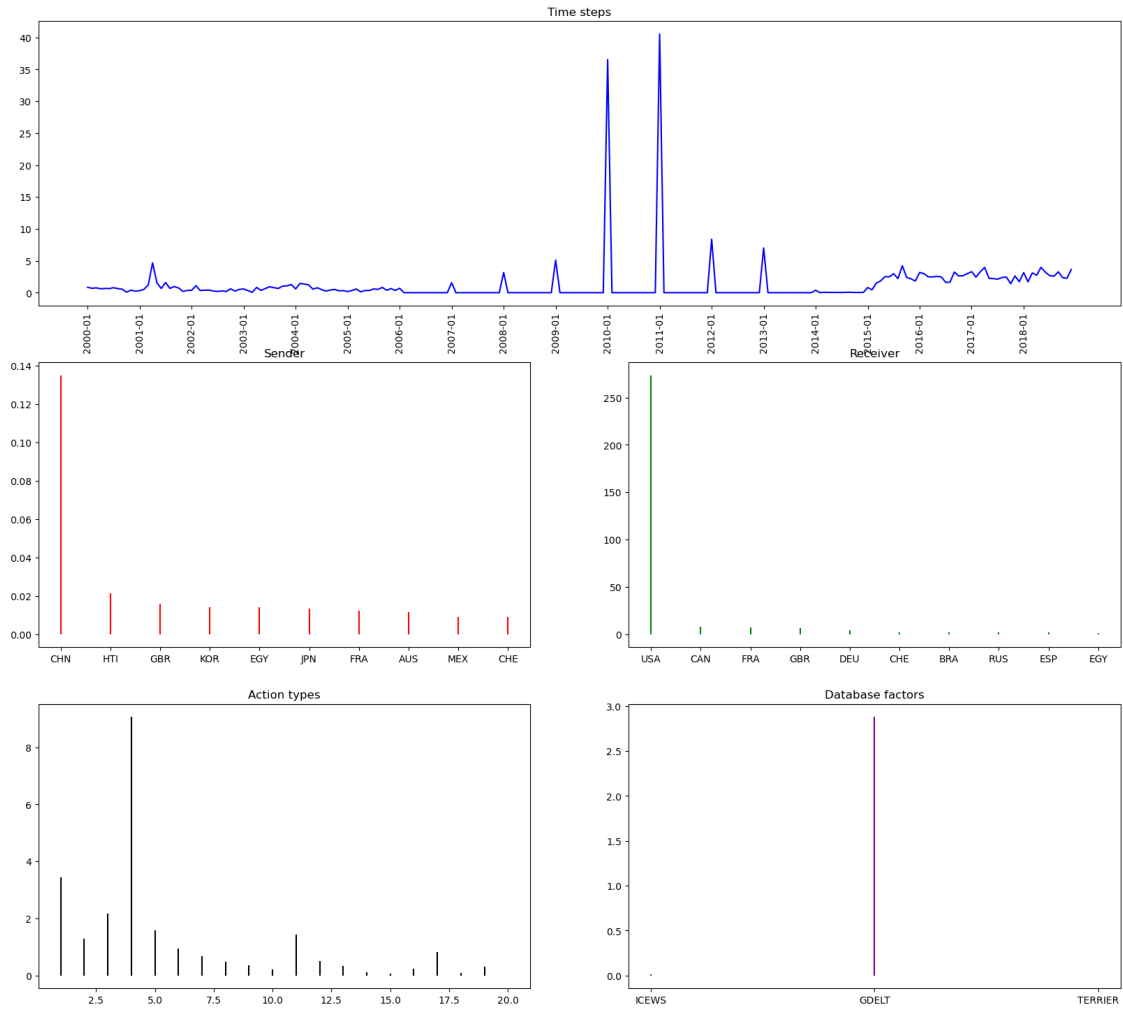


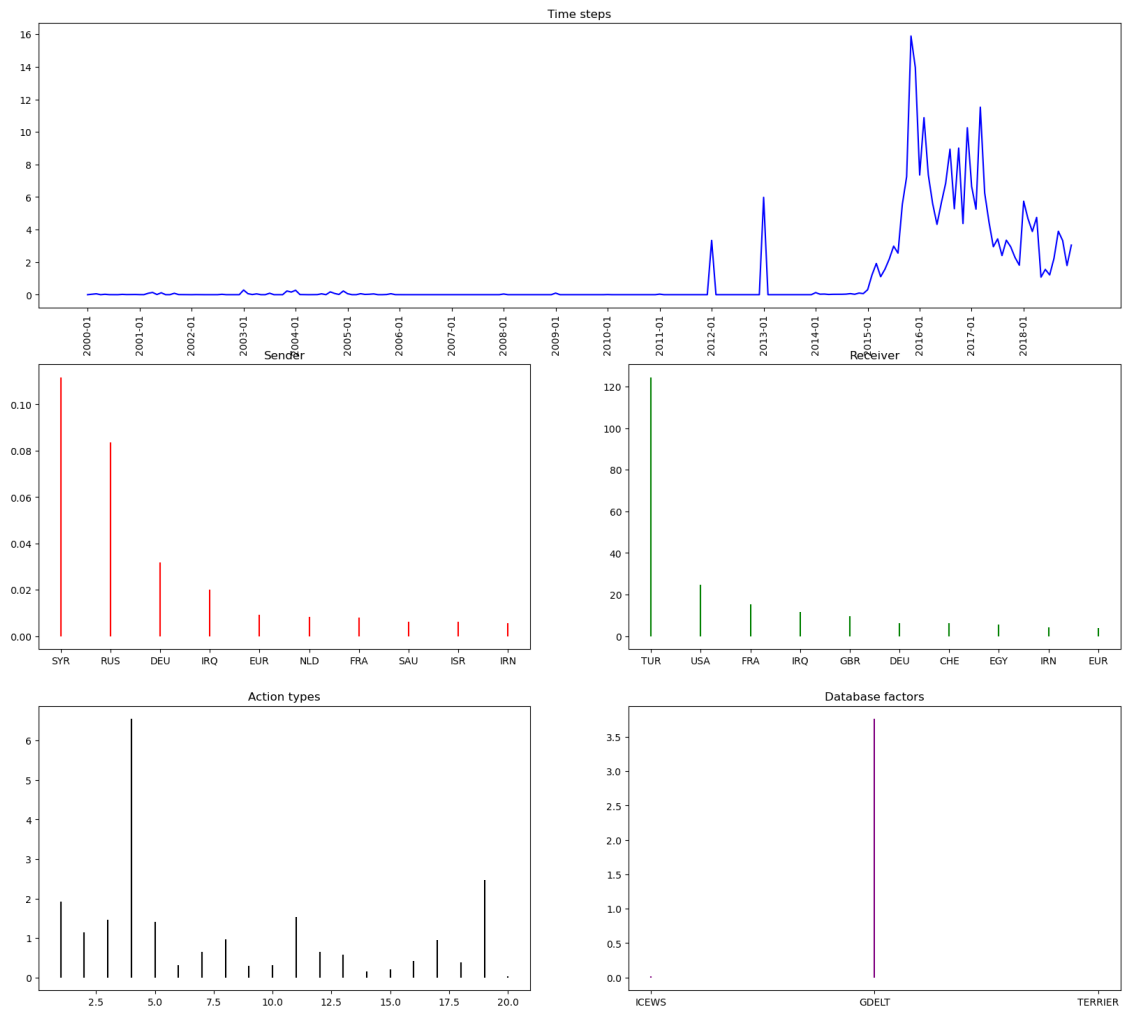




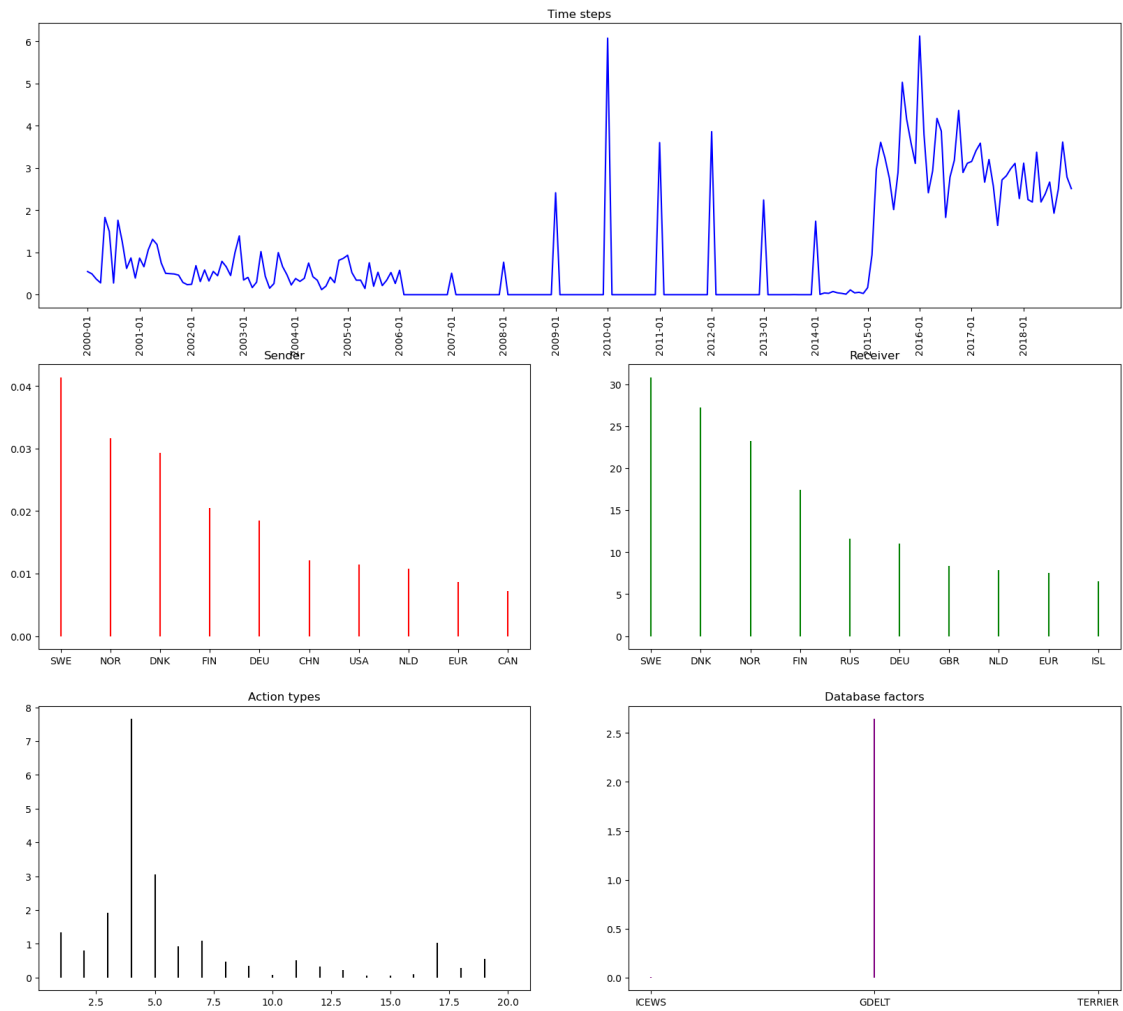
Component 48

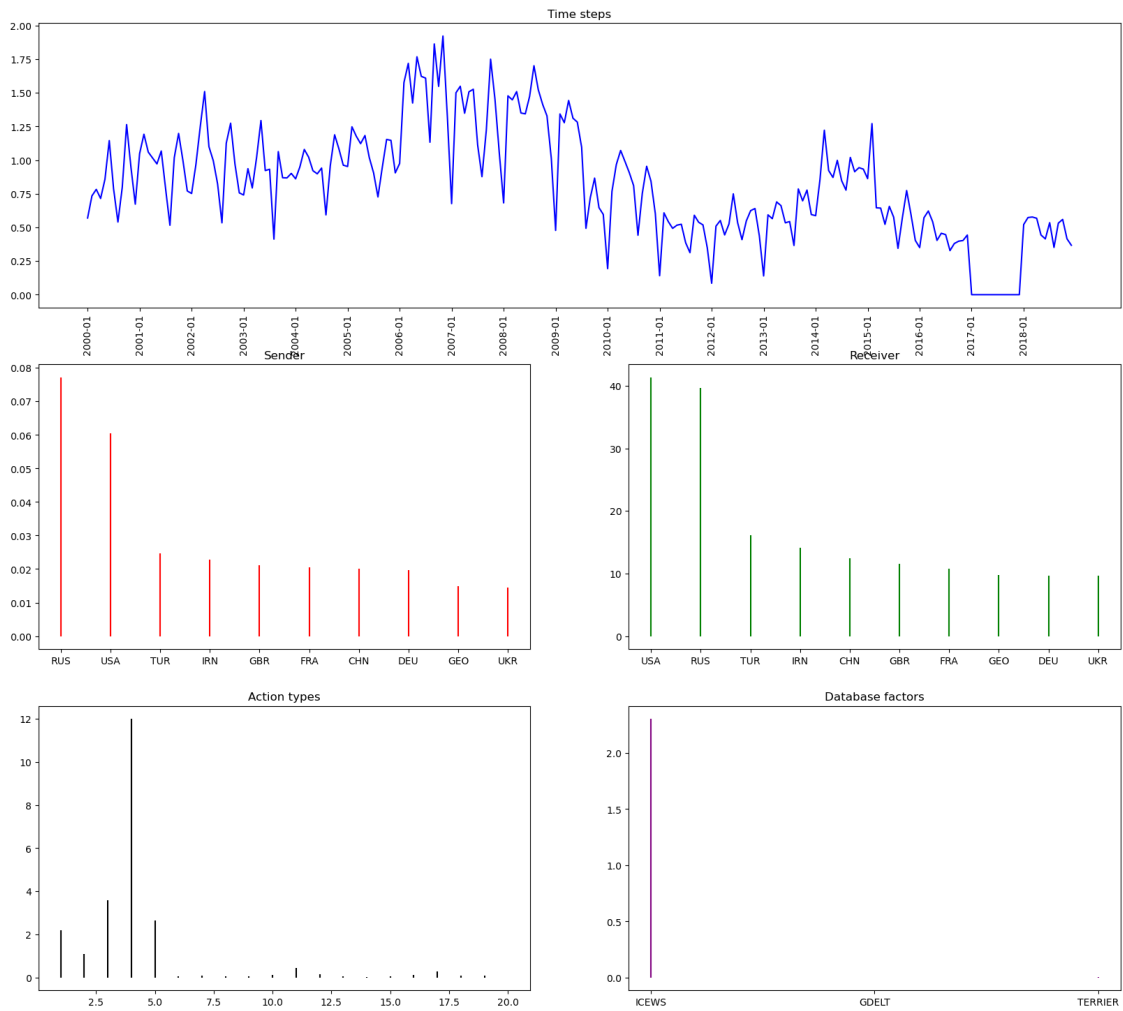


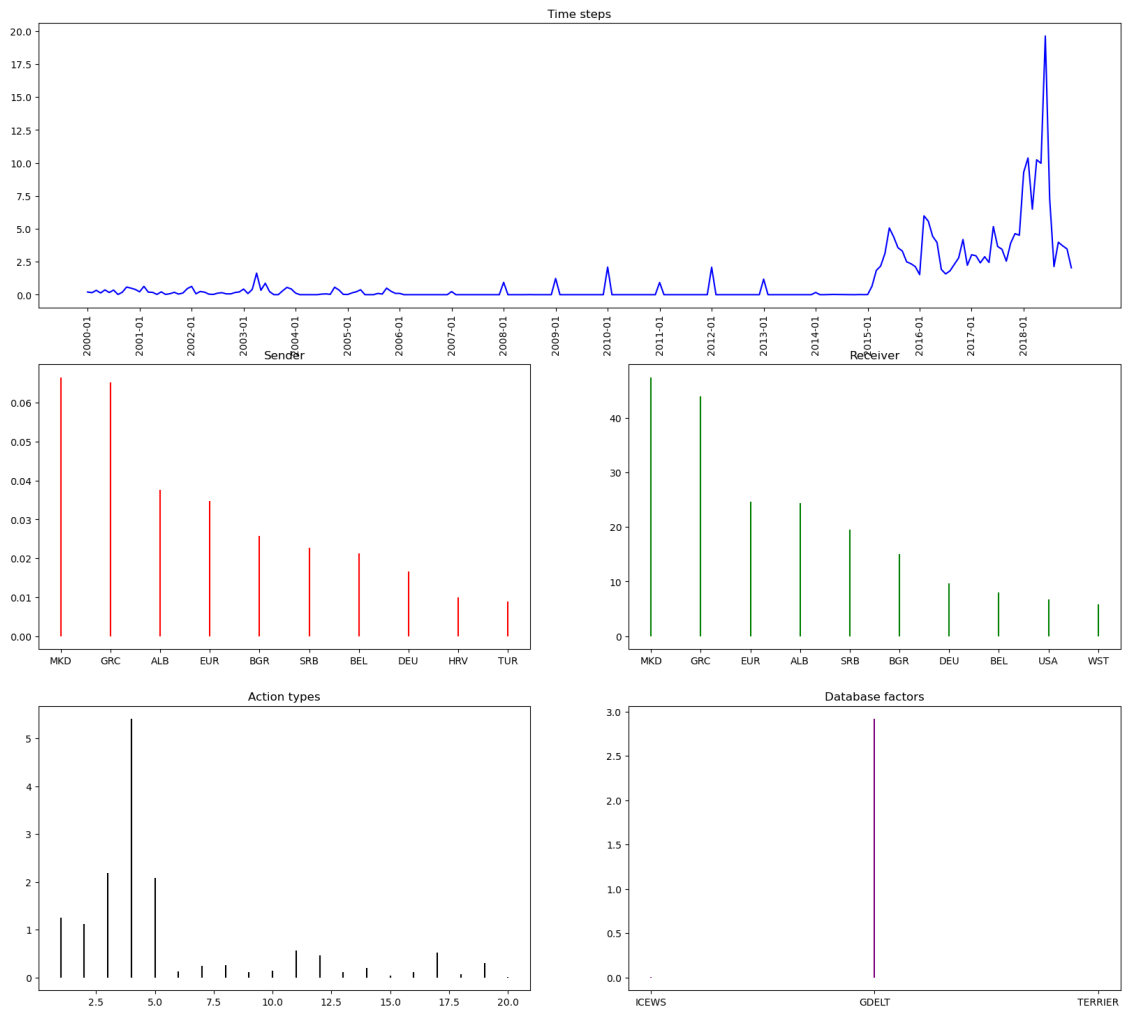




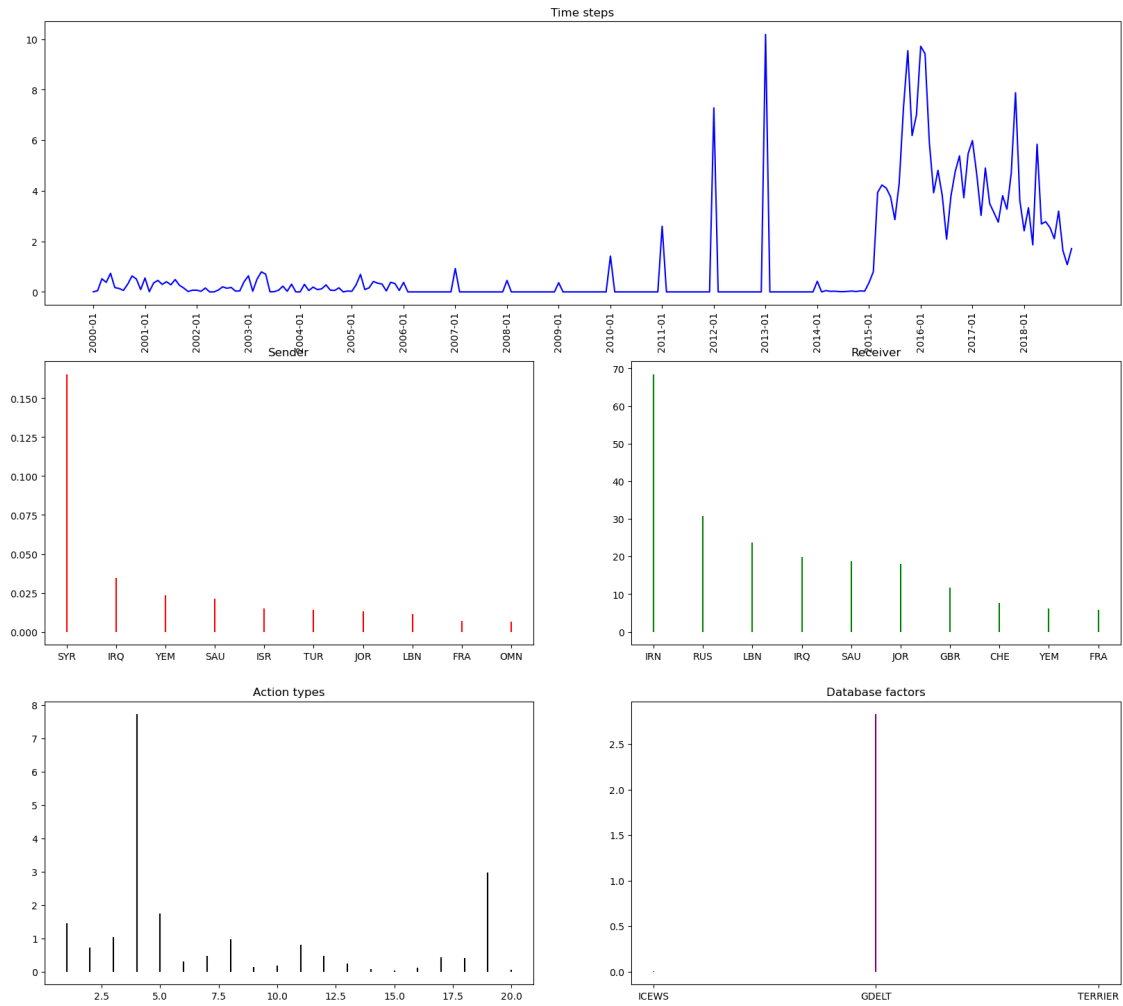
Component 45



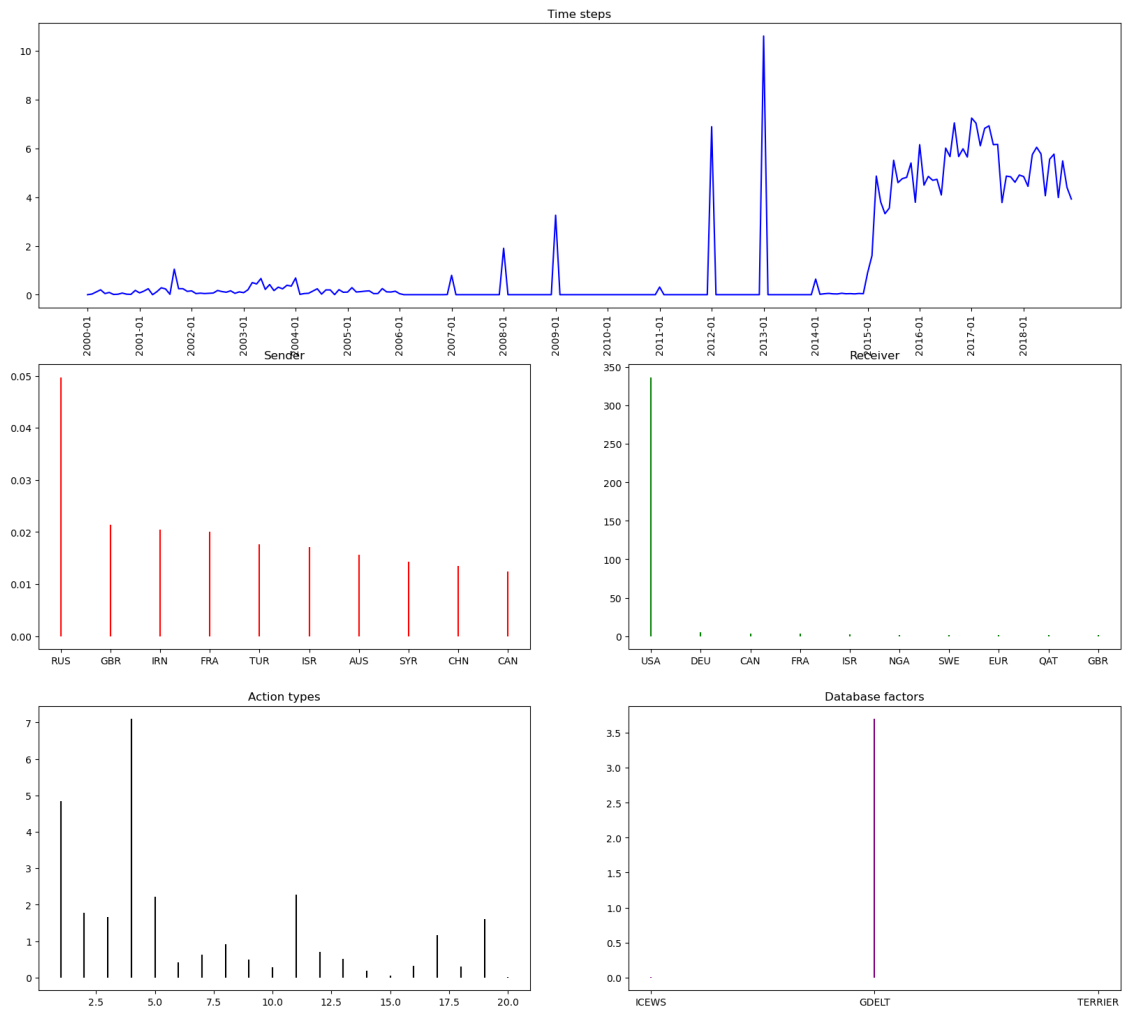


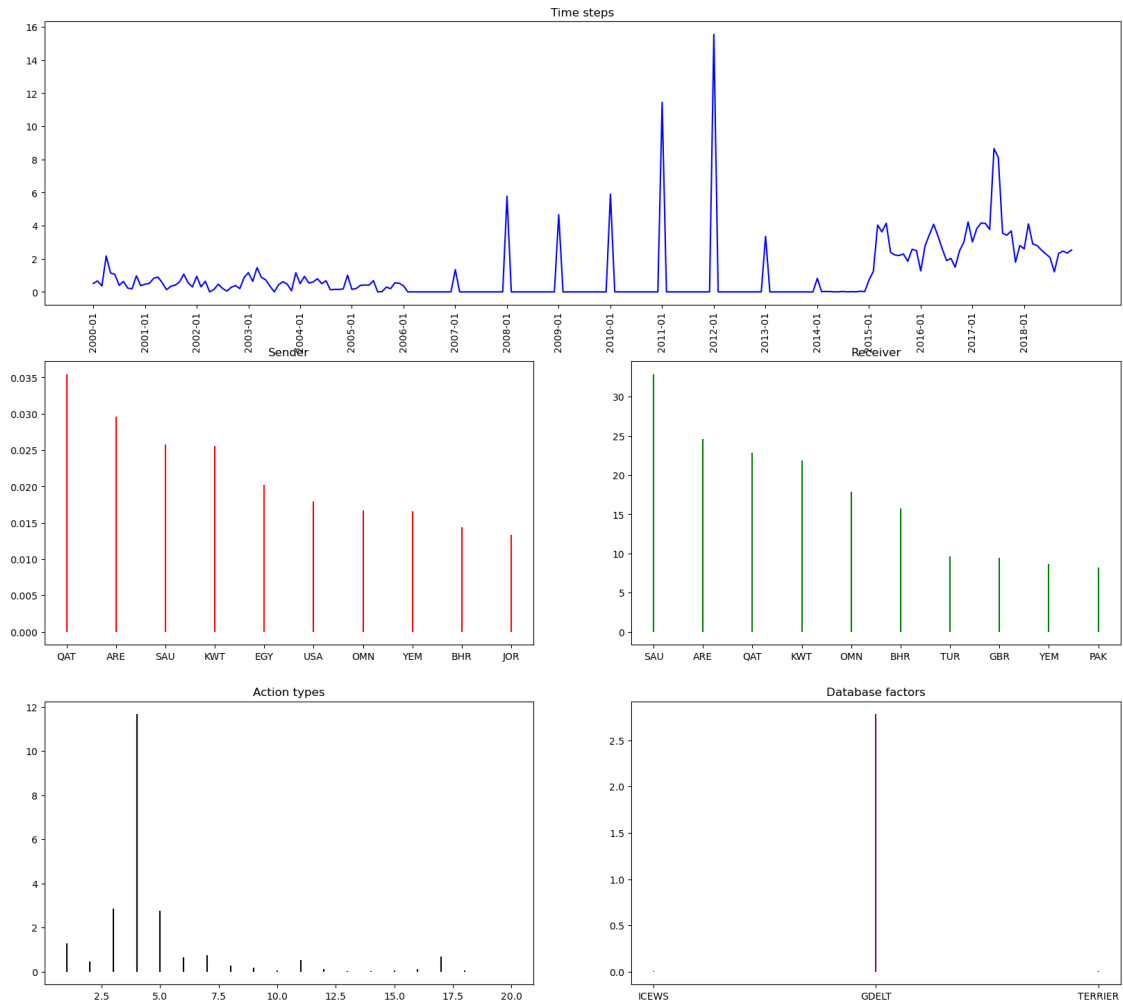


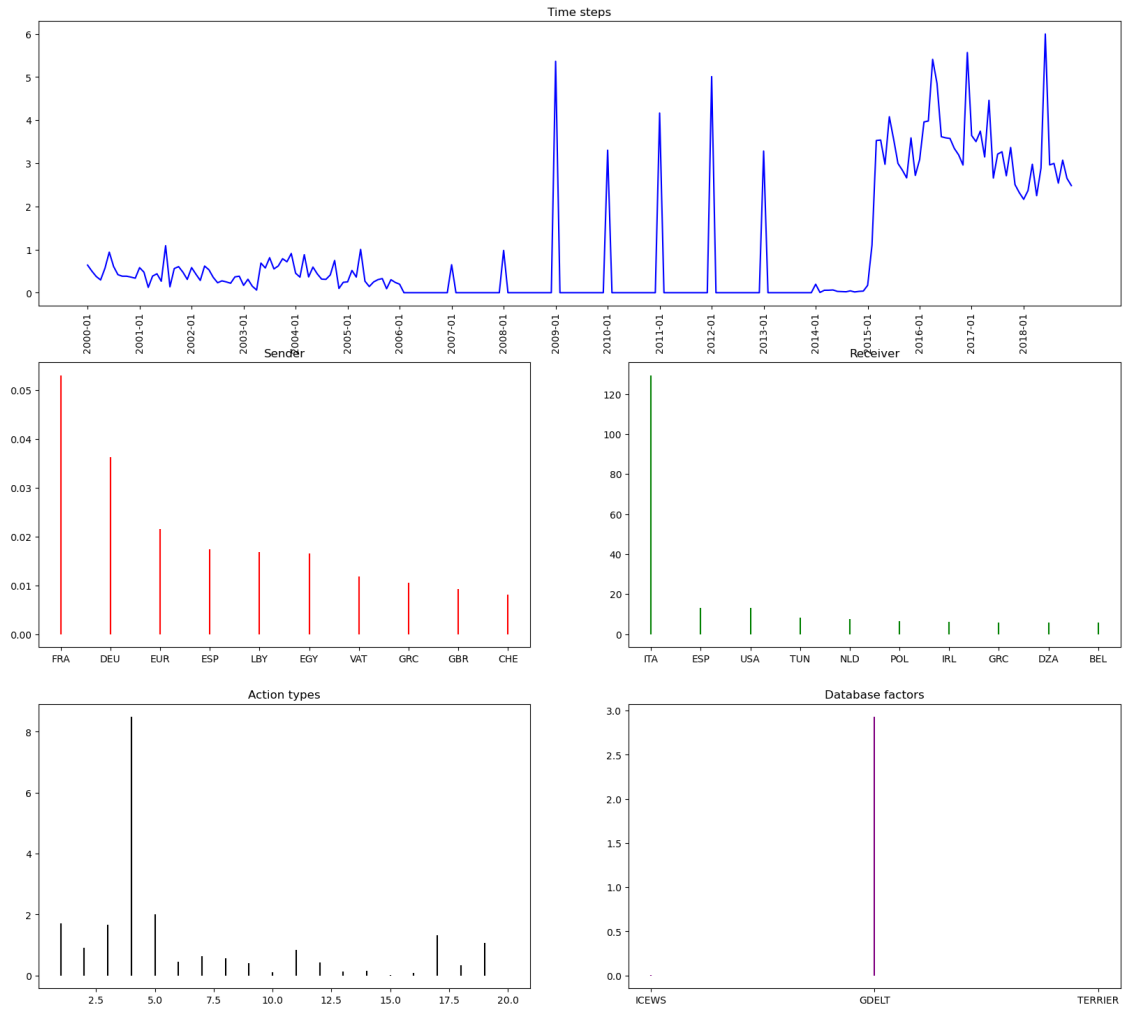
Component 10

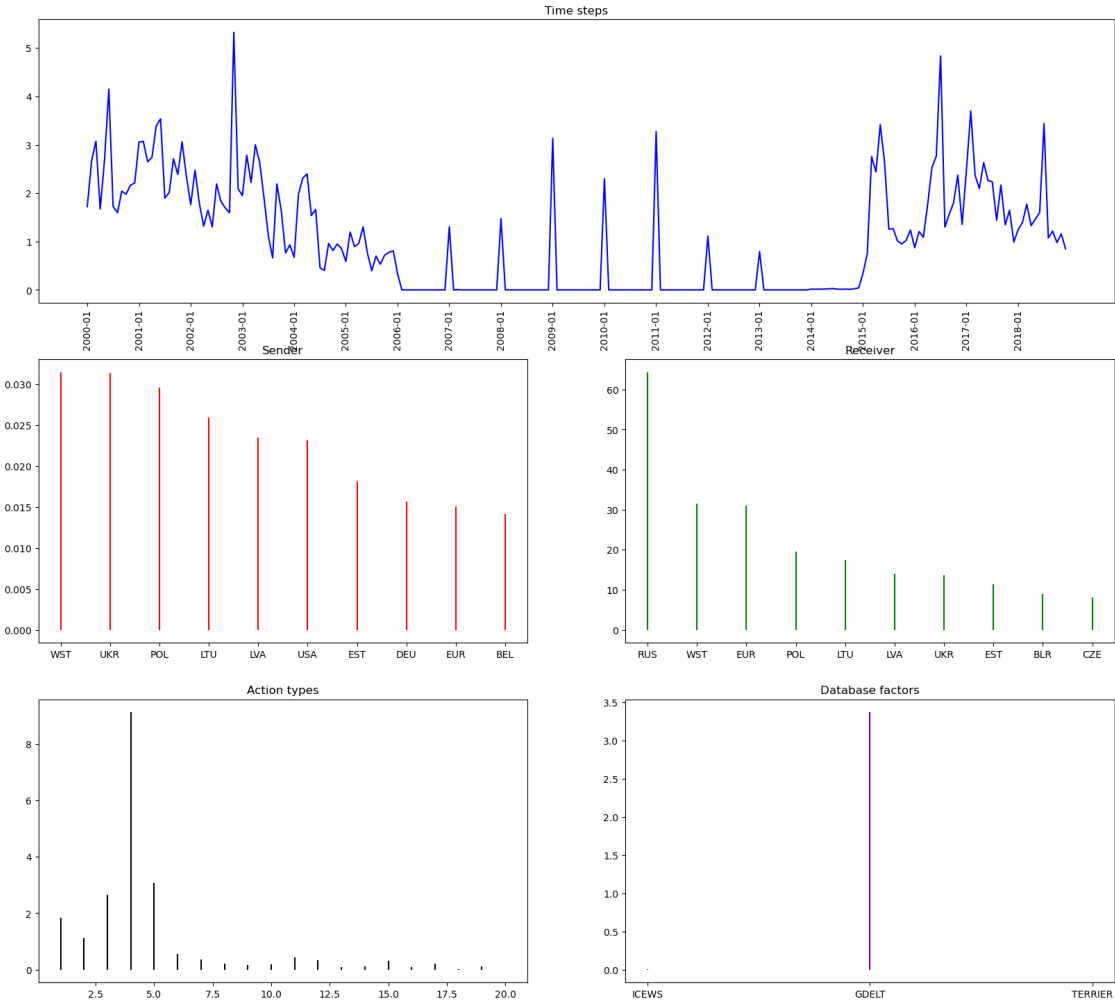


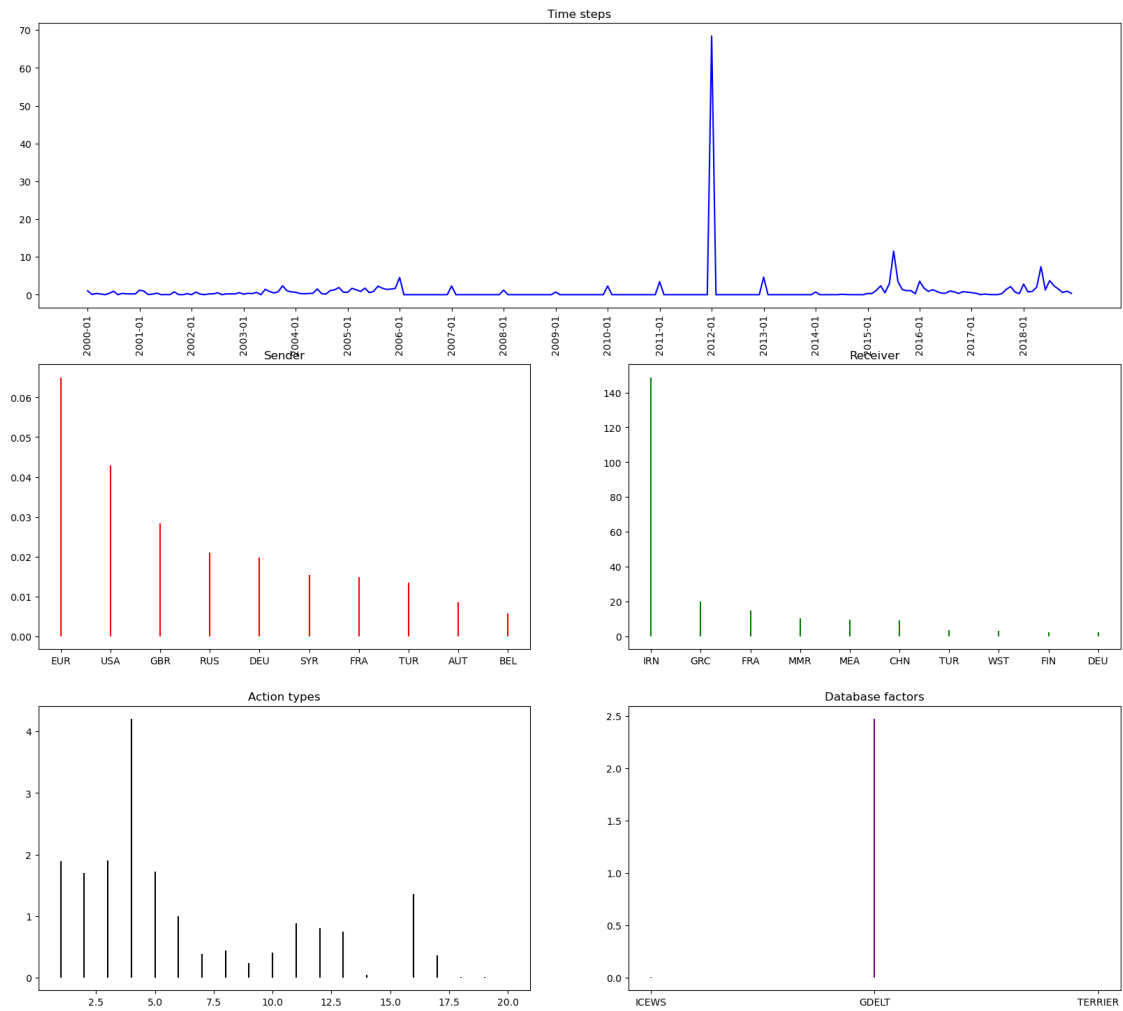
Component 18

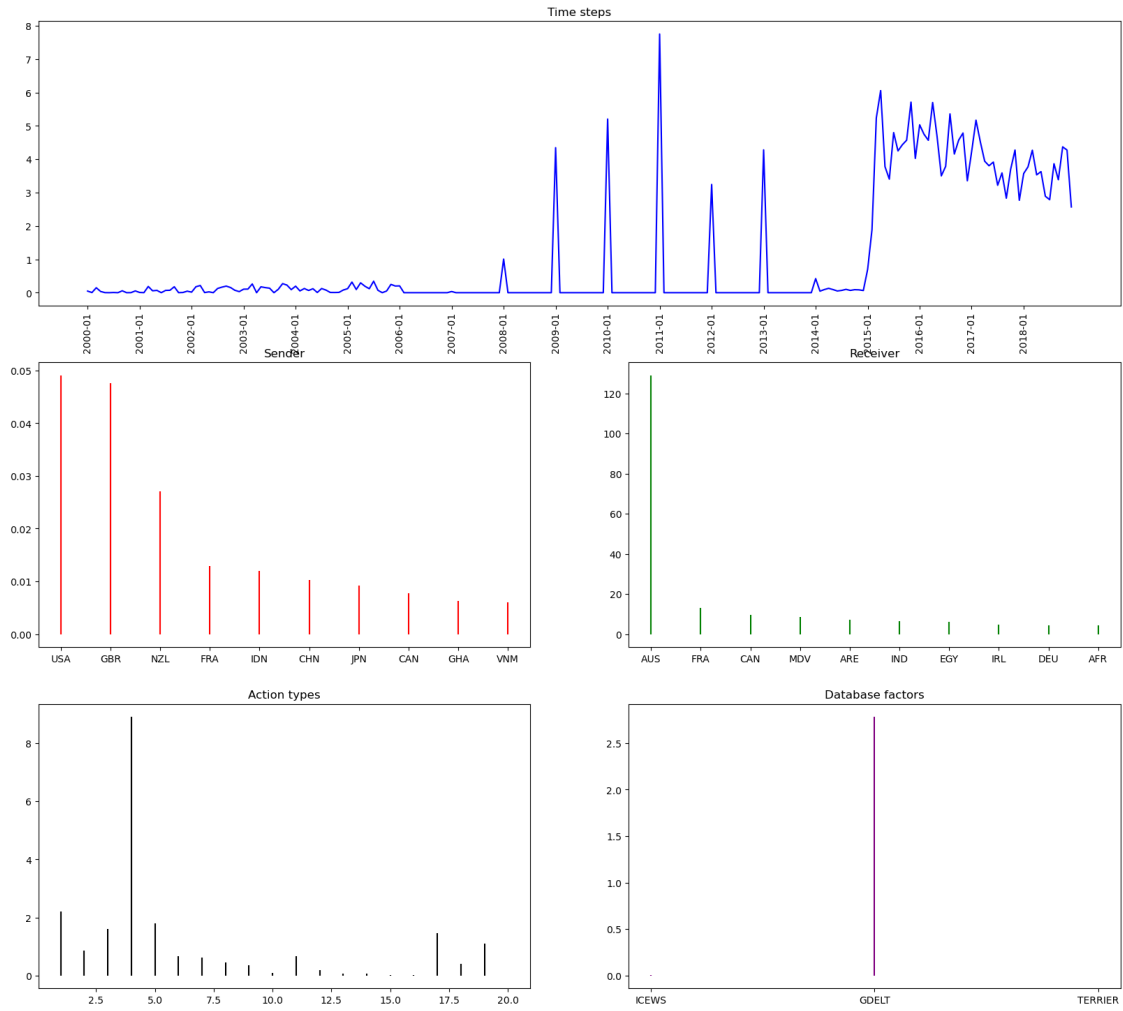




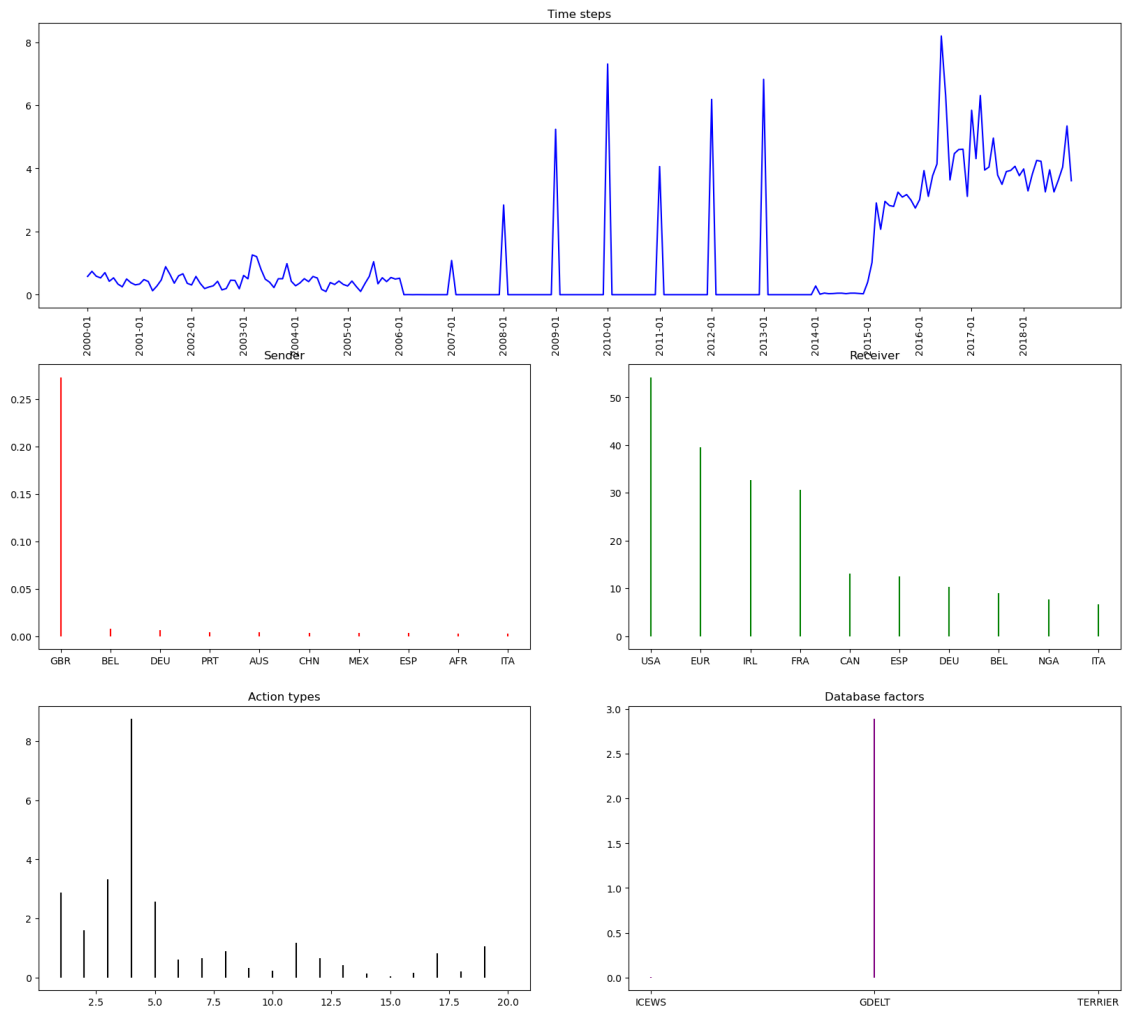


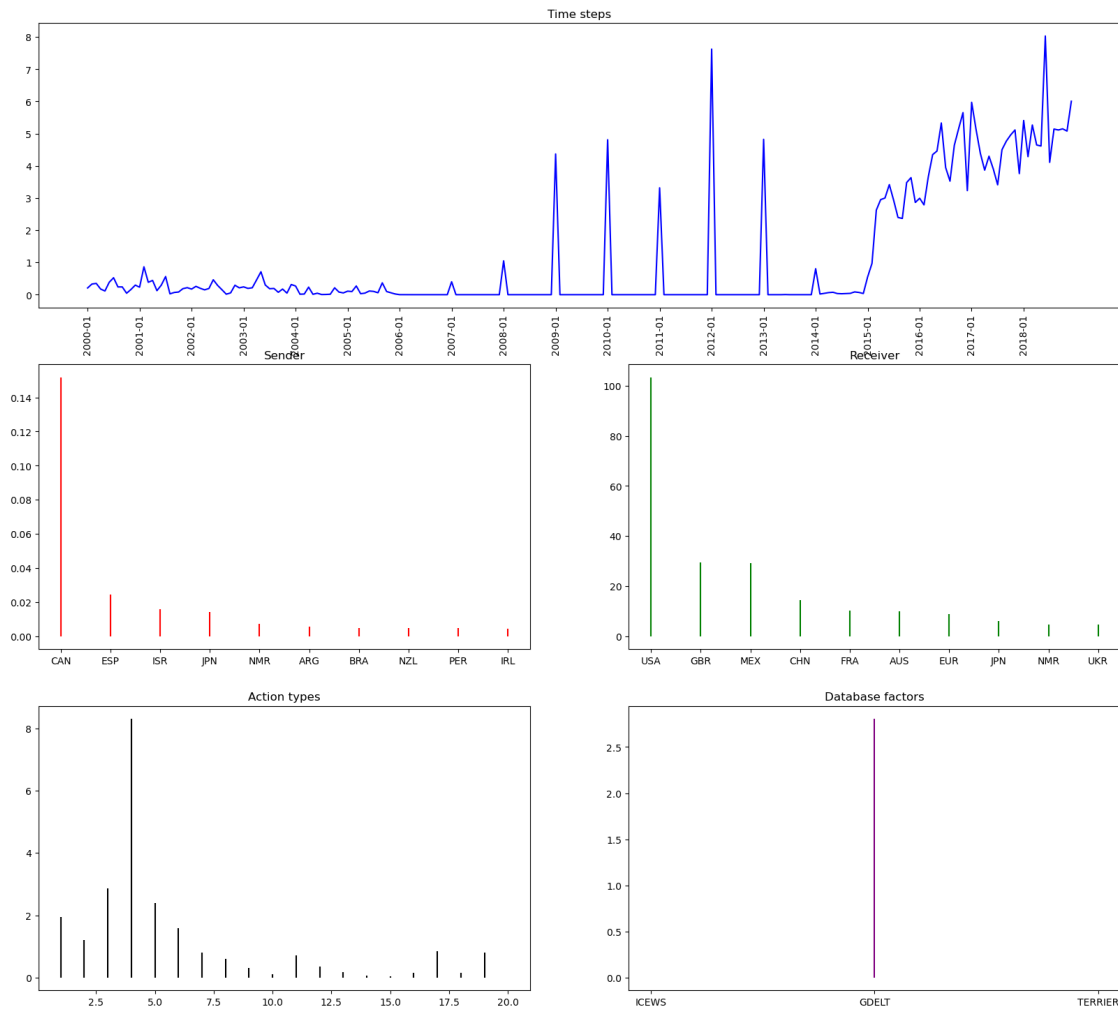




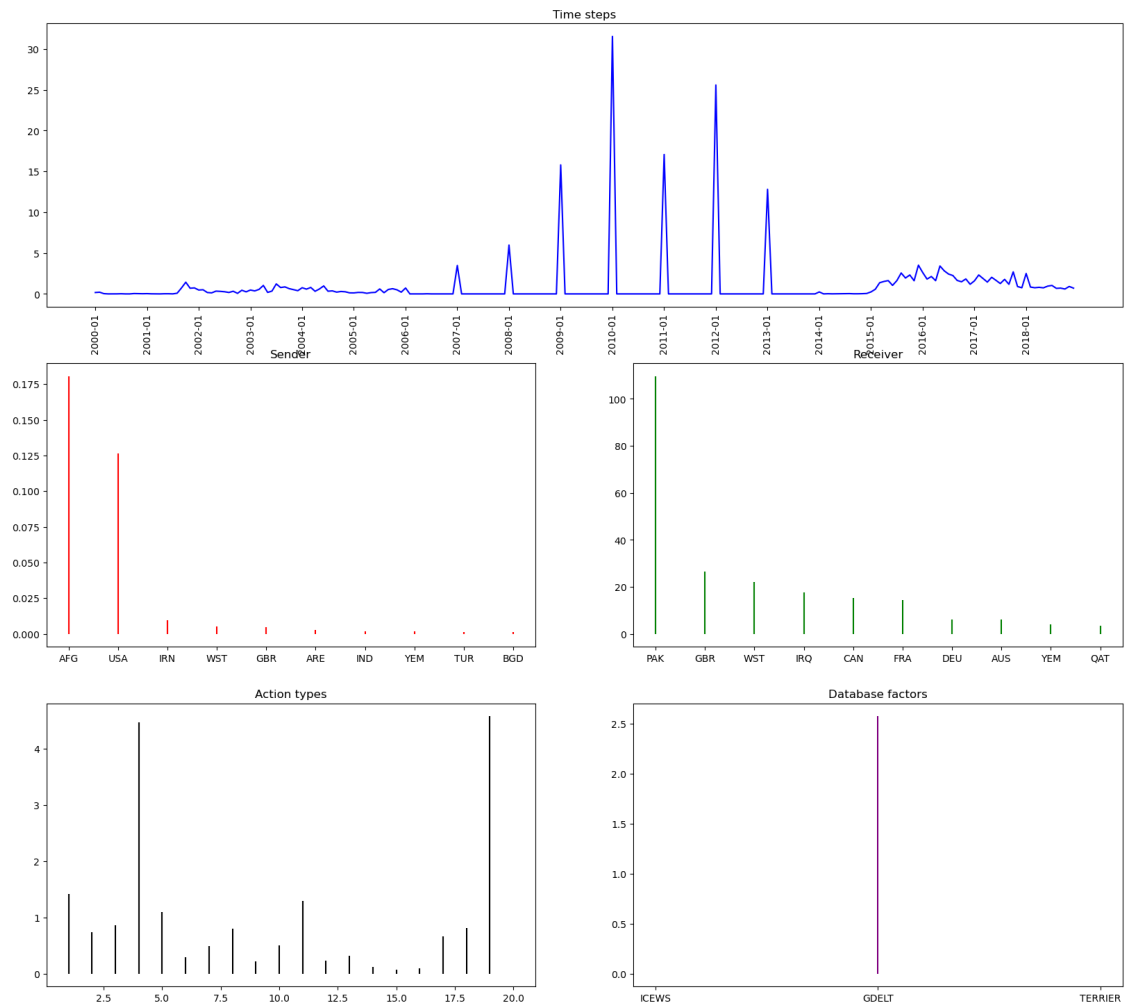


Component 68

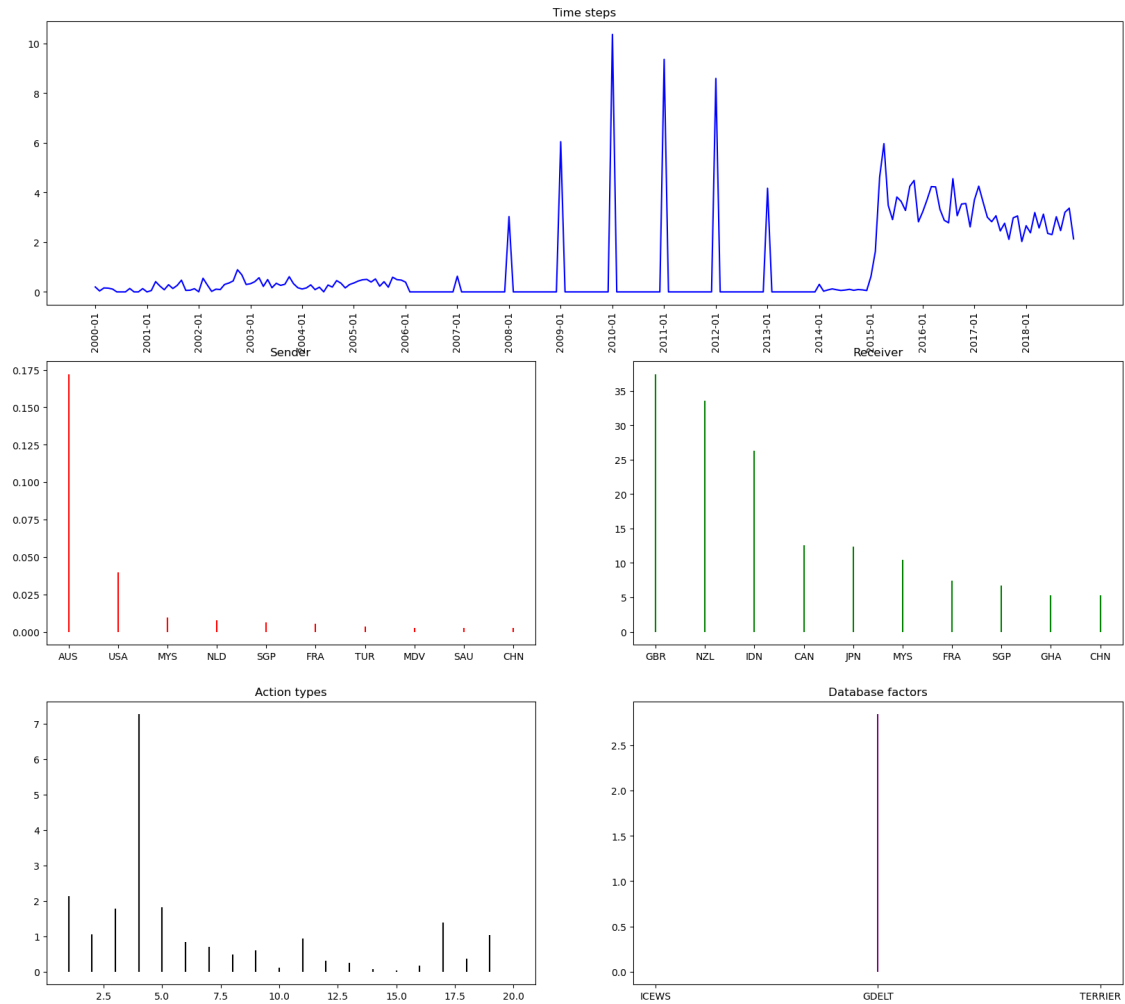




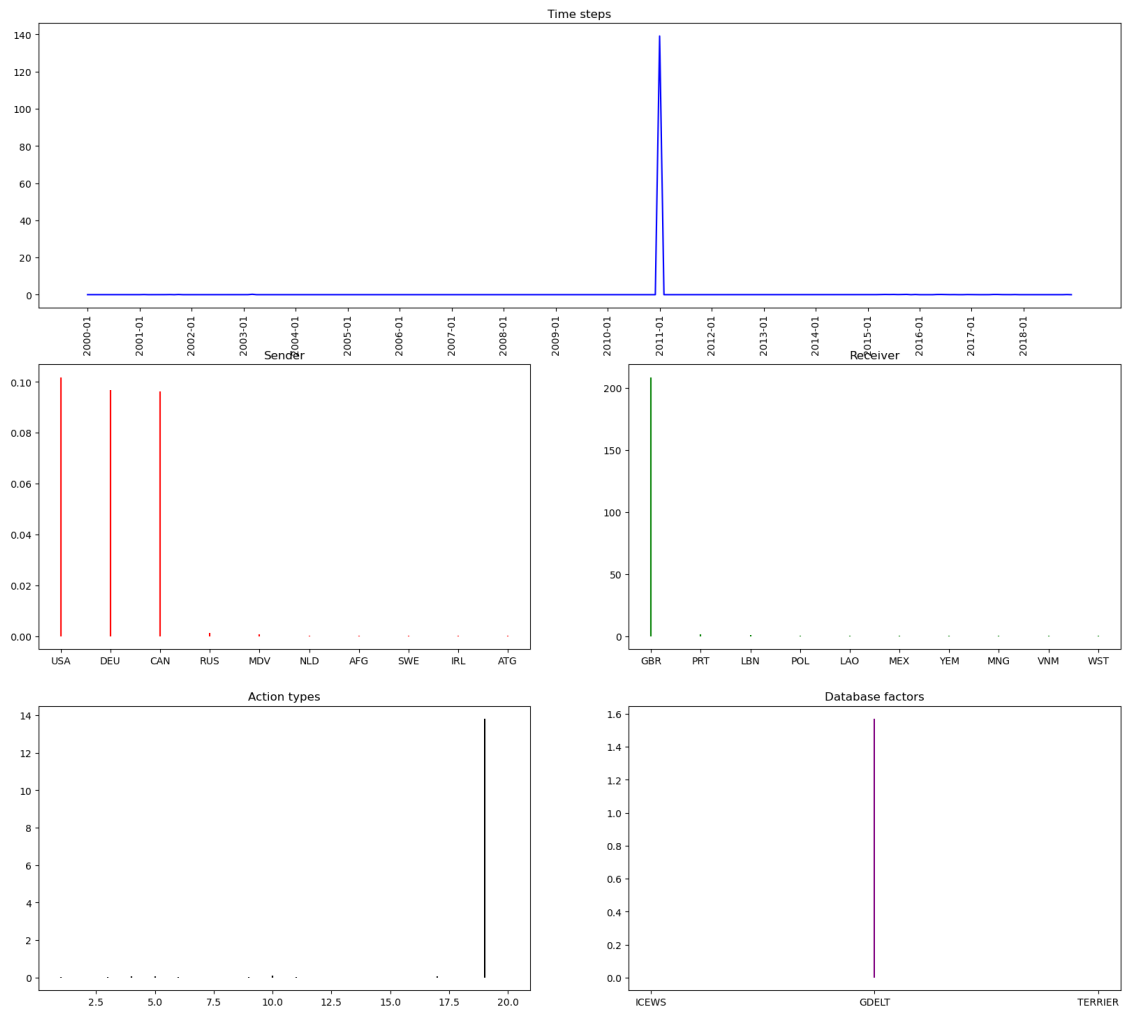
Component 15



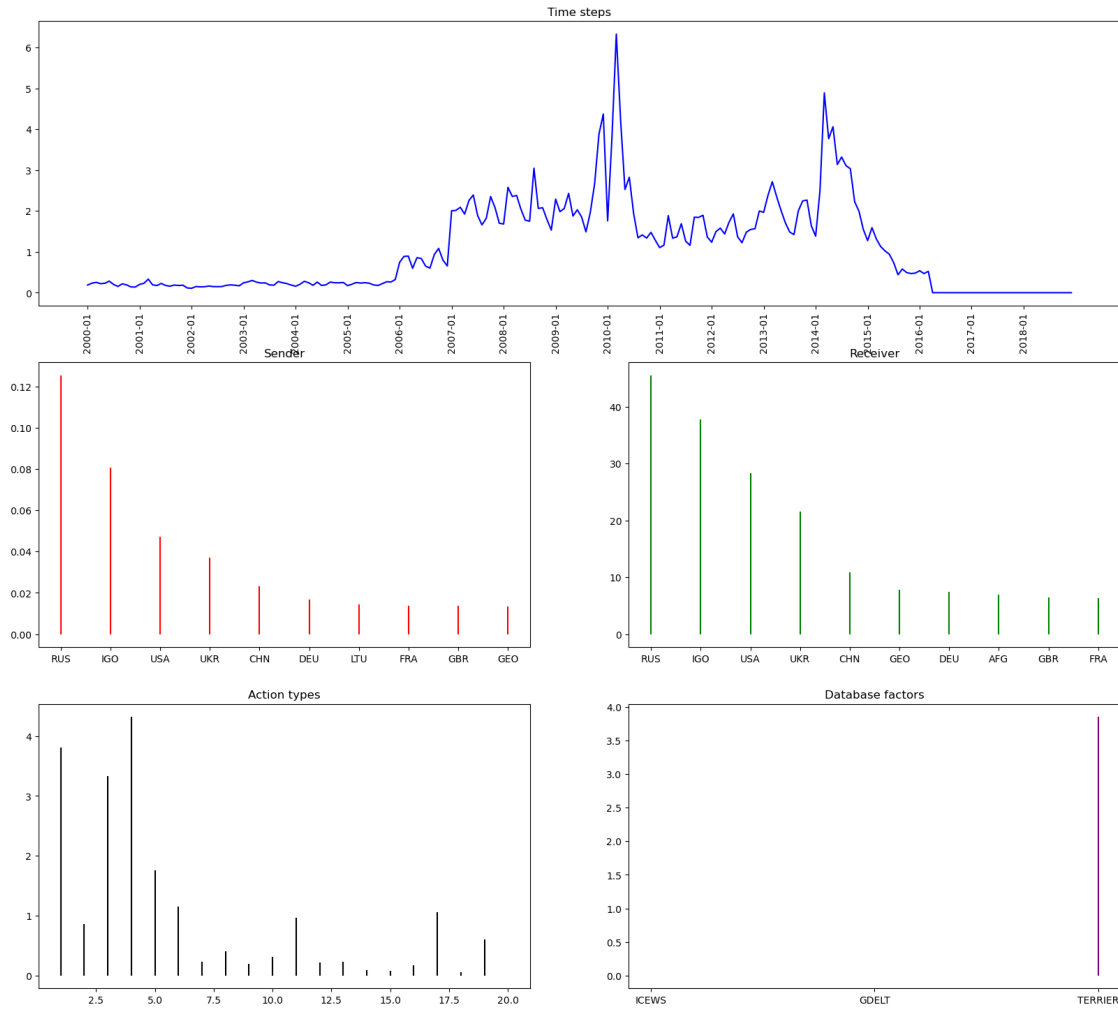
Component 61



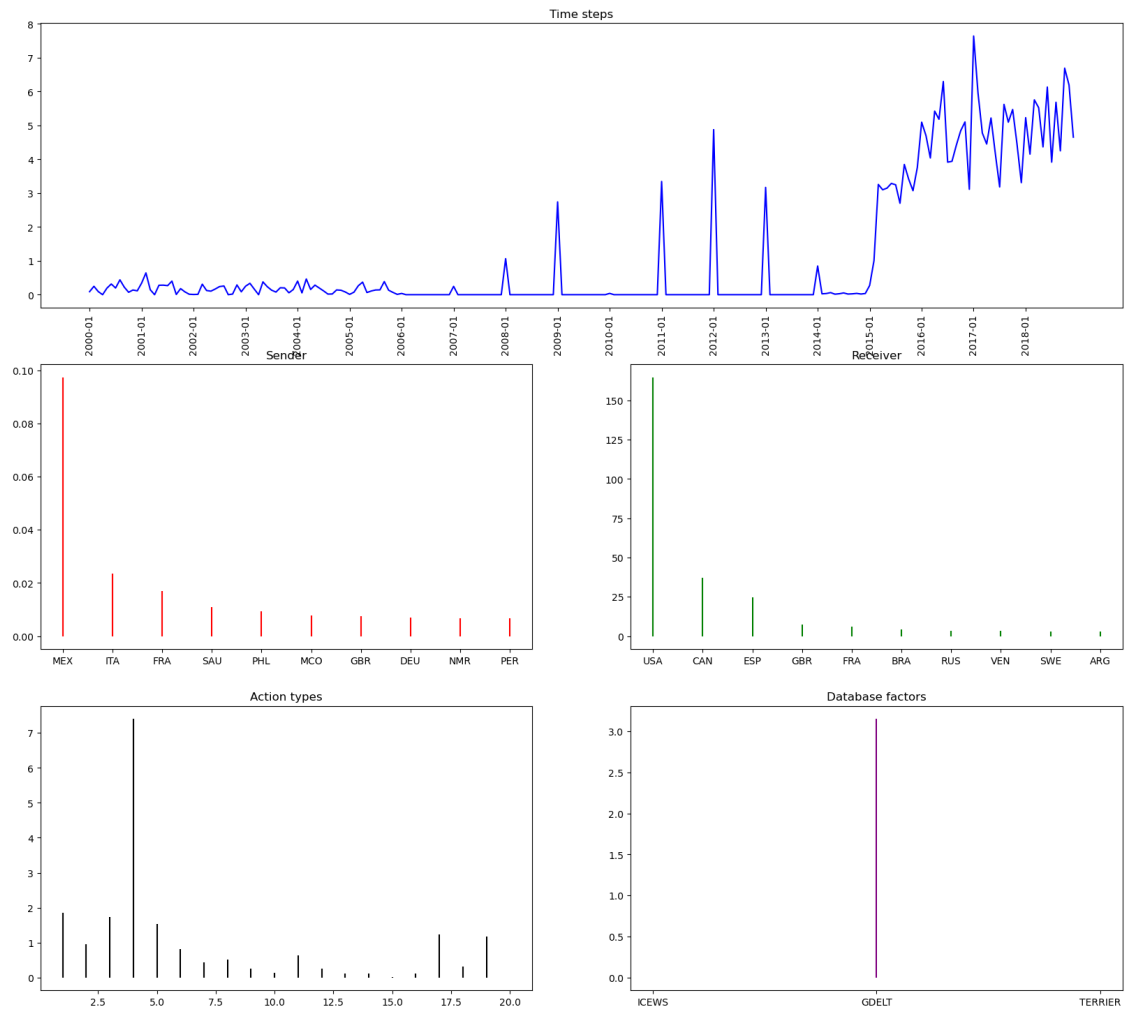
Component 51

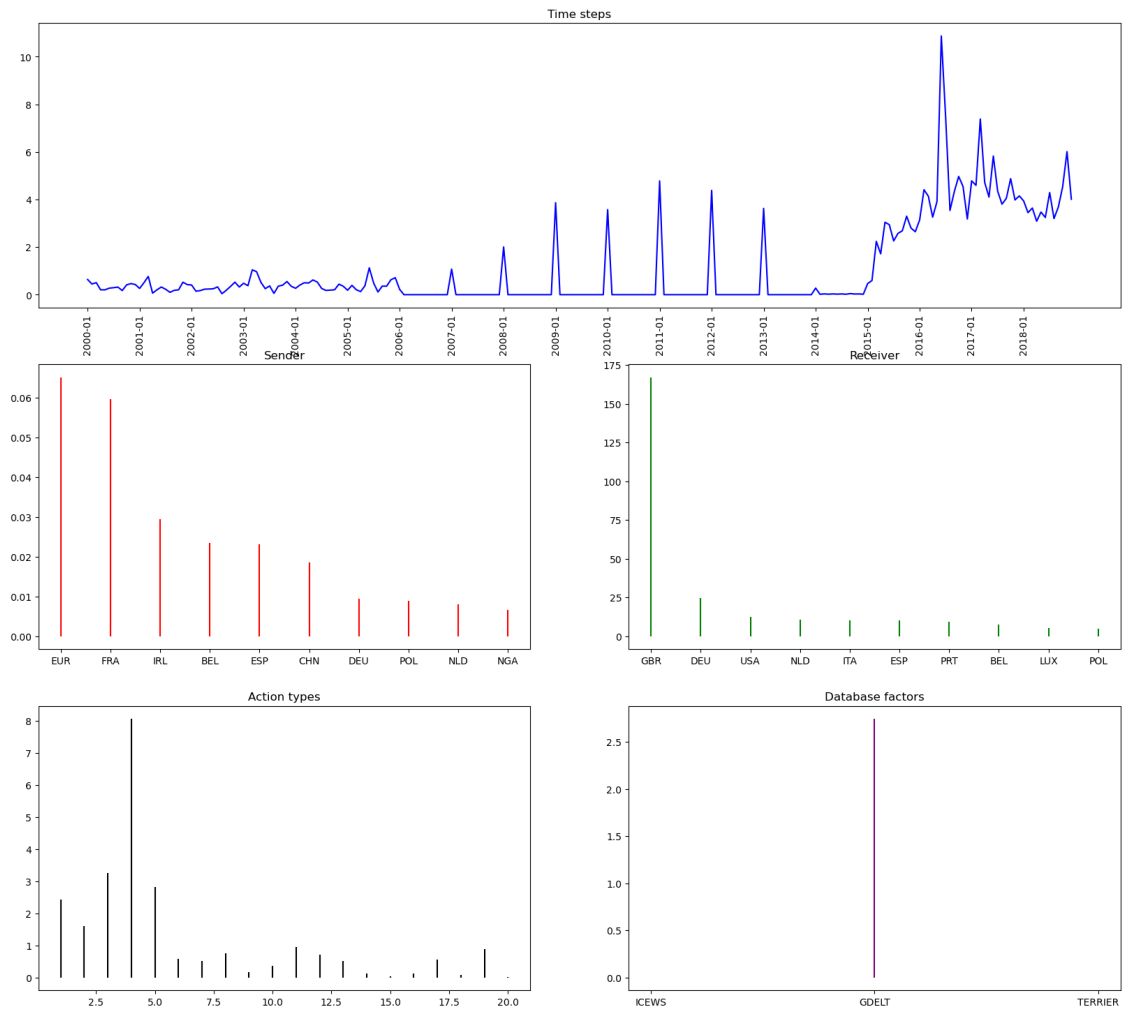


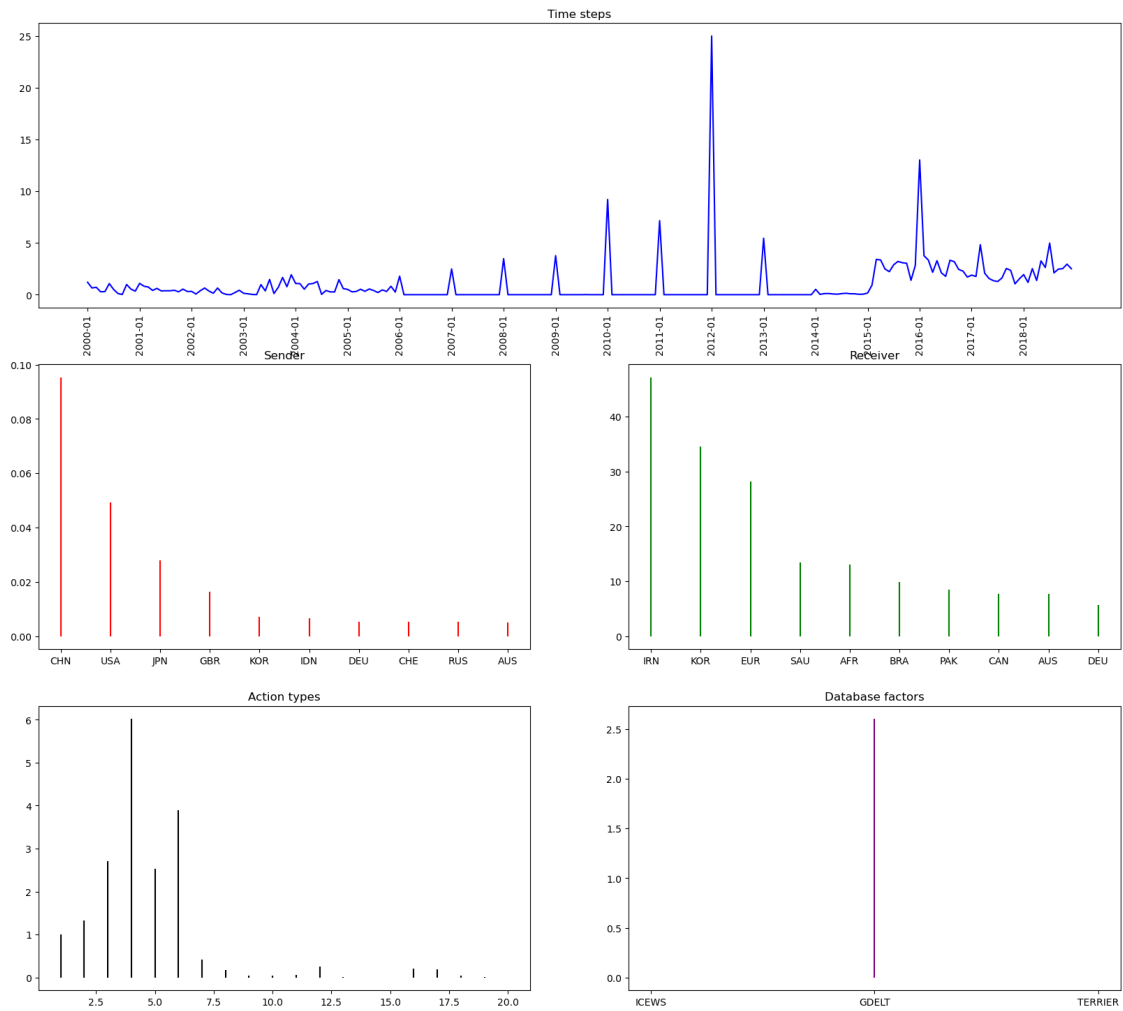
Component 19

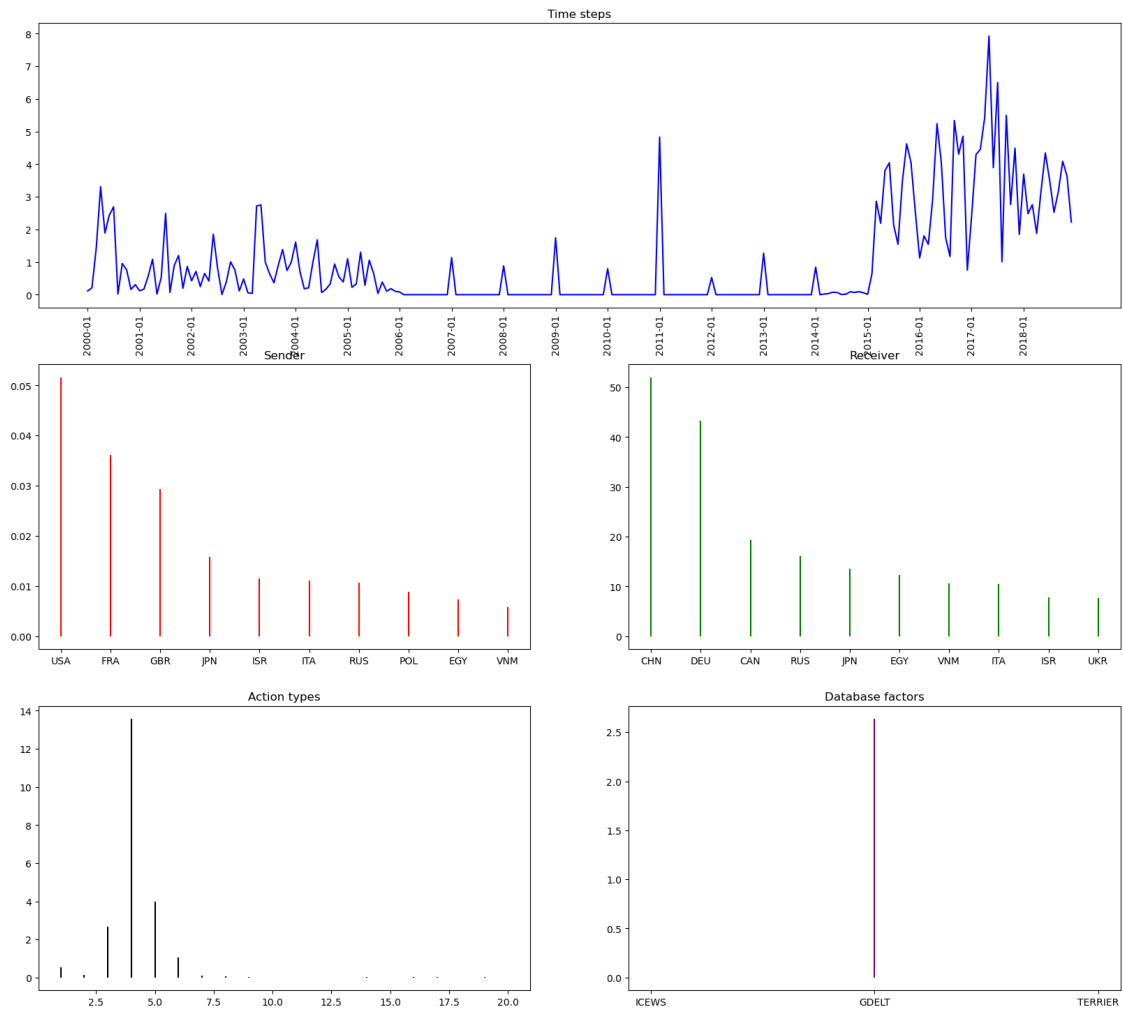


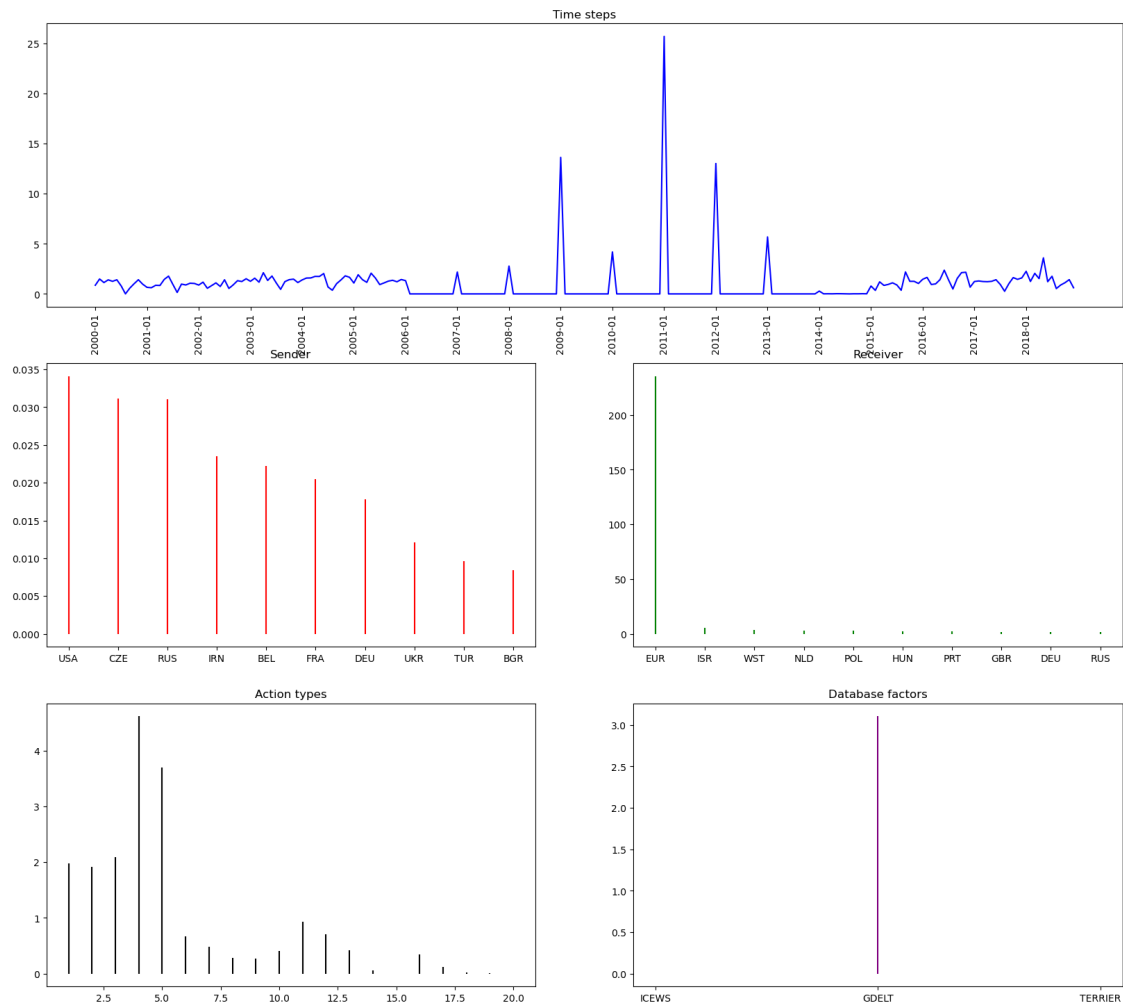
Component 7



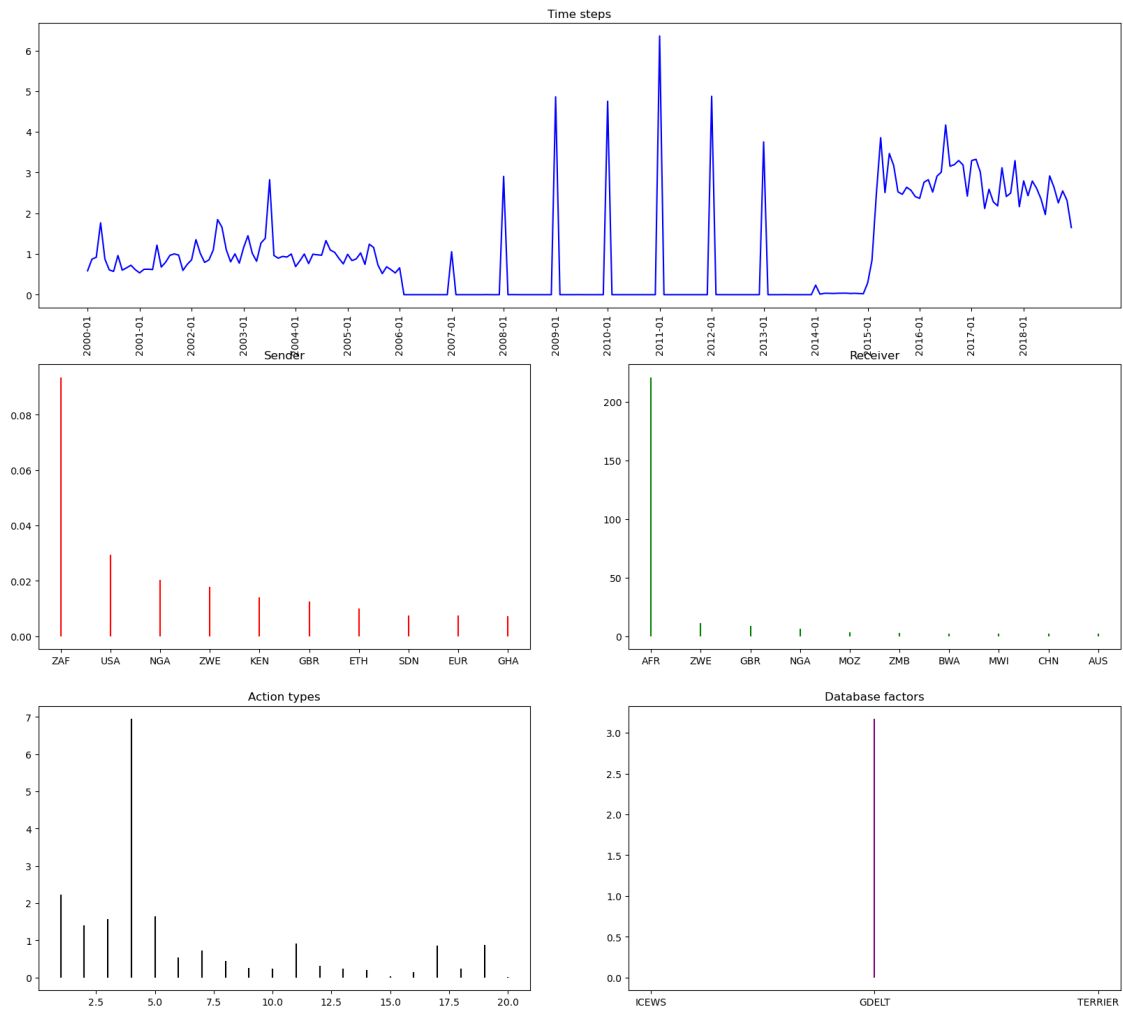




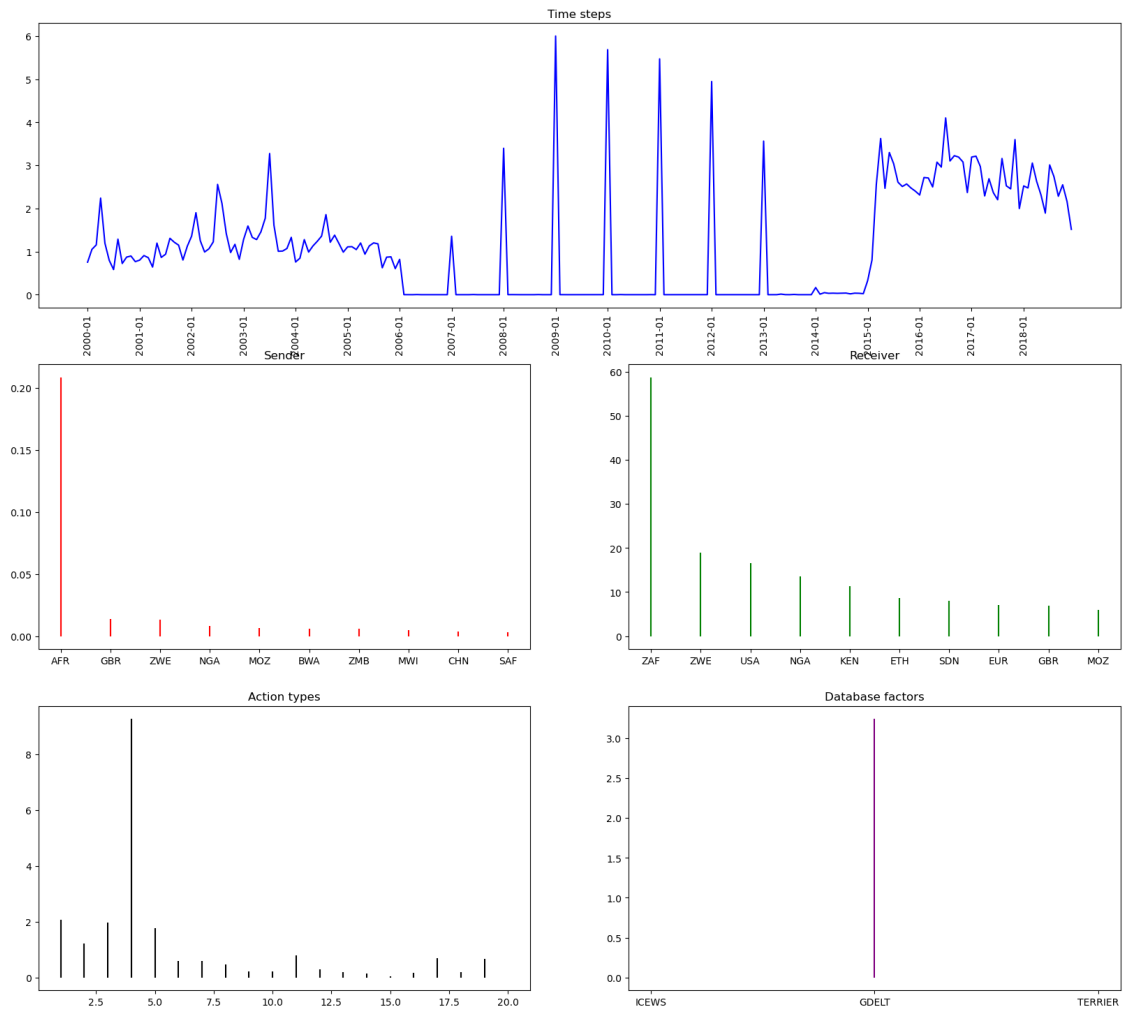




Component 30



Component 44



Component 8

